



SDK qb Compiler

Document Number: MCS002-KR
Version v1.0.0
Release Date: Feb. 02, 2026

Confidential. Please do not distribute.

면책 조항 및 고지사항

저작권

Copyright © 2019-2026, Mobilint, Inc. All rights reserved.

경고 (WARNING) 본 간행물의 어떠한 부분도 Mobilint, Inc.(이하 "MOBILINT")의 사전 서면 동의 없이 전자적·기계적 방식(복사, 녹음, 정보 저장 및 검색 시스템 포함) 또는 기타 어떠한 방법으로도 복제, 저장, 전송될 수 없습니다. 본 간행물은 MOBILINT가 지정한 수신자만을 위한 자료입니다. 본 간행물에는 「부정 경쟁방지법」, 「영업비밀보호법」 등 관련 법령에 따라 보호되는 MOBILINT의 기밀 정보(영업비밀 포함)가 포함되어 있습니다. 제3자에게 본 문서의 전부 또는 일부를 무단으로 접근, 공개, 배포, 복사, 재생산하거나 기타 방식으로 이용하는 행위는 엄격히 금지되며, 이는 불법 행위에 해당할 수 있습니다.

MOBILINT는 제3자의 무단 접근, 공개 또는 이용으로 인해 발생하는 MOBILINT의 영업비밀 및/또는 기밀 정보 침해에 대해, 관련 법령이 허용하는 범위 내에서 모든 법적 조치를 취할 권리를 보유하며, 이에 따른 손해에 대해 전면적인 배상을 청구할 수 있습니다.

중요 고지 (Important Notice)

MOBILINT는 사전 통지 없이 본 간행물에 포함된 정보의 일부 또는 전부를 변경할 권리를 보유합니다. 본 문서에 포함된 정보는 참고용으로 제공되며, MOBILINT는 정보의 오류, 누락 또는 본 정보를 사용함으로써 발생하는 어떠한 결과에 대해서도 책임을 지지 않습니다.

본 간행물은 MOBILINT 또는 제3자의 제품과 관련하여, 명시적 또는 묵시적으로 어떠한 라이선스도 부여하지 않으며, MOBILINT 및/또는 제3자의 지식재산권에 대한 권리를 이전하거나 허용하는 것으로 해석되어서는 안 됩니다.

MOBILINT는 특정 목적에 대한 제품의 적합성에 대해 어떠한 보증, 진술 또는 보장을 제공하지 않으며, 제품 또는 회로의 적용이나 사용으로 인해 발생하는 모든 책임을 부인합니다. 여기에는 결과적 손해 또는 부수적 손해에 대한 책임도 포함되며, 이에 국한되지 않습니다.

고객은 자사 제품 및 적용에 대한 모든 책임을 부담합니다. "일반적인(Typical)" 파라미터는 적용 환경에 따라 달라질 수 있으며, 모든 동작 파라미터("Typical" 값 포함)는 고객의 기술 전문가에 의해 각 적용 사례별로 검증되어야 합니다.

MOBILINT 제품은 인명 유지 시스템(life support system)을 지원하거나, 제품의 고장이 개인의 상해 또는 사망을 초래할 수 있는 응용 분야에서 사용하도록 설계·의도·허가되지 않았습니다. 고객은 MOBILINT 제품을 사용하는 응용과 관련된 모든 법적·규제적 요구 사항을 충족할 전적인 책임이 본인에게 있음을 인정하고 이에 동의합니다.

고객은 본 간행물에 제공된 정보의 사용 또는 MOBILINT 제품의 사용과 관련하여 발생하는 모든 청구, 비용, 손해, 경비, 책임 및 합리적인 변호사 비용 등에 대해 MOBILINT를 면책하고 보호하며, 이에 대한 책임을 부담합니다. 이는 의도하지 않았거나, 승인되지 않았거나, 불법적인 사용으로 인해 직접적 또는 간접적으로 발생하는 인적 상해 또는 사망을 포함하되 이에 한정되지 않습니다.

1 소개	1
2 설치	2
2.1 시스템 요구사항	2
2.1.1 기준 환경	2
2.1.2 패키지 요구사항 및 권장사항	2
2.2 SDK 설치	2
2.2.1 도커 컨테이너 빌드	3
2.2.2 qbcompiler 설치	3
3 빠른 시작	4
3.1 Calibration Data 준비	4
3.1.1 전처리 구성 YAML 파일을 활용한 방법	4
3.1.2 사용자 정의 전처리 함수를 사용하는 방법	5
3.2 딥러닝 모델 컴파일	6
3.2.1 ONNX 모델 컴파일	6
3.2.2 PyTorch 모델 컴파일	6
3.2.3 Torchscript 모델 컴파일	7
3.2.4 TensorFlow/Keras 모델 컴파일	8
3.2.5 TensorFlow Lite 모델 컴파일	8
3.3 (선택 사항) 양자화 방법 및 매개변수 설정	9
4 튜토리얼 개요	10
5 TorchVision의 이미지 분류 모델	11
5.1 소개	11
5.2 모델 준비	11
5.3 Calibration Dataset 준비	11
5.4 모델 컴파일	13
6 Ultralytics의 YOLO 모델	15
6.1 소개	15
6.2 모델 준비	15
6.3 Calibration Dataset 준비	16
6.4 모델 컴파일	16
7 HuggingFace로 배포된 Llama 모델	18
7.1 소개	18
7.2 모델 준비	18
7.2.1 HuggingFace 인증	18
7.2.2 모델 다운로드	20
7.3 Calibration Dataset 준비	20

7.3.1 Embedding Weight 추출	21
7.3.2 Calibration Dataset 생성	21
7.4 모델 컴파일	23
8 지원 프레임워크	25
8.1 Mobilit IR Operations List	25
9 CPU Offloading (Beta Version)	41
10 Changelog	42
10.1 [v1.0.0] - 2026-01-30	42
10.1.1 API	42
10.2 [v0.12.0.0] - 2026-01-02	42
10.2.1 API	42
10.3 [v0.11.0.0] - 2025-09-10	42
10.3.1 API	42
10.4 [v0.10.0.0] - 2025-07-24	43
10.4.1 API	43
10.5 [v0.9.0.5] - 2025-06-25	43
10.5.1 API	43
10.6 [v0.9.0.4] - 2025-05-22	43
10.6.1 API	43
10.7 [v0.9.0.3] - 2025-04-02	43
10.7.1 API	43
10.8 [v0.9.0.2] - 2025-01-16	43
10.8.1 API	43
10.9 [v0.9.0.1] - 2024-12-13	44
10.9.1 API	44
10.10 [v0.9.0.0] - 2024-11-27	44
10.10.1 API	44
10.10.2 Docker	44
10.11 [v0.8.5] - 2024-06-20	44
10.11.1 API	44
10.12 [v0.8.4] - 2024-05-20	44
10.12.1 API	44
10.12.2 Docker	44
10.13 [v0.8.3] - 2024-03-07	45
10.13.1 API	45
10.14 [v0.8.2] - 2024-02-23	45
10.15 [v0.8.1] - 2023-12-08	45
10.16 [v0.8.0] - 2023-11-02	45

10.16.1 API	45
10.17 [v0.7.12] - 2023-09-12	45
10.18 [v0.7.11] - 2023-08-31	45
10.18.1 API	45
10.19 [v0.7.10] - 2023-08-11	45
10.20 [v0.7.9] - 2023-08-11	45
10.21 [v0.7.8] - 2023-08-08	45
10.22 [v0.7] - 2023-03-23	45
10.22.1 API	46
10.23 [v0.6] - 2022-08-10	46
10.24 [v0.5] - 2022-07-01	46
10.24.1 Docker	46
10.24.2 Parser	46
10.24.3 API	46
10.25 [v0.4] - 2022-02-23	46
10.25.1 Optimizer	46
10.26 [v0.3] - 2022-02-05	47
10.26.1 Parser	47
10.26.2 API	47
10.27 [v0.2] - 2021-12-01	47
11 Open Source License Notice	48
12 Topic Index	49
12.1 API References	49
13 Hierarchical Index	50
13.1 Class Hierarchy	50
14 Class Index	51
14.1 Class List	51
15 Topic Documentation	53
15.1 API Reference	53
15.1.1 Detailed Description	54
15.1.2 Function Documentation	54
15.1.2.1 mxq_compile()	54
15.1.2.2 mbIt_compile()	58
15.1.2.3 run_mblt_model()	59
15.1.2.4 get_yaml()	59
15.1.2.5 make_calib_man()	60
15.1.2.6 make_calib()	61

15.1.2.7	<code>make_calib_llm()</code>	62
15.1.2.8	<code>preprocess_fnc()</code>	62
15.1.3	Variable Documentation	62
15.1.3.1	<code>DTYPE_GETDATA_MAP</code>	62
15.2	Compilation Configuration	62
15.2.1	Detailed Description	63
15.2.2	Function Documentation	63
15.2.2.1	<code>get_compile_config()</code>	63
15.3	Quantization Configuration	64
15.3.1	Detailed Description	66
15.3.2	Function Documentation	66
15.3.2.1	<code>__init__()</code> [1/3]	66
15.3.2.2	<code>default_config()</code> [1/3]	68
15.3.2.3	<code>from_kwargs()</code> [1/3]	68
15.3.2.4	<code>from_dict()</code> [1/3]	68
15.3.2.5	<code>to_dict()</code> [1/3]	68
15.3.2.6	<code>__init__()</code> [2/3]	69
15.3.2.7	<code>default_config()</code> [2/3]	70
15.3.2.8	<code>from_kwargs()</code> [2/3]	70
15.3.2.9	<code>from_dict()</code> [2/3]	70
15.3.2.10	<code>to_dict()</code> [2/3]	70
15.3.2.11	<code>__init__()</code> [3/3]	70
15.3.2.12	<code>default_config()</code> [3/3]	71
15.3.2.13	<code>from_kwargs()</code> [3/3]	71
15.3.2.14	<code>from_dict()</code> [3/3]	71
15.3.2.15	<code>to_dict()</code> [3/3]	71
15.3.2.16	<code>get_calibration_config()</code>	71
15.3.2.17	<code>get_bit_config()</code>	71
15.3.2.18	<code>get_quantization_config()</code>	72
15.3.3	Variable Documentation	72
15.3.3.1	<code>optional</code> [1/3]	72
15.3.3.2	<code>method_list</code>	72
15.3.3.3	<code>output_list</code>	72
15.3.3.4	<code>mode_list</code>	72
15.3.3.5	<code>search_type_list</code>	72
15.3.3.6	<code>DEFAULTS</code> [1/2]	73
15.3.3.7	<code>py_to_cpp_name</code> [1/3]	73
15.3.3.8	<code>min_max_check</code> [1/3]	74
15.3.3.9	<code>quantization_method</code>	74
15.3.3.10	<code>quantization_output</code>	74

15.3.3.11 quantization_mode	75
15.3.3.12 percentile	75
15.3.3.13 topk_ratio	75
15.3.3.14 max_each	75
15.3.3.15 max_total	75
15.3.3.16 hist_percentile	75
15.3.3.17 hist_use_gpu	75
15.3.3.18 hist_num_bins	76
15.3.3.19 hist_num_samples	76
15.3.3.20 hist_buffer_size	76
15.3.3.21 hist_min_bin_width	76
15.3.3.22 hist_search_percentile_min	76
15.3.3.23 hist_search_percentile_max	76
15.3.3.24 hist_num_search	76
15.3.3.25 hist_search_type	77
15.3.3.26 act_scale_min	77
15.3.3.27 weight_scale_min	77
15.3.3.28 min_clip_ratio	77
15.3.3.29 layer_overrides	77
15.3.3.30 optional [2/3]	77
15.3.3.31 DEFAULTS [2/2]	78
15.3.3.32 py_to_cpp_name [2/3]	78
15.3.3.33 min_max_check [2/3]	79
15.3.3.34 query_act_bits	79
15.3.3.35 key_act_bits	79
15.3.3.36 value_act_bits	79
15.3.3.37 output_act_bits	79
15.3.3.38 ffn_act_bits	80
15.3.3.39 head_act_bits	80
15.3.3.40 query_weight_bits	80
15.3.3.41 key_weight_bits	80
15.3.3.42 value_weight_bits	80
15.3.3.43 output_weight_bits	80
15.3.3.44 ffn_weight_bits	80
15.3.3.45 head_weight_bits	81
15.3.3.46 mixed_precision_apply	81
15.3.3.47 save_path	81
15.3.3.48 load_path	81
15.3.3.49 activation_16bits	81
15.3.3.50 weight_16bits	81

15.3.3.51 optional [3/3]	81
15.3.3.52 py_to_cpp_name [3/3]	82
15.3.3.53 min_max_check [3/3]	82
15.3.3.54 calibration	82
15.3.3.55 bit	82
15.4 Resource Management Configuration	82
15.4.1 Detailed Description	83
15.4.2 Function Documentation	83
15.4.2.1 __init__()	83
15.4.2.2 default_config()	84
15.4.2.3 from_kwargs()	84
15.4.2.4 from_dict()	84
15.4.2.5 to_dict()	84
15.4.2.6 get_resource_management_config()	85
15.4.3 Variable Documentation	85
15.4.3.1 optional	85
15.4.3.2 DEFAULTS	85
15.4.3.3 py_to_cpp_name	85
15.4.3.4 min_max_check	86
15.4.3.5 weight_memory_methods	86
15.4.3.6 weight_dtype	86
15.4.3.7 use_gpu_only_for_calibration	86
15.4.3.8 weight_memory_method	86
15.5 LLM Configuration	87
15.5.1 Detailed Description	87
15.5.2 Function Documentation	88
15.5.2.1 __init__()	88
15.5.2.2 default_config()	88
15.5.2.3 from_kwargs()	89
15.5.2.4 from_dict()	89
15.5.2.5 to_dict()	89
15.5.2.6 get_llm_config()	89
15.5.3 Variable Documentation	89
15.5.3.1 optional	89
15.5.3.2 DEFAULTS	90
15.5.3.3 py_to_cpp_name	90
15.5.3.4 min_max_check	90
15.5.3.5 llm_config_apply	91
15.5.3.6 max_data_length	91
15.5.3.7 max_sequence_length	91

15.5.3.8 max_cache_length	91
15.5.3.9 max_core_data_length	91
15.5.3.10 random_seq_length	91
15.5.3.11 use_full_seq_length	91
15.5.3.12 use_global_core	92
15.5.3.13 batch_size	92
15.5.3.14 npu_core_ids	92
15.5.3.15 dynamic_rope	92
15.6 AdvancedQuantization Configuration	92
15.6.1 Detailed Description	93
15.6.2 Function Documentation	93
15.6.2.1 __init__()	93
15.6.2.2 default_config()	93
15.6.2.3 from_dict()	94
15.6.2.4 to_dict()	94
15.6.2.5 get_advanced_quantization_config()	94
15.6.3 Variable Documentation	94
15.6.3.1 optional	94
15.6.3.2 optq	94
15.6.3.3 mod	95
15.6.3.4 equivalent_transformation	95
15.6.3.5 search_weight_scale	95
15.7 EquivalentTransformation Configuration	95
15.7.1 Detailed Description	98
15.7.2 Function Documentation	98
15.7.2.1 __init__()	98
15.7.3 NormConv	99
15.7.4 QK	99
15.7.5 UD	99
15.7.6 VO	100
15.7.7 FeedForwardMultiLUT	100
15.7.8 SpinR1	100
15.7.9 HeadOutChRotation	100
15.7.10 SpinR2	100
15.7.11 QKRotation	100
15.7.12 FlattenQuant	101
15.7.13 OptimizeFFN	101
15.7.13.1 check_valid()	101
15.7.13.2 default_config()	101
15.7.13.3 from_dict()	102

15.7.13.4 to_dict()	102
15.7.13.5 get_equivalent_transformation_config()	102
15.7.14 Variable Documentation	102
15.7.14.1 optional	102
15.7.14.2 py_to_cpp_name	102
15.7.14.3 DEFAULTS	102
15.7.14.4 min_max_check	103
15.7.14.5 seed	103
15.7.14.6 apply_hadamard_rotation_matrix	103
15.7.14.7 norm_conv_apply	103
15.7.14.8 norm_conv_learn	103
15.7.14.9 norm_conv_smoothing_factor	104
15.7.14.10 norm_conv_min_gamma	104
15.7.14.11 norm_conv_max_gamma	104
15.7.14.12 qk_apply	104
15.7.14.13 qk_smoothing_factor	104
15.7.14.14 qk_min_gamma	104
15.7.14.15 qk_max_gamma	105
15.7.14.16 ud_apply	105
15.7.14.17 ud_learn	105
15.7.14.18 ud_smoothing_factor	105
15.7.14.19 ud_min_gamma	105
15.7.14.20 ud_max_gamma	105
15.7.14.21 vo_apply	106
15.7.14.22 vo_smoothing_factor	106
15.7.14.23 vo_min_gamma	106
15.7.14.24 vo_max_gamma	106
15.7.14.25 ff_multi_lut_apply	106
15.7.14.26 ff_multi_lut_breakpoints	106
15.7.14.27 spin_r1_apply	107
15.7.14.28 spin_r1_matrix_path	107
15.7.14.29 head_out_ch_rotation_apply	107
15.7.14.30 head_out_ch_rotation_matrix_path	107
15.7.14.31 spin_r2_apply	107
15.7.14.32 spin_r2_learn	107
15.7.14.33 spin_r2_matrix_path	108
15.7.14.34 qk_rotation_apply	108
15.7.14.35 qk_rotation_matrix_path	108
15.7.14.36 flatten_quant_apply	108
15.7.14.37 flatten_quant_learn	108

15.7.14.38 flatten_quant_apply_threshold	108
15.7.14.39 flatten_quant_max_overhead	109
15.7.14.40 optimize_ffn_apply	109
15.7.14.41 optimize_ffn_ch_per_ffn	109
15.8 SearchWeightScale Configuration	109
15.8.1 Detailed Description	110
15.8.2 Function Documentation	110
15.8.2.1 __init__()	110
15.8.2.2 default_config()	111
15.8.2.3 from_dict()	111
15.8.2.4 to_dict()	111
15.8.2.5 get_search_weight_scale_config()	111
15.8.3 Variable Documentation	111
15.8.3.1 optional	111
15.8.3.2 DEFAULTS	112
15.8.3.3 apply	112
15.8.3.4 query	112
15.8.3.5 key	112
15.8.3.6 value	112
15.8.3.7 out	112
15.8.3.8 ffn	113
15.9 Mod Configuration	113
15.9.1 Detailed Description	114
15.9.2 Function Documentation	114
15.9.2.1 __init__()	114
15.9.2.2 default_config()	116
15.9.2.3 from_dict()	116
15.9.2.4 to_dict()	116
15.9.2.5 get_mod_config()	116
15.9.3 Variable Documentation	117
15.9.3.1 optional	117
15.9.3.2 DEFAULTS	117
15.9.3.3 py_to_cpp_name	117
15.9.3.4 min_max_check	118
15.9.3.5 apply	118
15.9.3.6 epochs	118
15.9.3.7 warmup_epochs	118
15.9.3.8 lr_act_scale	118
15.9.3.9 lr_zeropoint	119
15.9.3.10 lr_weight_scale	119

15.9.3.11 lr_weight	119
15.9.3.12 lr_bias	119
15.9.3.13 lr_min_ratio	119
15.9.3.14 batch_size	119
15.9.3.15 q_drop	119
15.9.3.16 post_processing	120
15.9.3.17 box_conf_thres	120
15.9.3.18 box_iou_thres	120
15.9.3.19 loss_type	120
15.9.3.20 use_outputs	120
15.9.3.21 kl_temperature	120
15.9.3.22 recon_prob	120
15.9.3.23 recon_coeff	121
15.9.3.24 lambda_0	121
15.9.3.25 lambda_1	121
15.9.3.26 lambda_2	121
15.9.3.27 anchors	121
15.9.3.28 use_xyxy	121
15.9.3.29 save_dir	121
15.9.3.30 seed	122
15.9.3.31 apply_layers	122
15.9.3.32 exclude_layers	122
15.9.3.33 custom_loss_jit_path	122
15.9.3.34 mod_after_layer_name	122
15.10 Optq Configuration	122
15.10.1 Detailed Description	123
15.10.2 Function Documentation	123
15.10.2.1 __init__()	123
15.10.2.2 default_config()	124
15.10.2.3 from_dict()	124
15.10.2.4 to_dict()	124
15.10.2.5 get_optq_config()	124
15.10.3 Variable Documentation	125
15.10.3.1 optional	125
15.10.3.2 DEFAULTS	125
15.10.3.3 py_to_cpp_name	125
15.10.3.4 min_max_check	125
15.10.3.5 apply	126
15.10.3.6 act_order	126
15.10.3.7 block_size	126

15.10.3.8 perc_damp	126
15.10.3.9 apply_layers	126
15.10.3.10 exclude_layers	126
15.11 Input Process Configuration	126
15.11.1 Detailed Description	128
15.11.2 Function Documentation	128
15.11.2.1 __init__() [1/3]	128
15.11.2.2 default_config() [1/3]	129
15.11.2.3 from_dict() [1/3]	129
15.11.2.4 to_dict() [1/3]	129
15.11.2.5 get_uint8_input_config()	129
15.11.2.6 __init__() [2/3]	129
15.11.2.7 default_config() [2/3]	130
15.11.2.8 from_dict() [2/3]	130
15.11.2.9 to_dict() [2/3]	130
15.11.2.10 get_preprocessing_config()	130
15.11.2.11 __init__() [3/3]	131
15.11.2.12 default_config() [3/3]	131
15.11.2.13 from_dict() [3/3]	131
15.11.2.14 to_dict() [3/3]	131
15.11.2.15 get_input_process_config()	131
15.11.3 Variable Documentation	132
15.11.3.1 optional [1/3]	132
15.11.3.2 DEFAULTS [1/3]	132
15.11.3.3 py_to_cpp_name [1/3]	132
15.11.3.4 min_max_check [1/3]	133
15.11.3.5 apply [1/2]	133
15.11.3.6 inputs	133
15.11.3.7 optional [2/3]	133
15.11.3.8 DEFAULTS [2/3]	133
15.11.3.9 py_to_cpp_name [2/3]	133
15.11.3.10 min_max_check [2/3]	134
15.11.3.11 apply [2/2]	134
15.11.3.12 auto_convert_format	134
15.11.3.13 pipeline	134
15.11.3.14 input_configs	134
15.11.3.15 optional [3/3]	134
15.11.3.16 DEFAULTS [3/3]	134
15.11.3.17 py_to_cpp_name [3/3]	135
15.11.3.18 min_max_check [3/3]	135

15.11.3.19 uint8_input	135
15.11.3.20 image_channels	135
15.11.3.21 preprocessing	135
16 Class Documentation	136
16.1 qbcompiler.configs.advanced_quantization_config.AdvancedQuantizationConfig Class Reference	136
16.1.1 Detailed Description	136
16.2 qbcompiler.configs.quantization_config.BitConfig Class Reference	137
16.2.1 Detailed Description	138
16.3 qbcompiler.configs.quantization_config.CalibrationConfig Class Reference	138
16.3.1 Detailed Description	139
16.4 qbcompiler.calibration.preprocess.CenterCrop Class Reference	140
16.4.1 Detailed Description	140
16.4.2 Constructor & Destructor Documentation	140
16.4.2.1 __init__()	140
16.4.3 Member Function Documentation	140
16.4.3.1 __call__()	140
16.4.4 Member Data Documentation	141
16.4.4.1 size	141
16.5 qbcompiler.configs.compile_config.CompileConfig Class Reference	141
16.5.1 Detailed Description	142
16.5.2 Constructor & Destructor Documentation	142
16.5.2.1 __init__()	142
16.5.3 Member Function Documentation	144
16.5.3.1 default_config()	144
16.5.3.2 from_json()	144
16.5.3.3 check_valid_str_configs()	145
16.5.3.4 to_dict()	145
16.5.3.5 _get_calibration_path()	145
16.5.4 Member Data Documentation	145
16.5.4.1 optional	145
16.5.4.2 py_to_cpp_name	145
16.5.4.3 min_max_check	146
16.5.4.4 inference_scheme_types	146
16.5.4.5 sample_dtypes	146
16.5.4.6 calib_data_path	146
16.5.4.7 save_path	146
16.5.4.8 model_nickname	146
16.5.4.9 save_msgpack_name	147

16.5.4.10	model_path	147
16.5.4.11	inference_scheme	147
16.5.4.12	save_sample	147
16.5.4.13	cpu_offload	147
16.5.4.14	optimize_option	147
16.5.4.15	sample_dtype	147
16.5.4.16	buffer_mode	148
16.5.4.17	version	148
16.5.4.18	input_shape_dict	148
16.5.4.19	resource_management	148
16.5.4.20	quantization	148
16.5.4.21	advanced_quantization	148
16.5.4.22	llm_config	148
16.5.4.23	input_process	149
16.5.4.24	calib_file	149
16.6	qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig	
	Class Reference	149
16.6.1	Detailed Description	151
16.7	qbcompiler.calibration.preprocess.GetImage Class Reference	151
16.7.1	Detailed Description	151
16.7.2	Constructor & Destructor Documentation	152
16.7.2.1	__init__()	152
16.7.3	Member Function Documentation	152
16.7.3.1	__call__()	152
16.7.3.2	get_image_data()	152
16.7.4	Member Data Documentation	152
16.7.4.1	to_float32	152
16.7.4.2	channel_order	153
16.7.4.3	cvtColor	153
16.7.4.4	COLOR_BGR2GRAY	153
16.7.4.5	COLOR_BGRA2GRAY	153
16.7.4.6	COLOR_BGRA2BGR	153
16.7.4.7	COLOR_BGR2RGB	153
16.7.4.8	COLOR_BGRA2RGB	153
16.7.4.9	COLOR_BGRA2RGBA	154
16.7.4.10	COLOR_BGR2BGRA	154
16.7.4.11	COLOR_BGR2RGBA	154
16.7.4.12	num_channel	154
16.8	qbcompiler.calibration.preprocess.GetNpy Class Reference	154
16.8.1	Detailed Description	154

16.8.2 Constructor & Destructor Documentation	155
16.8.2.1 <code>__init__()</code>	155
16.8.3 Member Function Documentation	155
16.8.3.1 <code>__call__()</code>	155
16.9 GetText Class Reference	155
16.9.1 Detailed Description	155
16.10 qbcompiler.configs.input_process_config.InputProcessConfig Class Reference	156
16.10.1 Detailed Description	156
16.11 qbcompiler.configs.llm_config.LlmConfig Class Reference	157
16.11.1 Detailed Description	158
16.12 qbcompiler.configs.mod_config.ModConfig Class Reference	158
16.12.1 Detailed Description	159
16.13 qbcompiler.frontend.Model_Dict Class Reference	160
16.13.1 Detailed Description	160
16.13.2 Constructor & Destructor Documentation	160
16.13.2.1 <code>__init__()</code>	160
16.13.3 Member Function Documentation	162
16.13.3.1 <code>compile()</code>	162
16.14 qbcompiler.calibration.preprocess.Normalize Class Reference	163
16.14.1 Detailed Description	163
16.14.2 Constructor & Destructor Documentation	164
16.14.2.1 <code>__init__()</code>	164
16.14.3 Member Function Documentation	164
16.14.3.1 <code>__call__()</code>	164
16.14.4 Member Data Documentation	164
16.14.4.1 <code>mean</code>	164
16.14.4.2 <code>std</code>	164
16.14.4.3 <code>to_float_div255</code>	165
16.15 qbcompiler.calibration.preprocess.NpyNormalize Class Reference	165
16.15.1 Detailed Description	165
16.15.2 Constructor & Destructor Documentation	165
16.15.2.1 <code>__init__()</code>	165
16.15.3 Member Function Documentation	166
16.15.3.1 <code>__call__()</code>	166
16.15.4 Member Data Documentation	166
16.15.4.1 <code>mean</code>	166
16.15.4.2 <code>std</code>	166
16.15.4.3 <code>to_float_div255</code>	166
16.16 qbcompiler.calibration.preprocess.NpyPad Class Reference	166
16.16.1 Detailed Description	167

16.16.2 Constructor & Destructor Documentation	167
16.16.2.1 __init__()	167
16.16.3 Member Function Documentation	167
16.16.3.1 __call__()	167
16.16.3.2 slice_padded_data()	167
16.16.4 Member Data Documentation	168
16.16.4.1 shape	168
16.16.4.2 padding_type	168
16.16.4.3 pad_val	168
16.17 qbcompiler.configs.optq_config.OptqConfig Class Reference	168
16.17.1 Detailed Description	169
16.18 qbcompiler.calibration.preprocess.Pad Class Reference	169
16.18.1 Detailed Description	169
16.18.2 Constructor & Destructor Documentation	170
16.18.2.1 __init__()	170
16.18.3 Member Function Documentation	170
16.18.3.1 __call__()	170
16.18.4 Member Data Documentation	170
16.18.4.1 pad_val	170
16.18.4.2 size_divisor	170
16.18.4.3 shape	171
16.18.4.4 right_bottom	171
16.19 qbcompiler.calibration.preprocess.preproc_builder Class Reference	171
16.19.1 Detailed Description	172
16.19.2 Constructor & Destructor Documentation	173
16.19.2.1 __init__()	173
16.19.3 Member Function Documentation	173
16.19.3.1 __call__()	173
16.19.3.2 __repre__()	173
16.19.3.3 __str__()	173
16.19.3.4 get_pre_list()	174
16.19.3.5 check_yaml_pre()	174
16.19.4 Member Data Documentation	174
16.19.4.1 preprocess	174
16.19.4.2 to_tensor	174
16.19.4.3 args	174
16.19.4.4 datatype	174
16.19.4.5 loadtype	175
16.20 qbcompiler.configs.input_process_config.PreprocessingConfig Class Reference	175
16.20.1 Detailed Description	176

16.21 qbcompiler.configs.quantization_config.QuantizationConfig Class Reference	176
16.21.1 Detailed Description	177
16.22 qbcompiler.calibration.preprocess.Resize Class Reference	177
16.22.1 Detailed Description	177
16.22.2 Constructor & Destructor Documentation	178
16.22.2.1 __init__()	178
16.22.3 Member Function Documentation	178
16.22.3.1 __call__()	178
16.22.4 Member Data Documentation	178
16.22.4.1 img_scale	178
16.22.4.2 keep_ratio	178
16.22.4.3 interpolation	178
16.23 qbcompiler.calibration.preprocess.ResizeTorch Class Reference	179
16.23.1 Detailed Description	179
16.23.2 Constructor & Destructor Documentation	179
16.23.2.1 __init__()	179
16.23.3 Member Function Documentation	179
16.23.3.1 __call__()	179
16.23.4 Member Data Documentation	179
16.23.4.1 size	179
16.23.4.2 interpolation	180
16.24 qbcompiler.configs.resource_management_config.ResourceManagementConfig Class Reference	180
16.24.1 Detailed Description	181
16.25 qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig Class Reference	181
16.25.1 Detailed Description	182
16.26 qbcompiler.calibration.preprocess.SetOrder Class Reference	182
16.26.1 Detailed Description	182
16.26.2 Constructor & Destructor Documentation	182
16.26.2.1 __init__()	182
16.26.3 Member Function Documentation	182
16.26.3.1 __call__()	182
16.26.4 Member Data Documentation	183
16.26.4.1 shape	183
16.27 TokenizeToEmbedding Class Reference	183
16.27.1 Detailed Description	183
16.28 qbcompiler.configs.input_process_config.Uint8InputConfig Class Reference	183
16.28.1 Detailed Description	184
16.29 YoloPre Class Reference	184

16.29.1 Detailed Description	184
Index	185

1 소개

모빌린트 모델 컴파일러(Mobilint® Model Compiler, 이하 컴파일러)는 딥러닝 프레임워크(ONNX, PyTorch, Keras, TensorFlow 등)의 모델을 모빌린트 NPU에서 실행 가능한 모빌린트 실행 파일(Mobilint® Model eXeCutable, 이하 MXQ)로 변환하는 도구입니다. 이 문서는 모빌린트의 SDK인 **qbcompiler**에 대한 문서입니다. 이 문서에서는 컴파일러의 사용 방법, 지원하는 프레임워크 등에 대해 배울 수 있습니다. 아래에는 SDK와 상호작용하기 위해 사용할 수 있는 함수 세트가 제공됩니다. qbcompiler에 대한 입력은 훈련된 딥러닝 모델, 입력 형태 및 calibration 데이터이며, 출력으로는 MXQ(컴파일된 모델)를 반환합니다.

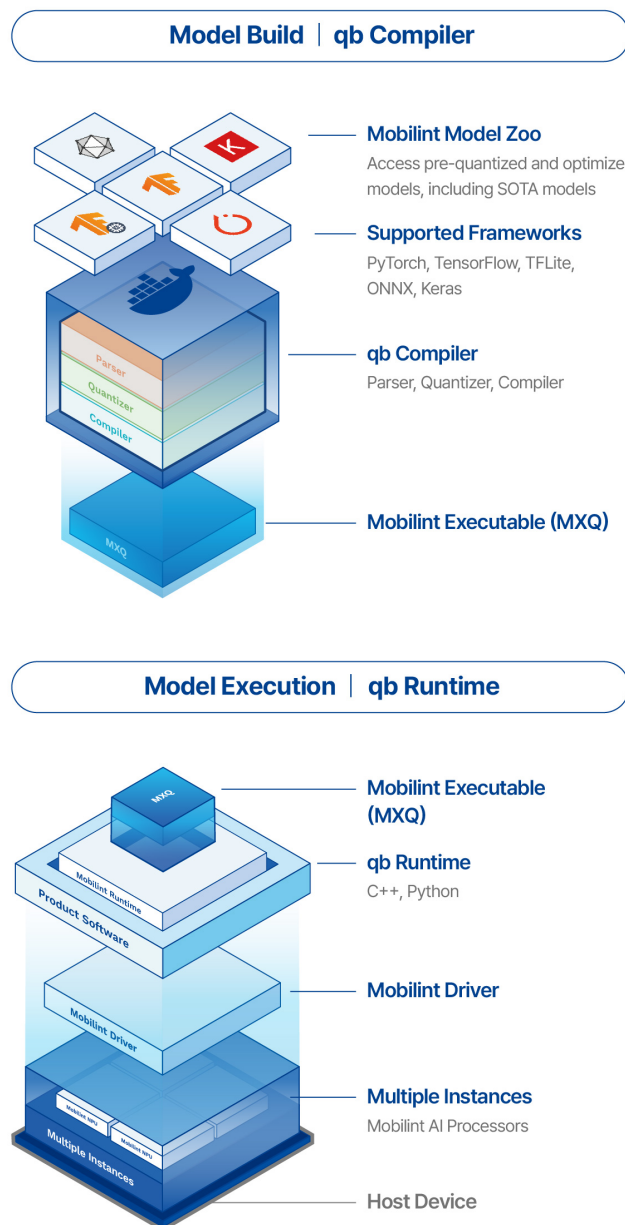


Figure 1.1 SDK 구성요소

2 설치

2.1 시스템 요구사항

NVIDIA GPU가 장착된 환경에서 qbcompiler를 설치하는 것을 권장합니다. 그러나 NVIDIA GPU가 없는 CPU 전용 버전의 qbcompiler도 지원합니다.

2.1.1 기준 환경

- Ubuntu 22.04.4 LTS
- NVIDIA Graphics Driver 545.29.06

2.1.2 패키지 요구사항 및 권장사항

- Ubuntu 20.04.6 LTS or Above
- NVIDIA Graphics Driver 535.183.01 or Above
- Docker
- NVIDIA Container Toolkit

2.2 SDK 설치

qbcompiler는 모빌린트의 공식 도커 컨테이너 상에 설치하는 것을 권장합니다. 큐비 컴파일러 v0.10.0.0부터는 버전의 앞 두 자리가 도커 이미지 앞 두 자리와 일치해야 합니다. 예를 들어 큐비 컴파일러 v1.0.0을 사용할 때는 1.0으로 시작하는 도커 이미지를 선택하세요. 또한 큐비 도커 0.11 이상부터는 cuda12.8.1 이미지를 기반으로 하여 NVIDIA 드라이버가 R570 계열 이상이어야 합니다. 따라서 NVIDIA 드라이버를 가장 최신으로 업데이트 하는 것을 권장드립니다.

- 도커 이미지: mobilint/qbcompiler:1.0-cuda12.8.1-ubuntu22.04
- Link: <https://hub.docker.com/r/mobilint/qbcompiler>

2.2.1 도커 컨테이너 빌드

다음의 명령어를 실행하여 도커 컨테이너를 빌드합니다.

```
# Download Docker Image
docker pull mobilint/qbcompiler:1.0-cuda12.8.1-ubuntu22.04
# mkdir {WORKING DIRECTORY} (if needed)
cd {WORKING DIRECTORY}
# Build the Docker container
docker run -it --gpus all --ipc=host --name {YOUR_CONTAINER_NAME} -v
$(pwd):/workspace mobilint/qbcompiler:1.0-cuda12.8.1-ubuntu22.04 /bin/bash
```

(권장사항) 훈련된 모델과 데이터셋이 다른 디렉토리에 저장되어 있는 경우, 다음과 같이 도커 컨테이너에 마운트할 수 있습니다.

```
docker run -it --gpus all --ipc=host --name {YOUR_CONTAINER_NAME} -v
$(pwd):/workspace -v {PATH TO MODEL DIR}:/models -v {PATH TO DATASET DIR}:/datasets
mobilint/qbcompiler:1.0-cuda12.8.1-ubuntu22.04 /bin/bash
```

도커 컨테이너에 NPU를 연결하고자 하는 경우, 도커 컨테이너를 생성할 때 `--device /dev/aries0` : `/dev/aries0` 옵션을 추가합니다.

(선택사항) CPU 전용 버전의 이미지를 사용하여 도커 컨테이너를 빌드하면, GPU가 없는 컴퓨터에서도 Mobilint Compiler를 실행할 수 있습니다.

```
# Download Docker Image
docker pull mobilint/qbcompiler:1.0-cpu-ubuntu22.04
cd {WORKING DIRECTORY}
# Build the Docker container
docker run -it --ipc=host --name {YOUR_CONTAINER_NAME} -v $(pwd):/workspace
mobilint/qbcompiler:1.0-cpu-ubuntu22.04 /bin/bash
```

2.2.2 qbcompiler 설치

qbcompiler 패키지는 [Mobilint® Software Development Kit \(SDK\)](#) 배포 사이트에서 다운로드 받을 수 있습니다. 사이트에 접속한 후, 사용하는 장치를 선택하고 해당하는 qbcompiler 패키지를 다운로드합니다. 패키지는 Python wheel 파일 형태로 제공되며, pip를 사용하여 설치할 수 있습니다. 다음의 명령어를 실행하여 도커 컨테이너에 qbcompiler를 설치합니다.

```
# Download qbcompiler-1.0.0+aries-py3-none-any.whl file
# Copy qbcompiler whl file to Docker
docker cp {Path to qbcompiler-1.0.0+aries-py3-none-any.whl} {YOUR_CONTAINER_NAME}:/
# Start Docker
docker start {YOUR_CONTAINER_NAME}
# Attach to Docker container
docker exec -it {YOUR_CONTAINER_NAME} /bin/bash
# Install qbcompiler
cd /
python -m pip install qbcompiler-1.0.0+aries-py3-none-any.whl
```

.whl 파일은 필요한 NPU 장치의 이름을 포함하고 있으므로, 설치 전에 확인해야 합니다. 현재 ARIES, REGULUS와 같은 장치에 대한 지원을 제공합니다.

3 빠른 시작

이 문서에서는 qbcompiler를 사용하여 calibration 데이터셋을 준비하고, 모델을 컴파일하며, 추론을 수행하는 방법에 대해 설명합니다. 더 많은 예제와 자세한 내용은 [튜토리얼](#)을 참조하세요.

3.1 Calibration Data 준비

모델을 컴파일하기 위해서는 calibration 데이터셋(모델이 실제로 사용하는 입력의 전처리된 형태)을 준비해야 합니다. qbcompiler는 calibration 데이터셋을 준비하기 위한 두 가지 방법을 제공합니다:

1. 사전 정의된 전처리 구성 YAML 파일을 활용하는 방법
2. 사용자가 직접 정의한 전처리 함수를 사용하는 방법

Remark Calibration 데이터셋을 생성하는 과정은 CPU Offloading 사용 여부에 따라 달라질 수 있습니다. 현재 qbcompiler는 기본적으로 CPU Offloading 없이 모델을 컴파일합니다. 이 경우, 사전 처리된 입력 형태는 (H, W, C) 형식이어야 합니다. 반면, CPU Offloading이 사용되는 경우, 사전 처리된 입력 형태는 원래 모델이 사용하는 입력 형태와 일치해야 합니다.

이하에서는 ImageNet으로부터 선별된 1000개의 이미지를 사용하여 분류 모델을 위한 calibration 데이터셋을 준비하는 방법에 대해 설명합니다.

3.1.1 전처리 구성 YAML 파일을 활용한 방법

이미지 크기 조정, 자르기, 정규화와 같은 이미지 전처리 기술은 머신 비전 작업에서 자주 사용됩니다. 사용자는 YAML 파일을 사용하여 전처리 구성을 작성하고, qbcompiler에서 제공하는 API인 `make_calib`를 통해 calibration 데이터셋을 준비할 수 있습니다. 이 방법은 원시 데이터가 이미지인 경우에만 사용할 수 있습니다. 아래는 예제 코드입니다. 이 코드는 calibration을 위한 이미지가 `/workspace/calibration/cali_1000` 디렉토리에 준비되어 있다고 가정합니다.

먼저, 실행 가능한 Python 스크립트(예: `prepare_calib.py`)를 생성하고 다음 내용을 작성합니다:

```
from qbcompiler.calibration import make_calib
make_calib(
    args_pre="/workspace/resnet50.yaml", # path to pre-processing configuration yaml
    file
    data_dir="/workspace/calibration/cali_1000", # path to folder of original
    calibration data files such as images
    save_dir="/workspace/calibration/", # path to folder to save pre-processed
    calibration data files
    save_name="resnet50", # tag for the generated calibration dataset
    max_size=100 # Maximum number of data to use for calibration
)
```

그 후, 위 코드에서 지정한 디렉토리에 사전 처리 구성 YAML 파일(예: `resnet50.yaml`)을 작성합니다. YAML 파일에는 사전 처리 함수와 해당 매개변수가 포함되어야 합니다. 아래는 ResNet50 사전 처리를 위한 YAML 파일의 예입니다.

```
# resnet50.yaml
Datatype: Image
```



```

GetImage:
  to_float32: false
  channel_order: RGB

Pre-Order: [ResizeTorch, CenterCrop, Normalize, SetOrder]
Pre-processing:
  ResizeTorch:
    size: 256
    interpolation: bilinear
  CenterCrop:
    size: [224, 224]
  Normalize:
    mean: [0.485, 0.456, 0.406]
    std: [0.229, 0.224, 0.225]
    to_float_div255: true
  SetOrder:
    shape: HWC

```

위의 코드를 실행하면, 지정된 디렉토리에 사전 처리된 calibration 데이터셋(NumPy 배열)이 포함된 디렉토리가 생성됩니다. 이 디렉토리는 /workspace/calibration/resnet50에 위치합니다. 또한, 사전 처리된 NumPy 파일의 경로를 포함하는 메타 txt 파일이 생성되며, 이름은 /workspace/calibration/resnet50.txt입니다.

Remark 사전 정의된 전처리 구성 YAML 파일을 사용하는 경우, calibration 데이터셋은 이미지 형식이 동일해야 합니다. 만약 일부가 컬러 이미지이고 다른 일부가 그레이스케일 이미지인 경우, calibration 데이터셋이 제대로 생성되지 않습니다.

3.1.2 사용자 정의 전처리 함수를 사용하는 방법

사용자 정의 전처리 함수를 사용하여 calibration 데이터셋을 준비할 수도 있습니다. 이 경우 qbcompiler에서 제공하는 API인 make_calib_man를 사용하여 calibration 데이터셋을 생성할 수 있습니다. 사용자 정의 전처리 함수는 이미지 경로를 입력으로 받아 NumPy 배열을 반환해야 합니다. 아래는 예제 코드입니다. 이 코드는 calibration을 위한 이미지가 /workspace/calibration/cali_1000 디렉토리에 준비되어 있다고 가정합니다.

이전의 YAML 파일을 사용하는 방법과 유사하게, 사용자 정의 전처리 함수를 사용하여 calibration 데이터셋을 준비하려면 먼저 실행 가능한 Python 스크립트(예: prepare_calib.py)를 생성하고 다음 내용을 작성합니다:

```

import torch
import numpy as np
from PIL import Image
import torchvision.transforms.functional as F
from torchvision.transforms import InterpolationMode
from qbcompiler.calibration import make_calib_man

def preprocess_resnet50(img_path: str):
    img = Image.open(img_path)
    resize_size=256
    crop_size=(224, 224)
    mean=[0.485, 0.456, 0.406]
    std=[0.229, 0.224, 0.225]
    out = F.pil_to_tensor(img)
    out = F.resize
    (out, size=resize_size, interpolation = InterpolationMode.BILINEAR)
    out = F.center_crop(out, output_size=crop_size)
    out = out.to(torch.float, copy=False) / 255.
    out = F.normalize(out, mean, std)
    out = np.transpose(out.numpy(), axes=[1, 2, 0])
    return out

make_calib_man(

```

```

    pre_ftn=preprocess_resnet50, # callable function to pre-process the calibration
data
    data_dir="/workspace/calibration/cali_1000", # path to folder of original
calibration data files such as images
    save_dir="/workspace/calibration/", # path to folder to save pre-processed
calibration data files
    save_name="resnet50", # tag for the generated calibration dataset
    max_size=100 # Maximum number of data to use for calibration
)

```

위의 코드를 실행하면, 지정된 디렉토리에 사전 처리된 calibration 데이터셋(NumPy 텐서)이 포함된 디렉토리가 생성됩니다. 이 디렉토리는 /workspace/calibration/resnet50에 위치합니다. 또한, 사전 처리된 NumPy 파일의 경로를 포함하는 메타 txt 파일이 생성되며, 이름은 /workspace/calibration/resnet50.txt입니다.

Remark 사용자 정의 전처리 함수를 사용하는 경우, 함수 내부적으로 적절한 예외처리가 포함되지 않는 한, calibration 데이터셋은 동일한 형식의 이미지로 구성되어야 합니다. 이전 방법과 마찬가지로, 일부가 컬러 이미지이고 다른 일부가 그레이스케일 이미지인 경우, calibration 데이터셋이 제대로 생성되지 않습니다.

3.2 딥러닝 모델 컴파일

calibration 데이터셋이 준비되면, qbcompiler를 사용하여 모델을 컴파일할 수 있습니다. qbcompiler는 ONNX, PyTorch, Keras, TensorFlow 및 TensorFlow Lite와 같은 다양한 딥러닝 프레임워크를 지원합니다. 컴파일 과정은 모델을 읽고 양자화하여 Mobilint IR(MXQ) 파일을 생성하는 과정을 포함합니다.

3.2.1 ONNX 모델 컴파일

ONNX는 훈련된 모델을 Mobilint IR로 컴파일하는 데 가장 권장되는 프레임워크입니다. 간단한 코드로 ONNX 모델을 직접 파싱하여 Mobilint IR을 얻을 수 있습니다. 아래는 예제 코드입니다. 이 코드는 calibration 데이터셋과 모델이 각각 /workspace/calibration/resnet50와 /workspace/resnet50.onnx 디렉토리에 준비되어 있다고 가정합니다. 먼저, 실행 가능한 Python 스크립트(예: compile_onnx.py)를 생성하고 다음 내용을 작성합니다:

```

""" Compile ONNX model"""
from qbcompiler import mxq_compile
onnx_model_path = "/workspace/resnet50.onnx"
calib_data_path = "/workspace/calibration/resnet50"
# calib_data_path can be replaced with the path to the calibration meta file such as
"/workspace/calibration/resnet50.txt"

mxq_compile(
    model=onnx_model_path,
    calib_data_path=calib_data_path,
    save_path="resnet50.mxq",
    backend="onnx"
)

```

3.2.2 PyTorch 모델 컴파일

qbcompiler는 PyTorch 모델의 다이렉트 컴파일을 지원합니다. 컴파일을 시작하기 전에 feed_dict를 준비해야 하며, 이는 모델 입력 이름을 key로 하고 해당 입력에 대응하는 토치텐서를 value로 갖는 딕셔너리입니다. 컴파일 대상 모델의 forward 함수 파라미터 이름과 feed_dict의 key는 정확히 일치해야 합니다. 예를 들어 torchvision의 resnet50 기본 입력 이름은 "x"입니다. 또한 feed_dict에 넣는 텐서는 컴파일하려는 모델의 입력 과 동일해야 하며, 그 크기로

forward를 호출했을 때 오류가 없어야 합니다. resnet50의 입력으로 (1, 3, 224, 224) 같은 텐서는 유효하지만, (1, 8, 224, 224)처럼 채널 수가 잘못된 텐서는 컴파일되지 않습니다.

```

""" Compile PyTorch model"""
from qbcompiler import mxq_compile

import torchvision
import torch
import numpy as np
calib_data_path = "/workspace/calibration/resnet50"
# A calibration meta file such as "/workspace/calibration/resnet50.txt" can be used
instead.

torch_model = torchvision.models.resnet50(pretrained=True)
torch_model.eval().cpu()
feed_dict = {"x": torch.randn(1, 3, 224, 224).cpu()}

mxq_compile(
    model=torch_model,
    calib_data_path=calib_data_path,
    backend="torch",
    save_path="resnet50.mxq",
    feed_dict=feed_dict,
)

```

3.2.3 Torchscript 모델 컴파일

qbcompiler는 TorchScript 형식으로 변환된 모델의 컴파일을 지원합니다. 아래 예시 코드는 torchvision ResNet-50 모델을 TorchScript 저장후 로드하여 컴파일 하는 예시 코드입니다.

```

""" Compile PyTorch model"""
from qbcompiler import mxq_compile

import torchvision
import torch
import numpy as np
calib_data_path = "/workspace/calibration/resnet50"
# A calibration meta file such as "/workspace/calibration/resnet50.txt" can be used
instead.

torch_model = torchvision.models.resnet50(pretrained=True)
torchscript_model_path = "/workspace/resnet50.pt"

dummy_input = np.random.randn(1,3,224,224).astype(np.float32)
feed_dict = {"input": dummy_input}
#When compiling with TorchScript, feed_dict information is required, which contains
the temporary input name and a sample input.
scripted_model = torch.jit.script(torch_model, torch.tensor(dummy_input))
torch.jit.save(scripted_model, torchscript_model_path)

mxq_compile(
    model=torchscript_model_path,
    calib_data_path=calib_data_path,
    backend="torchscript",
    save_path="resnet50.mxq",
    feed_dict=feed_dict,
)

```

3.2.4 TensorFlow/Keras 모델 컴파일

Keras는 현재 TensorFlow의 고수준 API로 통합되어 있으며, Keras 모델은 TensorFlow 모델로 간주됩니다. 따라서 Keras 모델을 Mobilint IR로 컴파일하기 위해서는 TensorFlow 백엔드를 사용하게 되며, 두 프레임워크의 모델 컴파일 과정은 동일합니다. 현재 Keras 모델은 TensorFlow의 SavedModel 또는 frozen graph 형식으로 저장되어야 합니다. qbcompiler는 SavedModel 또는 고수준 저장 형식(.keras 또는 h5)으로 저장된 TensorFlow 모델을 컴파일할 수 있습니다. 아래 코드는 calibration 데이터셋과 모델이 각각 /workspace/calibration/resnet50와 /workspace/tf_models/resnet50.h5 디렉토리에 준비되어 있다고 가정합니다.

먼저, 실행 가능한 Python 스크립트(예: compile_tf.py)를 생성하고 다음 내용을 작성합니다:

```
""" Compile Keras/TensorFlow model in SavedModel format """
from qbcompiler import mxq_compile
import tensorflow as tf

keras_model = tf.keras.applications.resnet50.ResNet50() # Load a Keras model
calib_data_path = "/workspace/calibration/resnet50"
# A calibration metadata file such as "/workspace/calibration/resnet50.txt" can be
used instead.

keras_model_save_path = "/workspace/tf_models/resnet50.h5" # path to save .h5 keras
model
keras_model.save(keras_model_save_path)

mxq_compile(
    model=keras_model_save_path,
    calib_data_path=calib_data_path,
    backend="tf",
    save_path="resnet50.mxq",
)
```

3.2.5 TensorFlow Lite 모델 컴파일

qbcompiler는 TensorFlow Lite 모델의 컴파일을 지원합니다. 아래 예시 코드는 Keras resnet50 모델을 tflite모델로 변경 후 컴파일을 진행합니다.

```
""" Compile Tensorflow Lite model """
from qbcompiler import mxq_compile
import tensorflow as tf

keras_model = tf.keras.applications.resnet50.ResNet50
() # Load a pre-trained Keras model
calib_data_path = "/workspace/calibration/resnet50"
# A calibration metadata file such as "/workspace/calibration/resnet50.txt" can be
used instead.

keras_model_save_path = "/workspace/tf_models/resnet50.h5"# path to save .h5 keras
model
tflite_model_path = "/workspace/tflite_models/resnet50.tflite"
keras_model.save(keras_model_save_path)

loaded_keras_model = tf.keras.models.load_model(keras_model_save_path)
tflite_model = tf.lite.TFLiteConverter.from_keras_model(loaded_keras_model).convert
() # Convert the model to TFLite format
with open(tflite_model_path, 'wb') as f:
    f.write(tflite_model)

mxq_compile(
    model=tflite_model_path,
```

```

        calib_data_path=calib_data_path,
        backend="tflite",
        save_path="resnet50.mxq",
    )

```

3.3 (선택 사항) 양자화 방법 및 매개변수 설정

학습된 모델을 준비한 후, qbcompiler는 모델을 양자화하여 크기를 줄이고 추론 속도를 향상시킵니다. qbcompiler는 대부분의 모델에 적합한 신중하게 조정된 기본 양자화 방법을 갖추고 있습니다. 그러나 사용자는 특정 모델에 대해 더 나은 성능을 달성하기 위해 양자화 방법과 매개변수를 사용자 정의할 수도 있습니다.

아래 예시는 사용자가 지정한 양자화 방법과 매개변수를 사용하여 모델을 컴파일하는 방법을 보여줍니다. 양자화 방법과 매개변수에 대한 자세한 내용은 mxq_compile 함수에서 확인할 수 있습니다.

```

""" Quantization Methods and Parameters """
from qbcompiler import mxq_compile
onnx_model_path = "/workspace/resnet50.onnx"
calib_data_path = "/workspace/calibration/resnet50"

mxq_compile(
    model=onnx_model_path,
    calib_data_path=calib_data_path,
    quantization_method=1, # per channel quantization
    quantization_mode=1, # max percentile quantization
    percentile=0.999,
    quantization_output=0, # per layer quantization for the output layer
    save_path="resnet50.mxq",
    backend="onnx"
)

```

4 튜토리얼 개요

이 문서에서는 Mobilint® Compiler SDK 중 qbcompiler의 사용법을 설명합니다. qbcompiler는 딥러닝 모델을 ARIES와 REGULUS 등의 모빌린트 NPU에서 사용 가능한 모빌린트 IR(MXQ) 형식으로 변환합니다.

이하에서 소개될 예시들은 ResNet50, YOLOv8, Llama-3.2 등의 오픈소스 딥러닝 모델을 Calibration Dataset 준비부터 컴파일까지의 과정을 처음부터 끝까지 다룹니다.

본격적인 작업에 앞서 Mobilint Compiler SDK가 설치된 도커 환경에서 작업하는 것을 가정한다는 것을 알려드립니다. 만약 아직 환경이 구축되지 않았다면 [설치](#)를 참고하여 환경을 먼저 구축하시기 바랍니다.

5 TorchVision의 이미지 분류 모델

5.1 소개

`torchvision.models` 패키지는 이미지 분류, 인스턴스 분할, 물체 탐지, 인물 자세 감지, 비디오 분류 등의 다양한 작업을 수행하기 위해 사전에 학습된 모델들을 제공합니다.

이 튜토리얼에서 우리는 이미지 분류 작업에서 널리 사용되는 [ResNet50](#) 모델을 사용합니다. ResNet50 모델은 CNN 모델의 일종으로, Residual Block 구조로 연결된 50개의 레이어로 구성되어 있습니다. 보다 자세한 설명은 공식 페이지에서 확인하실 수 있습니다.

5.2 모델 준비

TorchVision은 `torchvision.models`를 통해 여러 딥러닝 모델의 기본 구조와 그에 대응하는 가중치를 제공합니다. 이 과정은 모델 인스턴스를 초기화하는 과정에서 사전학습된 모델을 캐시 위치에 다운로드하여 불러오는 방식으로 이루어집니다.

우리는 이렇게 준비된 모델을 PyTorch에서 제공하는 코드를 사용하여 ONNX 형식으로 변환하여 사용하는 것을 추천합니다. 아래 내용을 포함한 실행가능한 파이썬 코드(예: `prepare_resnet.py`)를 만들어서 모델을 변환할 수 있습니다.

```
import torch
from torchvision.models import resnet50, ResNet50_Weights

# Using pretrained weights:
model = resnet50(weights=ResNet50_Weights.IMAGENET1K_V1)
model.eval()

# make dummy input depending on the model's input shape
input = torch.randn(1, 3, 224, 224)

# export to onnx
torch.onnx.export(model, input, "resnet50.onnx")
```

5.3 Calibration Dataset 준비

TorchVision은 자신들이 모델을 학습시킨 방법에 대한 세부사항들을 제공합니다. 이러한 학습방법에는 보통 모델을 불러오는 방법, 보간법을 사용하여 이미지 크기를 재조정하는 방법, 이미지를 잘라내는 크기, 픽셀값의 조정과 정규화 등이 포함되어 있습니다. 따라서 Calibration Dataset을 준비하기에 앞서 전처리에 포함된 이러한 연산들과 모델의 입력값 크기를 확인해야 합니다.

예를 들어, 우리가 사용할 ResNet50의 경우 `PIL.Images` 함수를 이용하여 원본 이미지를 불러오고, 이중선형보간법을 사용하여 이미지를 종횡비를 유지한 채, 짧은 변이 256이 되도록 리사이즈 하며, 중심의 (224, 224) 크기의 정사각형 부분을 잘라내고, 픽셀값을 [0, 1] 범위로 조정된 후, 평균 = [0.485, 0.456, 0.406], 표준편차 = [0.229, 0.224, 0.225]를 사용하여 정규화합니다.

CLASSvision.models.ResNet50_Weights(*value*) [\[SOURCE\]](#)

The model builder above accepts the following values as the `weights` parameter.

`ResNet50_Weights.DEFAULT` is equivalent to `ResNet50_Weights.IMAGENET1K_V2`. You can also use strings, e.g. `weights='DEFAULT'` or `weights='IMAGENET1K_V1'`.

ResNet50_Weights.IMAGENET1K_V1:

These weights reproduce closely the results of the paper using a simple training recipe.

acc@1 (on ImageNet-1K)	76.13
acc@5 (on ImageNet-1K)	92.862
min_size	height=1, width=1
categories	tench, goldfish, great white shark, ... (997 omitted)
num_params	25557032
recipe	link
GFLOPS	4.09
File size	97.8 MB

The inference transforms are available at `ResNet50_Weights.IMAGENET1K_V1.transforms` and perform the following preprocessing operations: Accepts `PIL.Image`, batched `(B, C, H, W)` and single `(C, H, W)` image `torch.Tensor` objects. The images are resized to `resize_size=[256]` using `interpolation=InterpolationMode.BILINEAR`, followed by a central crop of `crop_size=[224]`. Finally the values are first rescaled to `[0.0, 1.0]` and then normalized using `mean=[0.485, 0.456, 0.406]` and `std=[0.229, 0.224, 0.225]`.

Figure 5.1 TorchVision ResNet50 학습 정보

ResNet50과 같은 이미지 분류 모델을 학습할 때는 보통 [ImageNet](#) 데이터셋이 사용됩니다. 따라서 이미지 종류 및 분포의 측면에서 학습에 사용한 데이터와 유사한 데이터가 Calibration Dataset으로 사용되는 것이 장려됩니다. 이 튜토리얼을 위해 모빌린트의 엔지니어들이 ImageNet의 각 클래스별로 1개의 이미지를 추출하여 두었으며, `cali_1000.zip`이라는 파일명으로 제공되니, 해당 파일을 사용하시길 권장합니다.

빠른 시작에 나온 바와 같이 `qbcompiler`는 TorchVision 패키지를 사용한 표준화된 전처리연산을 제공합니다. 따라서 Calibration Dataset은 두 가지 방법으로 만들어질 수 있습니다.

첫 번째 방법은 다음과 같은 내용을 포함한 환경설정용 YAML 파일(예: `resnet50.yaml`)을 이용한 것입니다.

```
# resnet50.yaml
Datatype: Image
GetImage:
  to_float32: false
  channel_order: RGB

Pre-Order: [ResizeTorch, CenterCrop, Normalize, SetOrder]
Pre-processing:
  ResizeTorch:
    size: 256
    interpolation: bilinear
  CenterCrop:
    size: [224, 224]
  Normalize:
    mean: [0.485, 0.456, 0.406]
```



```

    std: [0.229, 0.224, 0.225]
    to_float_div255: true
  SetOrder:
    shape: HWC

```

그 다음, 위 YAML 파일을 읽어서 실제 전처리를 수행하는 Python 실행 스크립트(예: prepare_calib.py)를 작성합니다.

```

from qbcompiler.calibration import make_calib
make_calib(
    args_pre="/workspace/resnet50.yaml", # path to pre-processing configuration yaml
    file
    data_dir="/workspace/calibration/cali_1000", # path to folder of original
    calibration data files such as images
    save_dir="/workspace/calibration/", # path to folder to save pre-processed
    calibration data files
    save_name="resnet50_cali", # tag for the generated calibration dataset
    max_size=100 # Maximum number of data to use for calibration
)

```

두 번째 방법은 사용자 정의 전처리함수를 이용하는 방법입니다. qbcompiler는 사용자가 직접 만든 전처리함수로도 손쉽게 Calibration Dataset을 구성할 수 있도록 해줍니다.

```

import torch
import numpy as np
from PIL import Image
import torchvision.transforms.functional as F
from torchvision.transforms import InterpolationMode
from qbcompiler.calibration import make_calib_man

def preprocess_resnet50(img_path: str):
    img = Image.open(img_path)
    resize_size=256
    crop_size=(224, 224)
    mean=[0.485, 0.456, 0.406]
    std=[0.229, 0.224, 0.225]
    out = F.pil_to_tensor(img)
    out = F.resize
    (out, size=resize_size, interpolation = InterpolationMode.BILINEAR)
    out = F.center_crop(out, output_size=crop_size)
    out = out.to(torch.float, copy=False) / 255.
    out = F.normalize(out, mean, std)
    out = np.transpose(out.numpy(), axes=[1, 2, 0])
    return out

make_calib_man(
    pre_ftn=preprocess_resnet50, # callable function to pre-process the calibration
    data
    data_dir="/workspace/calibration/cali_1000", # path to folder of original
    calibration data files such as images
    save_dir="/workspace/calibration/", # path to folder to save pre-processed
    calibration data files
    save_name="resnet50_cali", # tag for the generated calibration dataset
    max_size=100 # Maximum number of data to use for calibration
)

```

5.4 모델 컴파일

변환된 모델과 Calibration Dataset이 준비되었다면 이제 모델을 컴파일할 수 있습니다. 예시에 사용된 ResNet50뿐만 아니라 TorchVision에서 제공되는 대부분의 이미지 분류 모델은 모빌린트의 컴파일러에서 100% 지원되며, 별도의 후처리 없이 작업에 사용할 수 있습니다.

```
""" Compile resnet """
from qbcompiler import mxq_compile
onnx_model_path = "/workspace/resnet50.onnx"
calib_data_path = "/workspace/calibration/resnet50_cali"

mxq_compile(
    model=onnx_model_path,
    calib_data_path=calib_data_path,
    quantization_method=1, # per channel quantization
    quantization_mode=1, # max percentile quantization
    percentile=0.999,
    quantization_output=0, # per layer quantization for the output layer
    save_path="resnet50.mxq",
    backend="onnx"
)
```

6 Ultralytics의 YOLO 모델

6.1 소개

You Only Look Once (YOLO)는 일련의 CNN 기반 실시간 물체 탐지 모델입니다. 첫 버전은 2015년에 발표되었고, 최신 버전(YOLO26)는 2026년 초에 발표되었습니다. 지난 10년동안 이 모델은 물체 탐지, 인스턴스 분할, 포즈 감지, 얼굴 탐지 등에 널리 사용되었습니다.

근래의 YOLO모델은 [Ultralytics](#)의 Python 패키지로 배포되고 있습니다. 이 튜토리얼에서는 Ultralytics가 공식 발표한 물체 탐지용 모델 중 하나인 YOLOv8n을 사용합니다.

6.2 모델 준비

모델을 다운로드하기에 앞서 Ultralytics의 패키지를 다음과 같이 설치합니다.

```
pip install ultralytics
```

그 후 다음의 CLI 명령어를 사용하여 모델을 다운로드합니다. 이 튜토리얼에서 우리는 YOLO모델의 기본 입력값 크기인 (640, 640)을 사용하기로 했기에 아래 명령어를 제외한 추가적인 입력이 필요하지 않습니다.

```
yolo export model=yolov8n.pt format=onnx
```

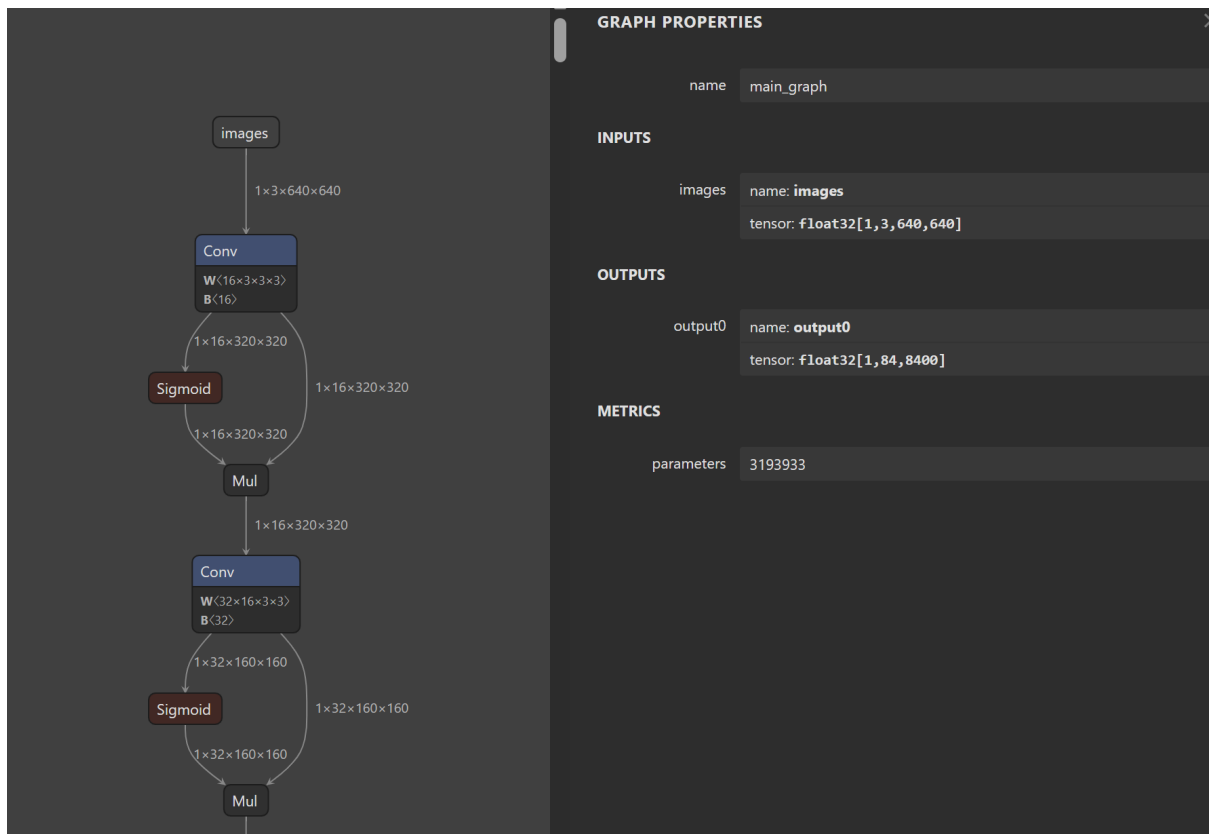


Figure 6.1 다운로드 된 YOLO 모델

6.3 Calibration Dataset 준비

물체 탐지, 인스턴스 분할을 위한 모델들은 보통 COCO 데이터셋을 사용하여 학습이 이루어집니다. 따라서 이 경우에도 학습에 사용된 이미지들과 유사한 분포를 가진 데이터셋을 Calibration Dataset으로 사용하는 것이 권장됩니다. 이 튜토리얼을 위해 모빌린트의 엔지니어들은 COCO의 학습용 데이터셋으로부터 100개의 이미지를 추출해서 제공하고 있습니다.

YOLO모델의 경우 버전에 따라 전처리과정이 상이할 수 있습니다. 이러한 이유로 표준화된 전처리 연산들을 조합하여 Calibration Dataset을 만드는 방식을 사용하기 곤란한 경우가 대부분입니다. 따라서 이 섹션에서는 사용자 정의 함수를 사용하는 예시만을 제공합니다. Ultralytics의 경우 전처리에 사용한 코드를 [GitHub](#)에 공개하고 있기에, 해당 코드를 바탕으로 다음과 같이 이용하실 수 있습니다.

```
import cv2
import numpy as np
from qbcompiler.calibration import make_calib_man

img_size = [640, 640] # or [1280, 1280]

def preprocess_yolo(img_path: str):
    img = cv2.imread(img_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    h0, w0 = img.shape[:2] # original hw
    r = min(img_size[0] / h0, img_size[1] / w0) # ratio
    new_unpad = int(round(w0 * r)), int(round(h0 * r))
    dh, dw = (
        img_size[0] - new_unpad[1],
        img_size[1] - new_unpad[0],
    ) # wh padding

    dw /= 2 # divide padding into 2 sides
    dh /= 2

    if not (img.shape[1], img.shape[0]) == new_unpad:
        img = cv2.resize(img, new_unpad, interpolation=cv2.INTER_LINEAR)
        top, bottom = int(round(dh - 0.1)), int(round(dh + 0.1))
        left, right = int(round(dw - 0.1)), int(round(dw + 0.1))
        img = cv2.copyMakeBorder(
            img, top, bottom, left, right, cv2.BORDER_CONSTANT, value=(114, 114, 114)
        ) # add border
    img = (img / 255).astype(np.float32)

    return img

make_calib_man(
    pre_ftn=preprocess_yolo, # callable function to pre-process the calibration
    data
    data_dir="/workspace/calibration/train2014_calib", # path to folder of original
    calibration data files such as images
    save_dir="/workspace/calibration/", # path to folder to save pre-processed
    calibration data files
    save_name="yolov8n_cali", # tag for the generated calibration dataset
    max_size=100, # Maximum number of data to use for calibration
)
```

6.4 모델 컴파일

변환된 모델과 Calibration Dataset이 준비되었다면 이제 모델을 컴파일할 수 있습니다.

```

""" Compile YOLO """
from qbcompiler import mxq_compile
onnx_model_path = "/workspace/yolov8n.onnx"
calib_data_path = "/workspace/calibration/yolov8n_cali"

mxq_compile(
    model=onnx_model_path,
    calib_data_path=calib_data_path,
    quantization_method=1, # per channel quantization
    quantization_mode=1, # max percentile quantization
    percentile=0.999,
    quantization_output=1, # per channel quantization for the output layer
    save_path="yolov8n.mxq",
    backend="onnx"
)

```

한 가지 유의할 점으로는, YOLO 모델에 포함된 일부 후처리 연산들은 NPU상에서 지원되지 않을 수 있거나, 연산의 정확도가 낮을 수 있습니다. 이 경우 qbcompiler 별도의 설정이 없는 한, 지원되지 않는 부분을 잘라내서 컴파일하게 됩니다. 따라서 컴파일 결과에 표시되는 연산 지원 범위를 확인하고, 지원되지 않는 연산은 별도의 후처리 함수를 외부에 연결하여 작업을 수행할 수 있도록 해야합니다. 지원되지 않는 연산에 대한 후처리 연결 방식은 [Mobilint Model Zoo](#)를 통해 배포되는 모델들의 예시를 참고하여 해결하실 수 있습니다.

```

[2025-09-10 13:53:52.457] random input name: images_0, input shape: [1, 640, 640, 3], data type: float32

===== Input Information =====
Input Name      = images_0
Input Shape     = [640, 640, 3]
Reshaped Shape  = [640, 640, 3]
=====

===== Output Information =====
Output Name     = /model.22/cv2.2/cv2.2/Conv_output_0
Output Shape    = [20, 20, 64]
Output Name     = /model.22/cv3.2/cv3.2/Conv_output_0
Output Shape    = [20, 20, 80]
Output Name     = /model.22/cv2.1/cv2.1/Conv_output_0
Output Shape    = [40, 40, 64]
Output Name     = /model.22/cv3.1/cv3.1/Conv_output_0
Output Shape    = [40, 40, 80]
Output Name     = /model.22/cv2.0/cv2.0/Conv_output_0
Output Shape    = [80, 80, 64]
Output Name     = /model.22/cv3.0/cv3.0/Conv_output_0
Output Shape    = [80, 80, 80]
=====

```

Figure 6.2 YOLO 컴파일 결과

7 HuggingFace로 배포된 Llama 모델

7.1 소개

생성형 AI가 발전함에 따라 대형 언어 모델(LLM)은 우리 일상의 한 부분이 되었습니다. 이로 인해 최근 사람들은 ChatGPT, Llama, Gemini, Qwen, Mistral, Exaone, DeepSeek, Claude, Clova, Grok 등 무수히 많은 모델들 중 적어도 하나 이상을 사용해본 경험이 있습니다.

이러한 시류에 맞춰 qbcompiler는 LLM을 컴파일하고, 모빌린트의 엣지 디바이스에서 사용할 수 있도록 지원하고 있습니다. 이 섹션에서는 Meta의 [Llama](#) 버전 3.2, 1B 크기의 경량 모델 중 Instruction Tuning을 거친 모델을 사용하여 그 방법에 대해 알아봅니다.

7.2 모델 준비

현재 LLM은 AI 모델 배포 플랫폼인 [HuggingFace](#)를 통해 공유되고 있습니다. 따라서 이 섹션에서의 모델 준비 과정은 HuggingFace의 transformers형식으로 배포되는 모델을 준비하는 과정에 대해 설명합니다.

HuggingFace의 transformers 형식의 모델들은 다음과 같이 준비할 수 있습니다.

7.2.1 HuggingFace 인증

모델을 다운로드하기에 앞서 Huggingface 웹페이지를 방문하여 계정을 만들어야 합니다. 그 다음 사이드바를 클릭하여 토큰 관리 페이지로 들어가야 합니다.

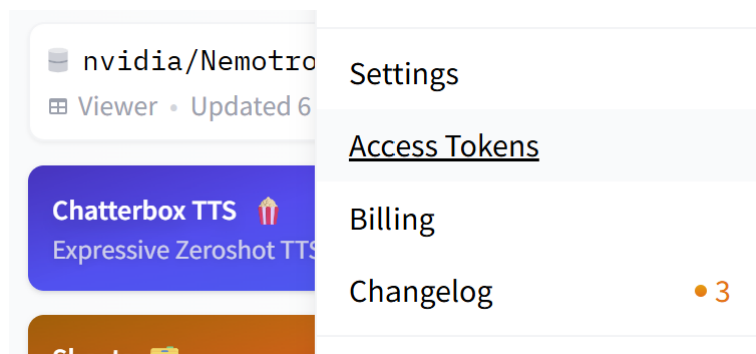


Figure 7.1 토큰 관리 페이지 열기

토큰 관리 페이지에서, 현재 생성된 토큰이 없다면 새로운 토큰을 생성해야 합니다. 모델을 다운받기 위해서는 별도의 설정 없이 읽기 전용 토큰만 생성해도 충분합니다.

Access Tokens

User Access Tokens

[+ Create new token](#)

Access tokens authenticate your identity to the Hugging Face Hub and allow applications to perform actions based on token permissions. **Do not share your Access Tokens with anyone;** we regularly check for leaked Access Tokens and remove them immediately.

Figure 7.2 새로운 토큰 생성

< Create new Access Token

Token type

Fine-grained **Read** Write

ⓘ This cannot be changed after token creation.

Token name

Token name

This token has read-only access to all your and your orgs resources and can be used to open pull requests and comment on discussions.

Create token

Figure 7.3 읽기 전용 토큰 생성

생성된 토큰을 복사하고, HuggingFace CLI 명령어를 실행하여 로그인합니다.

Access Tokens

User Access Tokens

+ Create new token

Access tokens authenticate your identity to the Hugging Face Hub and allow applications to perform actions based on token permissions. ⓘ **Do not share your Access Tokens with anyone**; we regularly check for leaked Access Tokens and remove them immediately.

Name	Value	Last Refreshed Date	Last Used Date	Permissions
CLI login	hf_...AbWv	less than a minute ago	7 days ago	READ ⋮ ↻ Invalidate and refresh 🗑 Delete

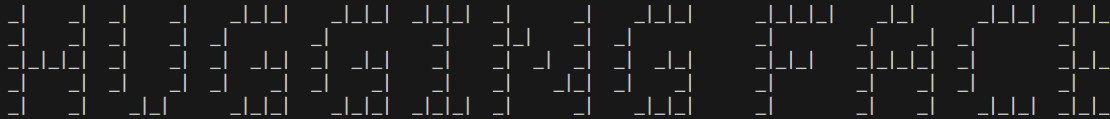
Figure 7.4 접속 토큰 복사

터미널에서 다음의 명령어를 실행하여 접근 권한을 인증합니다.

```
huggingface-cli login
```

로그인 과정에서 Git 인증서에 대한 질문에 대해 "예"라고 대답하는 것을 권장합니다. 만약 그랬을 때 다음과 같은 경고 메시지를 보게 된다면 경고 메시지에 적힌 지시사항을 수행하고 로그인 명령어를 다시 실행하여 절차를 한 번 더 진행해야 합니다.

```
root@274bd42e8b3e:/workspace/tutorial_test# huggingface-cli login
```



```
To log in, `huggingface_hub` requires a token generated from https://huggingface.co/settings/tokens .  
Enter your token (input will not be visible):  
Add token as git credential? (Y/n) y  
Token is valid (permission: read).  
Cannot authenticate through git-credential as no helper is defined on your machine.  
You might have to re-authenticate when pushing to the Hugging Face Hub.  
Run the following command in your terminal in case you want to set the 'store' credential helper as default.
```

```
git config --global credential.helper store
```

```
Read https://git-scm.com/book/en/v2/Git-Tools-Credential-Storage for more details.  
Token has not been saved to git credential helper.  
Your token has been saved to /root/.cache/huggingface/token  
Login successful
```

Figure 7.5 HuggingFace CLI 로그인

7.2.2 모델 다운로드

모델을 다운로드하기 위해서는 Git을 통해 대용량 파일을 관리할 수 있게 하는 Git LFS라는 도구를 설치해야 합니다. 다음의 명령어를 실행하여 설치할 수 있습니다.

```
apt-get update
apt-get install git-lfs
```

이제 [Llama-3.2-1B-Instruct](#) 페이지로 가서 요구된 양식을 작성합니다. 그 다음 "clone repository"라고 적힌 탭을 누르고, 보이는 지시사항에 따라 모델을 로컬 환경에 다운로드합니다.

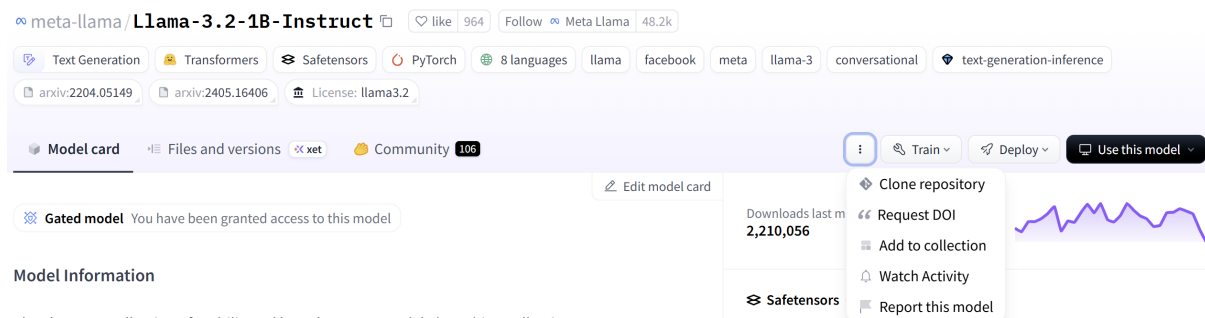


Figure 7.6 Llama-3.2-1B-Instruct 모델 페이지

7.3 Calibration Dataset 준비

모빌린트의 NPU상에서의 LLM 성능 향상을 위해, LLM의 Embedding 부분은 외부에서 처리하도록 구현되었습니다. 따라서 LLM을 컴파일하는 경우 문장에 대한 Token Tensor를 사용하는 것이 아니라 Embedded Tensor가 요구됩니다.

따라서 Calibration Dataset을 만들기 위해서도 외부에서 Embedding을 거치는 등의 추가적인 작업이 필요합니다. 아직 LLM에 대한 지원은 개발 중에 있기에 이 작업을 지원하는 도구들이 qbcompiler에 갖추어지지 않았으나, 추후 업데이트에 반영될 예정입니다.

7.3.1 Embedding Weight 추출

우선 해야할 것은 Embedding 레이어의 가중치를 추출하는 것입니다. transformers 형식으로 배포되는 모델들의 경우 "get_input_embeddings"라는 함수를 항상 구비하고 있고, 이 함수를 이용하면 손쉽게 가중치 텐서를 얻을 수 있습니다. 아래 코드는 그러한 방법으로 가중치 텐서를 "Llama-3.2-1B-instruct_embedding_weight.pt"라는 이름으로 저장합니다.

```
from transformers import AutoModelForCausalLM
import torch

model_dir = "/workspace/Llama-3.2-1B-Instruct" # path where the model was downloaded
model = AutoModelForCausalLM.from_pretrained(
    model_dir,
) # model
model.eval()

embedding_weight = model.get_input_embeddings().weight # (vocab_size, dim) tensor
torch.save(embedding_weight, model_dir.split("/")[-1] + "_embedding_weight.pt")
```

7.3.2 Calibration Dataset 생성

비전 모델에 대한 Calibration Dataset처럼, LLM의 Calibration Dataset도 실제 모델의 학습에 사용된 데이터와 유사한 입력값들로부터 생성하는 것이 권장됩니다. 이 경우 저희는 위키미디어 재단에서 공식 제공하는 다중 언어 데이터셋인 [Wikipedia](#) 데이터셋을 사용합니다.

아래 코드는 Wikipedia 데이터셋에 포함된 문서를 Embedded Tensor로 변환하는 과정을 구현하고 있습니다. 우선 문서를 Token Tensor로 변환, 지정된 범위의 사이즈로 조정하고, Embedding을 거쳐 Calibration Dataset을 생성합니다. 사용자는 아래 코드를 수정하여 Calibration Dataset에 사용되는 문장의 최대/최소 길이 및 전체 데이터 수를 조정할 수 있습니다.

```
from transformers import AutoTokenizer
from datasets import load_dataset
import torch
import numpy as np
import os

os.environ["CUDA_VISIBLE_DEVICES"] = "0" # set GPU device
model_dir_list = [ # tokenizer directories
    "/workspace/Llama-3.2-1B-Instruct",
]
embedding_dir_list = [ # corresponding embedding weight path
    "/workspace/Llama-3.2-1B-Instruct_embedding_weight.pt",
]

min_seqlen = 512 # sequence length minimum
max_seqlen = 2048 # sequence length maximum
max_calib = 128 # number of calibration

calib_dir = "/workspace"

for model_dir, embedding_dir in zip(model_dir_list, embedding_dir_list):
    tokenizer = AutoTokenizer.from_pretrained(
        model_dir,
        trust_remote_code=True,
    ) # tokenizer for model
    model_name = model_dir.split("/")[-1]

    subset_list = [ # subset for each language
```

```

        # "20231101.ko",
        # "20231101.en",
        # "20231101.ja",
        # "20231101.zh",
        # "20231101.fr",
        # "20231101.de",
        # "20231101.es",
        # "20231101.ru",
    ]

    for subset_name in subset_list:
        lang = subset_name.split(".")[1]
        dataset = load_dataset("wikimedia/wikipedia", subset_name, split="train")[
            "text"
        ]
        embedding_weight = torch.load(
            embedding_dir
        ) # (vocab_size, dim) embedding weight
        embedding_layer = torch.nn.Embedding.from_pretrained(embedding_weight).to(
            "cuda"
        ) # set embedding layer

        os.makedirs(
            os.path.join(calib_dir, f"{model_name}-Wikipedia-{lang}"), exist_ok=True
        ) # calibration save dir

        cur_num_calib = 0 # current number of calib
        for i, text in enumerate(dataset):
            print(f"Sentence {i}")
            token_ids = (
                tokenizer(text, return_tensors="pt")["input_ids"]
                .squeeze()
                .to("cuda")
            )
            print(token_ids.shape)
            if token_ids.ndim == 0: # empty
                continue
            embedded_text = embedding_layer(token_ids)
            if embedded_text.ndim == 1:
                embedded_text = embedded_text.unsqueeze(0).unsqueeze(0)
            elif embedded_text.ndim == 2:
                embedded_text = embedded_text.unsqueeze(0)
            print(embedded_text.shape)

            if embedded_text.shape[1] < min_seqlen:
                continue
            elif embedded_text.shape[1] > max_seqlen:
                embedded_text = embedded_text[:, :max_seqlen, :]

            np.save(
                os.path.join(calib_dir, f"{model_name}-Wikipedia-
                {lang}/inputs_embeds_{cur_num_calib}.np"),
                embedded_text.cpu().numpy(),
            )
            cur_num_calib += 1

            if cur_num_calib > max_calib - 1:
                break

```

7.4 모델 컴파일

HuggingFace transformers 형식으로 배포되는 모델을 컴파일하는 경우 `mxq_compile`은 추가적인 입력값들을 요구합니다. 우선 `"weight_dtype"` 변수를 원본 모델의 데이터 타입과 동일한 것으로 지정해주어야 합니다. 해당 타입은 원본 모델 폴더의 `"config.json"`에서 찾을 수 있습니다. 또한 LLM의 경우 연산량이 매우 많기 때문에 컴파일 시 GPU를 사용하는 것이 권장됩니다. 다만, GPU의 메모리 크기가 충분한지 확인이 필요합니다.

```
from qbcompiler import mxq_compile, get_llm_config

model_id = "meta-llama/Llama-3.2-1B-Instruct"
calib_path = "/workspace/Llama-3.2-1B-Instruct-Wikipedia-en" # folder of npy files
save_path = "./Llama-3.2-1B-Instruct.mxq"
hf_config = {
    "library": "transformers",
    "loader": "AutoModelForCausalLM",
    "tokenizer": "AutoTokenizer",
    "model_args": (),
    "model_kwargs": {
        "trust_remote_code": True,
    },
    "tokenizer_args": (),
    "tokenizer_kwargs": {},
}

# seqlen should be the same as cachelen on current version
llm_config = get_llm_config(
    llm_config_apply=True,
    max_sequence_length=4096,
    max_cache_length=4096,
    use_full_seq_length=True,
)

mxq_compile(
    model=model_id,
    calib_data_path=calib_path,
    save_path=save_path,
    backend="torch",
    quantization_output=0, # per layer quantization for the output layer
    hf_config=hf_config,
    use_gpu_only_for_calibration=True, # decrease GPU memory usage
    weight_dtype="bfloat16", # calibration weight data that should be same as dtype
    # of the original model
    llm_config=llm_config,
    device="gpu" # using GPU is recommended
)
```

```
===== Input Information =====
Input Name      = bridge0;model.embed_tokens_0
Input Shape     = [1, -1, 2048]
Reshaped Shape  = [1, -1, 2048]
=====

===== Output Information =====
Output Name     = lm_head
Output Shape    = [1, -1, 128256]
=====

[2025-06-17 10:31:07.703] Compiling the Quantized model...
[2025-06-17 10:31:08.165] Compile scheme: single
[2025-06-17 10:31:08.212] optimize network finished
[2025-06-17 10:31:08.213] incompatible constant activation size in weight
load_weights [*****] 100% layer731
[2025-06-17 10:33:25.683] optimize division finished
[2025-06-17 10:33:25.683] incompatible constant activation size in weight
[2025-06-17 10:33:38.387] done: attach parameters. cc00
[2025-06-17 10:34:28.563] Elaborating MXQ...
[2025-06-17 10:34:28.572] Exporting MXQ...
[2025-06-17 10:34:30.931] Verifying exported MXQ file...
[2025-06-17 10:34:32.043] Compilation was successful.
```

Figure 7.7 LLM 컴파일 결과

8 지원 프레임워크

qbcompiler는 ONNX, PyTorch, Keras, TensorFlow 및 TensorFlow Lite와 같은 일반적으로 사용되는 머신러닝 프레임워크 및 라이브러리들을 지원합니다.



Figure 8.1 지원 프레임워크

8.1 Mobilint IR Operations List

- 다음 목록은 Mobilint 하드웨어에서 네이티브로 지원되는 연산만 포함합니다. 명시적으로 나열되지 않은 연산도 컴파일 과정에서 그래프 수준 변환을 통해 지원 가능 연산으로 변환되는 경우 부분적으로 지원됩니다.
- Mobilint 하드웨어에서 지원되는 모든 연산은 배치 크기 1인 NHWC(channel-last) 데이터 포맷을 사용합니다.

연산	비고
AddPartial	num inputs: 2 num outputs: 1 attributes: - is_input0_larger: bool
AddingConstant	num inputs: 1 num outputs: 1 attributes: - constant: Weight - is_const_first: bool (default=false)
Adding	num inputs: 2 num outputs: 1
Batchnorm	num inputs: 1 num outputs: 1 attributes: - gamma: Weight - beta: Weight - moving_mean: Weight - moving_var: Weight - epsilon: Weight - channel_last: bool (default=true)
Celu	num inputs: 1 num outputs: 1 attributes: - alpha: float32
Clip	num inputs: 1 num outputs: 1 attributes: - clip_max_value: float32 (optional) - clip_min_value: float32 (optional)

연산	비고
Concatenate	num inputs: variable num outputs: 1 attributes: - original_key: str (optional) - axis: int32 (optional) - original_axis: int32 (optional) - original_inputs: int32[] (optional) - input_index: int32[] (optional) - input_names: str[] (optional) description: input_names: input activation names in original order. input_index: input index in computation order.
Convolution	num inputs: 1 num outputs: 1 attributes: - in_channels: int32 - out_channels: int32 - h_kernel: int32 - w_kernel: int32 - h_stride: int32 - w_stride: int32 - h_dilation: int32 - w_dilation: int32 - west_padding: int32 - east_padding: int32 - north_padding: int32 - south_padding: int32 - padding_list: int32[] (default=[]) - group_number: int32 - weight: Weight - bias: Weight (optional) - channel_last: bool (default=true)
DepthwiseConvolution	num inputs: 1 num outputs: 1 attributes: - in_channels: int32 - out_channels: int32 - h_kernel: int32 - w_kernel: int32 - h_stride: int32 - w_stride: int32 - h_dilation: int32 - w_dilation: int32 - west_padding: int32 - east_padding: int32 - north_padding: int32 - south_padding: int32 - padding_list: int32[] (default=[]) - group_number: int32 - weight: Weight - bias: Weight (optional) - channel_last: bool (default=true)

연산	비고
DeviceBridge	num inputs: 1 num outputs: 1 attributes: - squeeze_axes: int32[] - unsqueeze_axes: int32[] - permute_axes: int64[] - device: str (optional)
DivConstant	num inputs: 1 num outputs: 1 attributes: - constant: Weight - is_const_first: bool
Div	num inputs: 2 num outputs: 1
Elu	num inputs: 1 num outputs: 1 attributes: - leaky_alpha: float32
Erf	num inputs: 1 num outputs: 1
Exp	num inputs: 1 num outputs: 1
Gelu	num inputs: 1 num outputs: 1 attributes: - approximate: bool (default=false)
GLU	num inputs: 1 num outputs: 1
GroupConvolution	num inputs: 1 num outputs: 1 attributes: - in_channels: int32 - out_channels: int32 - h_kernel: int32 - w_kernel: int32 - h_stride: int32 - w_stride: int32 - h_dilation: int32 - w_dilation: int32 - west_padding: int32 - east_padding: int32 - north_padding: int32 - south_padding: int32 - padding_list: int32[] (default=[]) - group_number: int32 - weight: Weight - bias: Weight (optional) - channel_last: bool (default=true)

연산	비고
GroupNormalization	num inputs: 1 num outputs: 1 attributes: - num_groups: int32 - scale: Weight - bias: Weight (optional) - epsilon: Weight - channel_last: bool (default=true)
HardSigmoid	num inputs: 1 num outputs: 1 attributes: - alpha: float32 (optional) - beta: float32 (optional)
Hardtanh	num inputs: 1 num outputs: 1 attributes: - min_val: float32 (default=-1.0) - max_val: float32 (default=1.0)
HardSwish	num inputs: 1 num outputs: 1
HeaderView	num inputs: 1 num outputs: 1 attributes: - view_shape: int32[] (default=[]) - num_heads: int32 (default=-1) - batch_order: int32 (default=0) description: HeaderView simplifies the following sequence of operations. If batch_order == 0 (n,h,w,c)-[reshape]->(1, h, w, num_heads c/num_heads) -[transpose01324]->(1, h, num_heads, w, c/num_heads) -[reshape]->(1, h*num_heads, w, c/num_heads) If batch_order == 1 (n,h,w,c)-[reshape]->(1, h, w, num_heads, c/num_heads) -[transpose03124]->(1, num_heads, h, w, c/num_heads) -[reshape]->(1, num_heads*h, w, c/num_heads)

연산	비고
HeaderViewRevert	num inputs: 1 num outputs: 1 attributes: - revert_shape: int32[] (default=[]) - num_heads: int32 (default=-1) - batch_order: int32 (default=0) description: HeaderViewRevert simplifies the following sequence of operations. If batch_order == 0 (n,h,w,c)-[reshape]->(1, h/num_heads, num_heads, w, c) -[transpose01324]->(1, h/num_heads, w, num_heads, c) -[reshape]->(1, h/num_heads, w, num_heads * c) If batch_order == 1 (n,h,w,c)-[reshape]->(1, num_heads, h/num_heads, w, c) -[transpose02314]->(1, h/num_heads, w, num_heads, c) -[reshape]->(1, h/num_heads, w, num_heads*c)
Identity	num inputs: 1 num outputs: 1
InputConstant	num inputs: 0 num outputs: 1 attributes: - constant: Weight
Input	num inputs: 1 num outputs: 1

연산	비고
InstanceNormalization	num inputs: 1 num outputs: 1 attributes: - scale: Weight - bias: Weight (optional) - epsilon: Weight - channel_last: bool (default=true) - num_groups: int32 (default=-1) - group_order: int32 (default=-1) description: If num_groups and order are specified, the channel dimension is grouped and rearranged before normalization. For example, the following sequence: (n, 1, w, c*2)-[reshape]->(n, w, c, 2) -[transpose0312]->(n, 2, w, c) -[InstanceNorm, num_features=c]->(n, 2, w, c) -[transpose0231]->(n, w, c, 2) -[reshape]->(n, 1, w, c*2) is equivalent to a single operation: (n, 1, w, c*2)-[InstanceNormalization, num_features=c, num_groups=2, group_order=1]->(n, 1, w, c*2) The following sequence: (n, 1, w, 2*c)-[reshape]->(n,w,2,c) -[InstanceNorm, num_features=c]->(n, w, 2, c) -[reshape]v(n, 1, w, c*2) is equivalent to a single operation: (n, 1, w, 2*c)-[InstanceNormalization, num_features=c, num_groups=2, group_order=0]->(n, 1, w, 2*c)
L1Normalization	num inputs: 1 num outputs: 1 attributes: - norm_axis: int32[]
L2Normalization	num inputs: 1 num outputs: 1 attributes: - norm_axis: int32[] - epsilon: Weight (optional)

연산	비고
LSTM	num inputs: variable num outputs: 1 attributes: - W: Weight - R: Weight - B: Weight (optional) - lstm_group: str - input_mask: str - output_mask: str - P: Weight (optional) - direction: str (default="forward") - batch_first: bool (default=false) - hidden_size: int32 description: inputs:[input, sequence_lens, initial_h, initial_c] outputs:[output] or [hidden_out] or [cell_out] input_mask: "ishc" or "i", etc,... output_mask: "o" or "h" or "c"
LayerNormalization	num inputs: 1 num outputs: 1 attributes: - normalized_shape: int64[] (default=[]) - scale: Weight - bias: Weight (optional) - epsilon: Weight
Logit	num inputs: 1 num outputs: 1 attributes: - eps: float32 (optional) description: Compute $y = \ln(z / (1 - z))$. If eps is given, we clamp input $z = \text{clamp}(z, \text{min}=\text{eps}, \text{max}=1-\text{eps})$
RNN	num inputs: variable num outputs: 1 attributes: - W: Weight - R: Weight - B: Weight (optional) - rnn_group: str - input_mask: str - output_mask: str - direction: str (default="forward") - batch_first: bool (default=false) - hidden_size: int32 description: inputs:[input, sequence_lens, initial_h] outputs:[output] or [hidden_out] input_mask: "ish" or "i", etc,... output_mask: "o" or "h"

연산	비고
GRU	num inputs: variable num outputs: 1 attributes: - W: Weight - R: Weight - B: Weight (optional) - gru_group: str - input_mask: str - output_mask: str - direction: str (default="forward") - batch_first: bool (default=false) - linear_before_reset: bool (default=false) - hidden_size: int32 description: inputs:[input, sequence_lens, initial_h] outputs:[output] or [hidden_out] input_mask: "ish" or "i", etc,... output_mask: "o" or "h"
LeakyRelu	num inputs: 1 num outputs: 1 attributes: - leaky_alpha: float32
Log	num inputs: 1 num outputs: 1
LogSigmoid	num inputs: 1 num outputs: 1
MatMul	num inputs: variable num outputs: 1 attributes: - constant: Weight (optional) - is_const_first: bool (default=false) - repeat: int32[2] (default=[-1, -1]) description: If repeat=(0, 4), then input0 is repeated 4 times along the h-direction. For example, you can perform a matrix multiplication with input0 of shape (1, 8, 52, 64) and input1 of shape (1, 32, 52, 64) by concatenating input0 4 times, resulting in a input0 shape of (1, 32, 52, 64).
Mish	num inputs: 1 num outputs: 1
MultiplyConstant	num inputs: 1 num outputs: 1 attributes: - constant: Weight - is_const_first: bool (default=false)
Multiply	num inputs: 2 num outputs: 1

연산	비고
Neg	num inputs: 1 num outputs: 1
Output	num inputs: 1 num outputs: 1
Pad	num inputs: 1 num outputs: 1 attributes: - west_padding: int32 (default=-1) - east_padding: int32 (default=-1) - south_padding: int32 (default=-1) - north_padding: int32 (default=-1) - pad_mode: str - constant: float32 (default=0.0) - padding_list: int32[] (optional) description: padding_list format should be [x1_begin, x2_begin, ..., x1_end, x2_end,...], where xi_begin is the number of pad values added at the beginning of axis
Pooling	num inputs: 1 num outputs: 1 attributes: - h_kernel: int32 - w_kernel: int32 - h_stride: int32 - w_stride: int32 - h_dilation: int32 (default=1) - w_dilation: int32 (default=1) - west_padding: int32 - east_padding: int32 - north_padding: int32 - south_padding: int32 - padding_list: int32[] (default=[]) - pool_type: str - pad_mode: str - is_ceil_mode: bool (default=false) - is_include_pad: bool (default=false) - channel_last: bool (default=true)
GlobalAveragePooling	num inputs: 1 num outputs: 1 attributes: - channel_last: bool (default=true)
Pow	num inputs: 1 num outputs: 1 attributes: - exponent: float32
QuickGelu	num inputs: 1 num outputs: 1

연산	비고
ReduceL2	num inputs: 1 num outputs: 1 attributes: - keep_dims: bool - reduce_axes: int32[] - noop_with_empty_axes: int32
ReduceMax	num inputs: 1 num outputs: 1 attributes: - keep_dims: bool - reduce_axes: int32[] - noop_with_empty_axes: int32
ReduceMean	num inputs: 1 num outputs: 1 attributes: - keep_dims: bool - reduce_axes: int32[] - noop_with_empty_axes: int32
ReduceProd	num inputs: 1 num outputs: 1 attributes: - keep_dims: bool - reduce_axes: int32[] - noop_with_empty_axes: int32
ReduceSum	num inputs: 1 num outputs: 1 attributes: - keep_dims: bool - reduce_axes: int32[] - noop_with_empty_axes: int32
Relu	num inputs: 1 num outputs: 1
Reshape	num inputs: 1 num outputs: 1 attributes: - new_shape: int32[] (optional) - allowzero: int32 (default=0) - is_const_first: bool (default=false)

연산	비고
Resize	num inputs: 1 num outputs: 1 attributes: - resize_type: str (optional) - resize_mode: str (optional) - nearest_mode: str (optional) - transform_mode: str (optional) - roi: int32[] (optional) - scales: float32[] (optional) - sizes: int32[] (optional) - cubic_coeff_a: float32 (optional) - exclude_outside: int32 (optional) - extrapolation_value: float32 (optional) - antialias: int32 (optional) - axes: int32[] (optional) - keep_aspect_ratio_policy: str (optional)
RmsNormalization	num inputs: 1 num outputs: 1 attributes: - normalized_shape: int64[] (default=[]) - epsilon: Weight - scale: Weight
Rsqrt	num inputs: 1 num outputs: 1
Rstd	num inputs: 1 num outputs: 1 attributes: - reduce_axes: int32[] - epsilon: Weight - num_groups: int32 (default=1) description: computes $1/\text{std}(x)$. if groups > 1, then activation is grouped along channel axis.
Sigmoid	num inputs: 1 num outputs: 1 attributes: - alpha: float32 (default=1.0) description: alpha is added to deal with more general case like $1 / (1 + \exp(-\alpha * x))$
Slice	num inputs: 1 num outputs: 1 attributes: - axis: int32[] - start: int64[] - end: int64[] - step: int64[] (optional)

연산	비고
Softmax	num inputs: 1 num outputs: 1 attributes: - beta: float32 (default=1.0) - axis: int32
Softplus	num inputs: 1 num outputs: 1
Sqrt	num inputs: 1 num outputs: 1
StaggeredPadding	num inputs: 1 num outputs: 1 attributes: - axis: int32 (default=3) - reverse: bool (default=false) - window_size: int32 (default=-1) description: StaggeredPadding does the following: <pre> [[[1,1,1], [2,2,2], [3,3,3], [4,4,4], [5,5,5]]]] => [[[[1,1,1,0,0,0], [0,2,2,2,0,0,0], [0,0,3,3,3,0,0], [0,0,0,4,4,4,0], [0,0,0,0,5,5,5]]]] </pre> That is, each original row is shifted to the right so that its non-zero block lies on a "diagonal," and all other elements are zero-padded. Then slice last dim by (2*window_size + 1). For example, if window_size=1 in the above, 5=2*1+1 elements are sliced. <pre> => [[[[1,1,0,0,0], [2,2,2,0,0], [0,3,3,3,0], [0,0,4,4,4], [0,0,0,5,5]]]] </pre> If reverse <pre> [[[1,1,1], [2,2,2], [3,3,3], [4,4,4], [5,5,5]]]] => [[[[0,0,0,0,1,1,1], [0,0,0,2,2,2,0], [0,0,3,3,3,0,0], [0,4,4,4,0,0,0], [5,5,5,0,0,0,0]]]] </pre>

연산	비고
StatefulAttentionMaskWrapper	num inputs: 1 num outputs: 1 attributes: - past_seqlen: int32 (default=0) - is_sliding: bool (default=false) - sliding_window: int32 (default=1024) - mask_type: str (default="causal_mask") description: mask_type: "causal_mask", "padding_mask"
StatefulKVCacheWrapper	num inputs: 1 num outputs: 1 attributes: - cache_name: str - past_seqlen: int32 (default=0) - is_sliding: bool (default=false) - sliding_window: int32 (default=1024) - max_cache_len: int32 (default=8192)
StatefulCausalConvCacheWrapper	num inputs: 1 num outputs: 1 attributes: - cache_name: str - cache_len: int32 (default=0)
StatefulLSTMWrapper	num inputs: variable num outputs: 3 attributes: - W: Weight - R: Weight - B: Weight (optional) - input_mask: str - output_mask: str - P: Weight (optional) - direction: str (default="forward") - batch_first: bool (default=false) - hidden_size: int32 - hidden_state_name: str - cell_state_name: str - subgraph: str description: When input_mask is i shc, then inputs are [input, sequence↵ _lens, initial_h, initial_c], and if input_mask is i hc, then inputs are [input, initial_h, initial_c]. The output_mask is h means that among [output, hidden_out, cell_out], only the hidden_out is being used.

연산	비고
StatefulRNNWrapper	num inputs: variable num outputs: 2 attributes: - W: Weight - R: Weight - B: Weight (optional) - input_mask: str - output_mask: str - direction: str (default="forward") - batch_first: bool (default=false) - hidden_size: int32 - hidden_state_name: str - subgraph: str description: When input_mask is i sh, then inputs are [input, sequence_↵ lens, initial_h], and if input_mask is i h, then inputs are [input, initial_h]. The output_mask is h means that among [output, hidden_out], only the hidden_out is being used.
StatefulGRUWrapper	num inputs: variable num outputs: 2 attributes: - W: Weight - R: Weight - B: Weight (optional) - input_mask: str - output_mask: str - direction: str (default="forward") - batch_first: bool (default=false) - linear_before_reset: bool (default=false) - hidden_size: int32 - hidden_state_name: str - subgraph: str description: When input_mask is i sh, then inputs are [input, sequence_↵ lens, initial_h], and if input_mask is i h, then inputs are [input, initial_h]. The output_mask is h means that among [output, hidden_out], only the hidden_out is being used.
StatefulRoPEWrapper	num inputs: 3 num outputs: 1 attributes: - rope_type: int32 (default=0) - seqlen: int32 (default=0) - rotary_emb_dim: int32 (default=2) description: StatefulRoPEWrapper inputs: (q_or_k, con, sin) outputs: (q_or_k_embed) rope_type=> 0: llama, 1: cohere2(aya-vision)

연산	비고
SubConstant	num inputs: 1 num outputs: 1 attributes: - constant: Weight - is_const_first: bool
Sub	num inputs: 2 num outputs: 1
Swish	num inputs: 1 num outputs: 1
Tanh	num inputs: 1 num outputs: 1
TransposeConvolution	num inputs: 1 num outputs: 1 attributes: - in_channels: int32 - out_channels: int32 - h_kernel: int32 - w_kernel: int32 - h_stride: int32 - w_stride: int32 - h_dilation: int32 - w_dilation: int32 - west_padding: int32 - east_padding: int32 - north_padding: int32 - south_padding: int32 - padding_list: int32[] (default=[]) - og_padding_list: int32[] (default=[]) - og_out_padding_list: int32[] (default=[]) - group_number: int32 - weight: Weight - bias: Weight (optional) - channel_last: bool (default=true)
Transpose	num inputs: 1 num outputs: 1 attributes: - transpose_axes: int64[]
Weightedsum	num inputs: 2 num outputs: 1 attributes: - weight0: Weight - weight1: Weight

연산	비고
OrderedPatchify	num inputs: 1 num outputs: 1 attributes: - h_kernel: int32 - w_kernel: int32 - order: int32 (default=0) description: This operation splits a 4D input tensor into non-overlapping patches and rearranges them into a sequence according to the specified order. Supports row-major (order=0) and column-major (order=1) layouts. If order=0 (row-major), the equivalent is: 256 patches, each 12×12, (h_kernel=w_kernel=12) (1,192,192,64)- [reshape]-> (1,16,12,16,12,64)- [transpose 013245]->(1,16,16,12,12,64)-[reshape]->(1,256,144,64) Note that order0 is its own inverse. If order=1 (column-major), the equivalent is: 144 patches, each 16×16, (h_kernel=w_kernel=12) (1,192,192,64)- [reshape]-> (1,16,12,16,12,64)- [transpose 024135]->(1,12,12,16,16,64)-[reshape]->(1,144,256,64) if order=2 (reversing order=1), h_kernel=12, w_kernel=16 (1,144,256,64)- [reshape]-> (1,12,12,16,16,64)- [transpose 031425]->(1,16,12,16,12,64)-[reshape]->(1,192,192,64)

9 CPU Offloading (Beta Version)

Remark CPU Offloading을 지원하는 런타임이 설치된 경우에만, CPU Offloading을 지원하도록 컴파일된 MXQ 파일을 사용하여 추론을 수행할 수 있습니다.

qbcompiler v0.7부터 MXQ 컴파일을 위한 CPU Offloading(Beta 버전)을 제공합니다. CPU Offloading은 Mobilint NPU가 지원하지 않는 연산을 CPU로 자동으로 Offloading하여 사용자가 모델을 컴파일하는 과정을 용이하게 합니다. 예를 들어, 모델에 포함된 전처리 또는 후처리 작업이 NPU에서 지원하지 않는 연산을 포함하는 경우, 사용자는 컴파일 후 수동으로 구현해야 했습니다. 그러나 CPU Offloading은 이러한 대부분의 작업을 처리하여 추가 작업의 필요성을 줄여줍니다.

CPU Offloading이 적용된 경우 calibration 데이터셋 준비 및 모델 컴파일 절차는 약간 다릅니다. 다음과 같은 점에 유의해야 합니다:

1. 사전 처리된 입력 형태는 원래 모델의 입력 형태와 일치해야 합니다. CPU Offloading 없이 모델을 컴파일할 때는 사전 처리된 입력 형태는 (H, W, C) 형식이어야 합니다.
2. 함수 *mxq_compile*의 인수 *cpu_offload*를 True로 설정하여 CPU Offloading을 활성화해야 합니다.

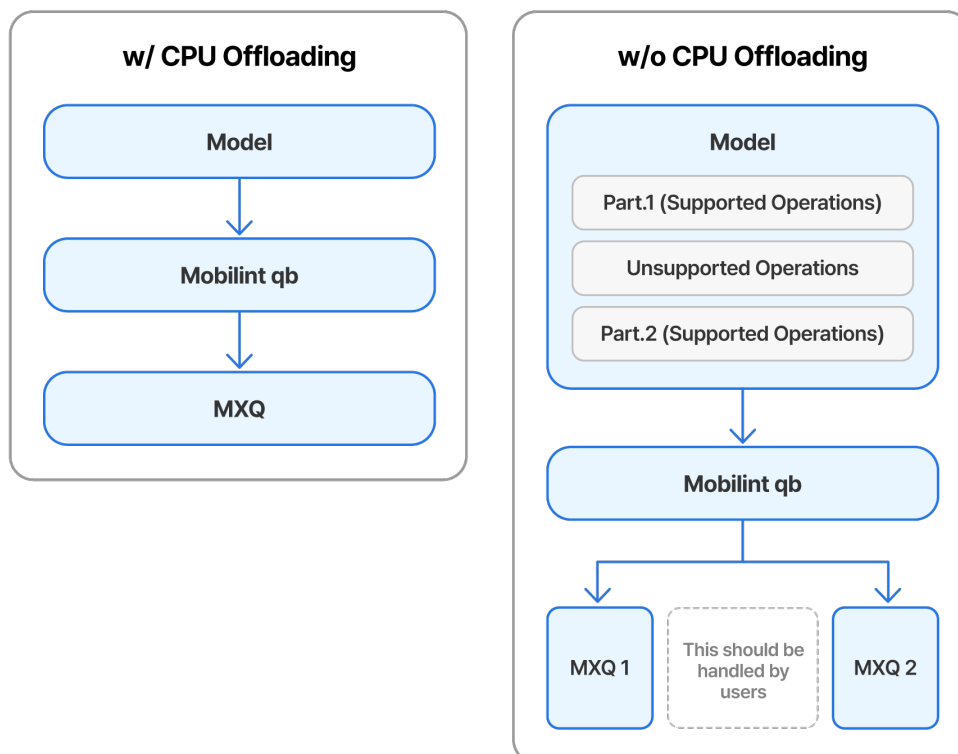


Figure 9.1 SDK CPU Offloading

10 Changelog

10.1 [v1.0.0] - 2026-01-30

10.1.1 API

- Introduced support for Qwen3 LLMs.
- Expanded compatibility to include the YOLO26 series.
- Removed the deprecated singlecore compile and startdramoffset options.
- Added support for custom masks and dynamic RoPE.
- Added NPU-accelerated execution for parts of the pre-processing pipeline, along with new input process configuration.
 - Added support for uint8 inputs.
 - Added NPU-based normalization.
- Added support for Torch tensors and image files as calibration data.
- Removed redundant quantization configuration options and unified Percentile, MSE, and KL into a histogram-based observer.
- Added support for dynamic core and memory allocation for all models.

10.2 [v0.12.0.0] - 2026-01-02

10.2.1 API

- Refactored compilation and quantization configuration.
- Optimized CPU memory usage during compilation.
- Added support for Transformers v4.57.1 (aligned with the 0.12 Mobilint Docker release).
- Added support for the MiniCPM model.

10.3 [v0.11.0.0] - 2025-09-10

10.3.1 API

- Added support for Torch parser
- Added support for Yolo12l and Yolo12x
- Expanded support range for ViT models
- Fixed minor bugs for GRU/RNN/LSTM

10.4 [v0.10.0.0] - 2025-07-24

10.4.1 API

- Added Inference_scheme global4/global8 modes
- Added support for YOLOv10 series, YOLO11 series, YOLO12n, YOLO12s, and YOLO12m
- Added LLM config options
- Added support for GRU/RNN/LSTM

10.5 [v0.9.0.5] - 2025-06-25

10.5.1 API

- Changed YOLO model decoding to be linked with the model zoo
- Added layer config options

10.6 [v0.9.0.4] - 2025-05-22

10.6.1 API

- Added support for direct compilation of HuggingFace LLM models

10.7 [v0.9.0.3] - 2025-04-02

10.7.1 API

- Added support for saving mbt using mmap
- Added support for diverse visualization types

10.8 [v0.9.0.2] - 2025-01-16

10.8.1 API

- Added API for saving model architecture
- Added support for Visualization tool
- Added support for Run min output difference QAT mode

10.9 [v0.9.0.1] - 2024-12-13

10.9.1 API

- Added API for global core (beta)
- Added API for YOLO post (beta)
- Added support for diverse hardware (aries, regulus, aries2)

10.10 [v0.9.0.0] - 2024-11-27

10.10.1 API

- Updated the high-level parsing processes

10.10.2 Docker

- ONNX: 1.13.0 -> 1.16.2
- TensorFlow: 2.9.0 -> 2.17.0
- Torch: 1.13.0 -> 2.4.1

10.11 [v0.8.5] - 2024-06-20

10.11.1 API

- Added support for FastPercentile quantization method

10.12 [v0.8.4] - 2024-05-20

10.12.1 API

- Connected TF backend to ONNX backend by TF2ONNX
- Enabled compilation of models with custom input shape
- Supported more operations

10.12.2 Docker

- ONNX: 1.12.0 -> 1.13.0

10.13 [v0.8.3] - 2024-03-07**10.13.1 API**

- Added support for TF Lite backend

10.14 [v0.8.2] - 2024-02-23**10.15 [v0.8.1] - 2023-12-08****10.16 [v0.8.0] - 2023-11-02****10.16.1 API**

- Deprecated TVM backend

10.17 [v0.7.12] - 2023-09-12**10.18 [v0.7.11] - 2023-08-31****10.18.1 API**

- Added support for TorchScript backend

10.19 [v0.7.10] - 2023-08-11**10.20 [v0.7.9] - 2023-08-11****10.21 [v0.7.8] - 2023-08-08****10.22 [v0.7] - 2023-03-23**

- Added multi-channel quantization
- Supported more operations

10.22.1 API

- Improved calibration dataset processing
- Added support for CPU offloading (beta version)

10.23 [v0.6] - 2022-08-10

- Made minor updates

10.24 [v0.5] - 2022-07-01

10.24.1 Docker

- Switched from Conda to Virtualenv
- Python: 3.7.7 -> 3.8.10
- Torch: 1.8.1 -> 1.10.1
- TensorFlow: 1.15.0 -> 2.3.0
- ONNX:1.6.0 -> 1.11.0

10.24.2 Parser

- Refactored code

10.24.3 API

- Enabled saving sample inference results (inputs and outputs)

10.25 [v0.4] - 2022-02-23

10.25.1 Optimizer

- Made minor updates in fusing reshape

10.26 [v0.3] - 2022-02-05

10.26.1 Parser

- Identified preprocess and postprocess of the model
- Excluded preprocess and postprocess if they were unsupported by the NPU

10.26.2 API

- Added integer inference simulation in the Python API

10.27 [v0.2] - 2021-12-01

- First release

11 Open Source License Notice

PyTorch

- <https://github.com/pytorch/pytorch>
- BSD-like License

TensorFlow

- <https://github.com/tensorflow/tensorflow>
- Apache 2.0 License

ONNX

- <https://github.com/onnx/onnx>
- Apache 2.0 License

ONNX Runtime

- <https://github.com/microsoft/onnxruntime>
- MIT License

Keras

- <https://github.com/keras-team/keras>
- Apache 2.0 License

12 Topic Index

12.1 API References

Here is a list of API references with brief descriptions:

API Reference	53
Compilation Configuration	62
Quantization Configuration	64
Resource Management Configuration	82
LLM Configuration	87
AdvancedQuantization Configuration	92
EquivalentTransformation Configuration	95
SearchWeightScale Configuration	109
Mod Configuration	113
Optq Configuration	122
Input Process Configuration	126

13 Hierarchical Index

13.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

qbcompiler.calibration.preprocess.CenterCrop	140
Compiler	
qbcompiler.frontend.Model_Dict	160
ConfigABC	
qbcompiler.configs.advanced_quantization_config.AdvancedQuantizationConfig	136
qbcompiler.configs.compile_config.CompileConfig	141
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig	149
qbcompiler.configs.input_process_config.InputProcessConfig	156
qbcompiler.configs.input_process_config.PreprocessingConfig	175
qbcompiler.configs.input_process_config.Uint8InputConfig	183
qbcompiler.configs.llm_config.LlmConfig	157
qbcompiler.configs.mod_config.ModConfig	158
qbcompiler.configs.optq_config.OptqConfig	168
qbcompiler.configs.quantization_config.BitConfig	137
qbcompiler.configs.quantization_config.CalibrationConfig	138
qbcompiler.configs.quantization_config.QuantizationConfig	176
qbcompiler.configs.resource_management_config.ResourceManagementConfig	180
qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig	181
qbcompiler.calibration.preprocess.GetImage	151
qbcompiler.calibration.preprocess.GetNpy	154
GetText	155
qbcompiler.calibration.preprocess.Normalize	163
qbcompiler.calibration.preprocess.NpyNormalize	165
qbcompiler.calibration.preprocess.NpyPad	166
qbcompiler.calibration.preprocess.Pad	169
qbcompiler.calibration.preprocess.preproc_builder	171
qbcompiler.calibration.preprocess.Resize	177
qbcompiler.calibration.preprocess.ResizeTorch	179
qbcompiler.calibration.preprocess.SetOrder	182
TokenizeToEmbedding	183
YoloPre	184

14 Class Index

14.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

qbcompiler.configs.advanced_quantization_config.AdvancedQuantizationConfig	
Configuration for advanced quantization options used during quantization	136
qbcompiler.configs.quantization_config.BitConfig	
Configuration for bit precision	137
qbcompiler.configs.quantization_config.CalibrationConfig	
Configuration for calibration during quantization	138
qbcompiler.calibration.preprocess.CenterCrop	
Perform a centered crop on the image	140
qbcompiler.configs.compile_config.CompileConfig	
Configuration for equivalent transformation techniques	141
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig	
Configuration for equivalent transformation techniques	149
qbcompiler.calibration.preprocess.GetImage	
Retrieve an image tensor from a file path or NumPy array using OpenCV	151
qbcompiler.calibration.preprocess.GetNpy	
Load NumPy data from an in-memory array or a .npy file	154
GetText	
Extract text strings from raw inputs	155
qbcompiler.configs.input_process_config.InputProcessConfig	
Unified configuration for input processing	156
qbcompiler.configs.llm_config.LlmConfig	
Configuration for Large Language Model (LLM) compilation	157
qbcompiler.configs.mod_config.ModConfig	
Configuration for Minimum Output Difference algorithm	158
qbcompiler.frontend.Model_Dict	
Wrapper around the Mobilint compiler to support compilation and inference workflows	160
qbcompiler.calibration.preprocess.Normalize	
Apply mean/std normalization to images	163
qbcompiler.calibration.preprocess.NpyNormalize	
Pad NumPy arrays to a desired shape	165
qbcompiler.calibration.preprocess.NpyPad	
Pad NumPy arrays to a desired shape	166
qbcompiler.configs.optq_config.OptqConfig	
Configuration for OPTQ algorithm	168
qbcompiler.calibration.preprocess.Pad	
Pad images to a target shape or to meet a size divisor	169
qbcompiler.calibration.preprocess.preproc_builder	
Build a calibration preprocessing pipeline from a YAML or dict configuration	171
qbcompiler.configs.input_process_config.PreprocessingConfig	
Configuration for input preprocessing pipeline	175
qbcompiler.configs.quantization_config.QuantizationConfig	
Unified quantization configuration	176
qbcompiler.calibration.preprocess.Resize	
Resize images with optional aspect-ratio preservation	177
qbcompiler.calibration.preprocess.ResizeTorch	
Configuration for resource management during model compilation	179
qbcompiler.configs.resource_management_config.ResourceManagementConfig	
Configuration for searching weight scale	180
qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig	
Configuration for searching weight scale	181
qbcompiler.calibration.preprocess.SetOrder	
Configuration for searching weight scale	182

TokenizeToEmbedding	
Convert tokenized text into embeddings using a Hugging Face model	183
qbcompiler.configs.input_process_config.Uint8InputConfig	
Configuration for uint8 input handling	183
YoloPre	
Apply YOLO-style resize, padding, and normalization to images	184

15 Topic Documentation

15.1 API Reference

Mobilint qb Compiler API provides functions for compiling models from various framework(onnx, pytorch, transformers, etc.) to MXQ format.

Classes

- class [qbcompiler.frontend.Model_Dict](#)
Wrapper around the Mobilint compiler to support compilation and inference workflows.
- class [qbcompiler.calibration.preprocess.GetNpy](#)
Load NumPy data from an in-memory array or a .npy file.
- class [qbcompiler.calibration.preprocess.NpyNormalize](#)
- class [qbcompiler.calibration.preprocess.NpyPad](#)
Pad NumPy arrays to a desired shape.
- class [qbcompiler.calibration.preprocess.GetImage](#)
Retrieve an image tensor from a file path or NumPy array using OpenCV.
- class [qbcompiler.calibration.preprocess.Pad](#)
Pad images to a target shape or to meet a size divisor.
- class [qbcompiler.calibration.preprocess.Normalize](#)
Apply mean/std normalization to images.
- class [qbcompiler.calibration.preprocess.ResizeTorch](#)
- class [qbcompiler.calibration.preprocess.Resize](#)
Resize images with optional aspect-ratio preservation.
- class [qbcompiler.calibration.preprocess.CenterCrop](#)
Perform a centered crop on the image.
- class [qbcompiler.calibration.preprocess.SetOrder](#)
- class [qbcompiler.calibration.preprocess.preproc_builder](#)
Build a calibration preprocessing pipeline from a YAML or dict configuration.
- class [GetText](#)
Extract text strings from raw inputs.
- class [TokenizeToEmbedding](#)
Convert tokenized text into embeddings using a Hugging Face model.
- class [YoloPre](#)
Apply YOLO-style resize, padding, and normalization to images.

Functions

- [qbcompiler.frontend.mxq_compile](#) (model, str calib_data_path="", int save_subgraph_type=0, output_subgraph_path="", str model_nickname=None, str save_path=None, backend="onnx", device="cpu", feed_dict=None, dynamic_axes=None, in_dformats=None, save_sample=False, inference_scheme="single", yolo_decode_include=False, use_random_calib=False, cpu_offload=False, optimize_option=1, sample_dtype="float", buffer_mode=1, input_shape_dict=dict

(`use_custom_mask=False`, `exclude_first_subgraph=False`, `Optional[CompileConfig]` `compile_config=None`, `Optional[ResourceManagementConfig]` `resource_management_config=None`, `Optional[QuantizationConfig]` `quantization_config=None`, `Optional[AdvancedQuantizationConfig]` `advanced_quantization_config=None`, `Optional[LlmConfig]` `llm_config=None`, `Optional[InputProcessConfig]` `input_process_config=None`, `**kwargs`)

Compile a model into a Mobilint eXeCutable (MXQ) package for execution on Mobilint NPUs.

- `qbcompiler.frontend.mblt_compile` (`str`|Any model, `str` `mblt_save_path`, `backend="onnx"`, `device="cpu"`, `feed_dict=None`, `dynamic_axes=None`, `in_dformats=None`, `yolo_decode_include=False`, `cpu_offload=False`, `use_custom_mask=False`, `exclude_first_subgraph=False`, `**kwargs`)

Export a model to the Mobilint .mblt format without producing an MXQ package.

- `qbcompiler.frontend.run_mblt_model` (`str` `path_to_mblt`, `bool` `save_all_outputs=True`, `Optional[str]` `inference_output_path=None`, `Optional[int]` `target_subgraph=None`)
- `qbcompiler.calibration.make_calib_data.get_yaml` (`str` `yaml_path`)
- `qbcompiler.calibration.make_calib_data.make_calib_man` (`pre_ftn`, `str` `data_dir`, `str` `save_dir`, `str` `save_name`, `str` `anno_json=None`, `str` `file_format="%012d.jpg"`, `int` `max_size=-1`, `bool` `remove_npy=False`, `int` `seed=2023`, `bool` `save_calib_msg=False`, `str` `msg_path=None`)

From images and a user-provided preprocessing function, generate calibration NumPy files and a manifest.

- `str` `qbcompiler.calibration.make_calib_data.make_calib` (`Union[str, dict]` `args_pre`, `str` `data_dir`, `str` `save_dir`, `str` `save_name=None`, `str` `anno_json=None`, `str` `file_format="%012d.jpg"`, `int` `max_size=-1`, `bool` `remove_npy=False`, `int` `seed=2023`, `bool` `save_calib_msg=False`, `str` `msg_path=None`)

Create calibration NumPy files and a manifest using a YAML or dict preprocessing configuration.

- `List[str]` `qbcompiler.calibration.make_calib_data.make_calib_llm` (`Union[str, dict]` `args_pre`, `str` `dataset_name`, `Union[List[str], str]` `subset_list`, `str` `text_field`, `str` `split`, `str` `save_dir`, `int` `max_calib=512`, `Optional[str]` `save_prefix=None`)

Generate calibration embeddings for LLM models using Hugging Face datasets.

- `qbcompiler.calibration.preprocess.preprocess_fnc` (`fnc`, `Datatype="Image"`)

Variables

- `dict` `qbcompiler.calibration.preprocess.DTYPE_GETDATA_MAP` = {"Image": "GetImage", "Npy": "GetNpy", "LLM": "GetText"}

15.1.1 Detailed Description

Mobilint qb Compiler API provides functions for compiling models from various framework(onnx, pytorch, transformers, etc.) to MXQ format.

15.1.2 Function Documentation

15.1.2.1 mxq_compile()

```
qbcompiler.frontend.mxq_compile (
    model,
    str  calib_data_path = "",
    int  save_subgraph_type = 0,
    str  output_subgraph_path = "",
    str  model_nickname = None,
```

```

str  save_path = None,
    backend = "onnx",
    device = "cpu",
    feed_dict = None,
    dynamic_axes = None,
    in_dformats = None,
    save_sample = False,
    inference_scheme = "single",
    yolo_decode_include = False,
    use_random_calib = False,
    cpu_offload = False,
    optimize_option = 1,
    sample_dtype = "float",
    buffer_mode = 1,
    input_shape_dict = dict(),
    use_custom_mask = False,
    exclude_first_subgraph = False,
Optional[CompileConfig]  compile_config = None,
Optional[ResourceManagementConfig]  resource_management_config = None,
Optional[QuantizationConfig]  quantization_config = None,
Optional[AdvancedQuantizationConfig]  advanced_quantization_config = None,
Optional[LlmConfig]  llm_config = None,
Optional[InputProcessConfig]  input_process_config = None,
** kwargs)

```

Compile a model into a Mobilint eXeCutable (MXQ) package for execution on Mobilint NPUs.

Parameters

model	string or model instance. Model path. When using backend="onnx", this should be the path to an ONNX model file. When backend is "torchscript", provide a TorchScript module; when "torch", provide a standard PyTorch model. For backend="tf", pass the directory that contains the TensorFlow SavedModel graph and assets. For backend="tflite", pass the TF Lite model path.
calib_data_path	string. Path to the calibration dataset. Accepts either a text/json file that lists NumPy files or a directory that contains the pre-processed NumPy files.
save_subgraph_type	int. Controls optional MBLT subgraph exports: 0 disables exports; 1 saves only the graph structure; 2 saves graph structure plus weights; 3 saves the graph structure split into multiple subgraphs; 4 saves both structure and weights split into multiple subgraphs.
output_subgraph_path	string. Destination path for the exported .mblt file when save_subgraph_type is 1, 2, 3, or 4. The resulting file can be used for visualization.
model_nickname	string. Friendly name stored in qbcompiler. Defaults to the model basename (for example, the file "/workspace/onnx/resnet50.onnx" becomes "resnet50"). Falls back to "temporary" when it cannot be inferred.
save_path	string. Output MXQ filename. Defaults to "{model_nickname}.mxq" when omitted.
backend	string. Framework used to generate the Mobilint IR. Must be one of "onnx", "tf", "tflite", "torchscript", or "torch". Defaults to "onnx".

Parameters

device	string. Compilation and inference device: "cpu" or "gpu". Defaults to "cpu".
feed_dict	dict. Example input tensors for shape inference and inference validation.
dynamic_axes	dict. Marks model axes as dynamic. Keys are input names and values map dimension indices to aliases (for example {"input": {2: "seq_len"}}).
in_dformats	dict. Describes input tensor data formats.
save_sample	bool. Stores input/output tensors in ./sampleInOut when True to aid debugging. Defaults to False.
inference_scheme	string. NPU inference scheme. One of "single", "multi", "global", "global4", or "global8".
yolo_decode_include	bool. Determines whether YOLO decode runs on NPU.
use_random_calib	bool. Generates random calibration data to validate model compilability. Defaults to False.
cpu_offload	bool. Enables CPU offloading during NPU inference. Defaults to False.
optimize_option	int. Compiler optimization strategy selector.
sample_dtype	string. Data type for saved sample inference outputs: "float" or "int8".
buffer_mode	int. Buffer serialization mode: 0 uses a naïve buffer, 1 uses an mmap-backed buffer. Defaults to 1.
input_shape_dict:multi-shape	compilation specification for STT/TTS models only (e.g., Conformer-CTC, MeloTTS). The dynamic axis is defined with respect to each input tensor's shape. During compilation, the same axis is varied across all model inputs using the provided values. Currently, only a single entry key ("multi_shape0") is supported. Example: {"multi_shape0": {"axis": 2, "values": [100, 200, 300]}}
use_custom_mask	bool. For non-LLM models that include a mask, use a custom mask provided as a separate input instead of the causal mask. For LLMs that incorporate RoPE, setting LLMConfig.dynamic_rope to True not only enables the use of dynamic RoPE provided via an additional input, but also activates the custom mask.
exclude_first_subgraph	bool. Applies only when CPU offloading is enabled: exclude the first subgraph from the final graph if it is unsupported.
compile_config	string. Path to JSON configuration file containing all quantization-related settings (resourceManagement, quantization, advancedQuantization, llmConfig). If provided, this takes precedence over individual sub-config parameters.
resource_management_config	ResourceManagementConfig. Resource management configuration object.
quantization_config	QuantizationConfig. Quantization configuration object.
advanced_quantization_config	AdvancedQuantizationConfig. Advanced quantization configuration object.
llm_config	LLMConfig. LLM configuration object.

Parameters

input_process_config	InputProcessConfig. Unified input processing configuration containing uint8 input handling, image channels, and preprocessing pipeline settings.
kwargs	dict. Additional compiler arguments propagated to the backend implementation. Moreover, If you want to override only a subset of configuration values, you can pass them to <code>mxq_compile</code> as keyword arguments (kwargs), and they will be applied. Note that <code>AdvancedQuantizationConfig</code> and its sub-configurations are not updated automatically from kwargs. In contrast, <code>ResourceManagementConfig</code> , <code>QuantizationConfig</code> , and <code>LlmConfig</code> support automatic updates via each configuration class's <code>from_kwargs</code> method. See Example 3 below for a usage example.

Returns

None.

Using configuration

There are two ways to configure quantization settings:

1. Load all settings from a JSON config file:

```
from qbcompiler import mxq_compile

mxq_compile(
    model="path/to/model.onnx",
    calib_data_path="path/to/calib",
    compile_config="path/to/config.json",
    device="gpu",
)
```

2. Pass individual sub-config objects:

```
from qbcompiler import mxq_compile
from qbcompiler.configs import (
    ResourceManagementConfig,
    QuantizationConfig,
    AdvancedQuantizationConfig,
    LlmConfig,
)

resource_mgmt = ResourceManagementConfig(weight_dtype="float32")
quant_cfg = QuantizationConfig(...)
adv_quant_cfg = AdvancedQuantizationConfig(...)
llm_cfg = LlmConfig(...)

mxq_compile(
    model="path/to/model.onnx",
    calib_data_path="path/to/calib",
    resource_management_config=resource_mgmt,
    quantization_config=quant_cfg,
    advanced_quantization_config=adv_quant_cfg,
    llm_config=llm_cfg,
    device="gpu",
)
```

3. Automatically applied partial configuration overrides:

```
from qbcompiler import mxq_compile

mxq_compile(
    model="path/to/model.onnx",
    calib_data_path="path/to/calib",
    quantization_method=1, # per channel quantization
    quantization_mode=1, # max percentile quantization
    percentile=0.999, # percentile value for max percentile quantization
    quantization_output=0, # per layer quantization for the output layer
    device="gpu",
)
```

Please refer to the `mxq_compile` function and the quantization configuration section for the meaning of the quantization-related numeric values.

Compiling models with custom inputs (ONNX/Torch/TensorFlow)

Provide NumPy inputs whenever the model omits shape information so that qbcompiler can infer unknown dimensions and data formats.

```
from qbcompiler import mxq_compile
import numpy as np

example_input = {
    "input_node_name_1": np.random.randn(1, 3, 224, 224).astype(np.float32),
    "input_node_name_2": np.random.randn(1, 8, 224, 224).astype(np.float32),
    "input_node_name_3": np.random.randn(1, 4, 56, 56).astype(np.float32),
}

in_dformats = {
    "input_node_name_1": "NCHW",
    "input_node_name_2": "NCHW",
    "input_node_name_3": "NCHW",
}

onnx_model_path = "path/to/your/model.onnx"
mxq_compile(
    model=onnx_model_path,
    feed_dict=example_input,
    in_dformats=in_dformats,
    backend="onnx",
    compile_config="path/to/config.json",
)
```

Definition at line 128 of file [frontend.py](#).

15.1.2.2 mblt_compile()

```
qbcompiler.frontend.mblt_compile (
    str | Any model,
    str mblt_save_path,
    backend = "onnx",
    device = "cpu",
    feed_dict = None,
    dynamic_axes = None,
```

```

    in_dformats = None,
    yolo_decode_include = False,
    cpu_offload = False,
    use_custom_mask = False,
    exclude_first_subgraph = False,
    **kwargs)

```

Export a model to the Mobilint .mblt format without producing an MXQ package.

Parameters

model	string or model instance. Source model or path to compile.
mblt_save_path	string. Output path for the .mblt artifact.
backend	string. Framework identifier ("onnx", "tf", "tflite", "torchscript", or "torch"). Defaults to "onnx".
device	string. Compilation device ("cpu" or "gpu"). Defaults to "cpu".
feed_dict	dict. Example inputs used for shape inference.
dynamic_axes	dict. Declares dynamic axes per input name.
in_dformats	dict. Input dataformat metadata.
yolo_decode_include	bool. Runs YOLO decode on NPU when True.
cpu_offload	bool. Enables CPU offloading for unsupported groups.
use_custom_mask	bool. For non-LLM models that include a mask, use a custom mask provided as a separate input instead of the causal mask. For LLMs that incorporate RoPE, setting LLMConfig.dynamic_rope to True not only enables the use of dynamic RoPE provided via an additional input, but also activates the custom mask.
exclude_first_subgraph	bool. Applies only when CPU offloading is enabled: exclude the first subgraph from the final graph if it is unsupported.
kwargs	dict. Additional arguments forwarded to the compiler.

Returns

None.

Definition at line [424](#) of file [frontend.py](#).

15.1.2.3 run_mblt_model()

```

qbcompiler.frontend.run_mblt_model (
    str path_to_mblt,
    bool save_all_outputs = True,
    Optional[str] inference_output_path = None,
    Optional[int] target_subgraph = None)

```

Definition at line [483](#) of file [frontend.py](#).

15.1.2.4 get_yaml()

```

qbcompiler.calibration.make_calib_data.get_yaml (
    str yaml_path)

```

Definition at line [29](#) of file [make_calib_data.py](#).

15.1.2.5 make_calib_man()

```
qbcompiler.calibration.make_calib_data.make_calib_man (
    pre_ftn,
    str data_dir,
    str save_dir,
    str save_name,
    str anno_json = None,
    str file_format = "%012d.jpg",
    int max_size = -1,
    bool remove_npy = False,
    int seed = 2023,
    bool save_calib_msg = False,
    str msg_path = None)
```

From images and a user-provided preprocessing function, generate calibration NumPy files and a manifest.

Parameters

pre_ftn	Callable. Pre-processing function that accepts an image path and returns a NumPy array.
data_dir	string. Directory containing the raw calibration images.
save_dir	string. Directory in which to store the generated NumPy files and metadata text file.
save_name	string. Basename for the saved assets. NumPy files are written to {save_dir}/{save_name}.npy and the manifest to {save_dir}/{save_name}.txt. Defaults to the basename of data_dir when omitted.
anno_json	string. Optional COCO-style annotation file. When provided, samples are randomly selected while maintaining class balance.
file_format	string. Filename format that uses image_idx (for example "%012d.jpg").
max_size	int. Maximum number of calibration samples. A value of -1 disables the limit.
remove_npy	bool. Remove existing NumPy outputs before generating new data.
seed	int. Random seed applied to sampling and class-balanced selection. Defaults to 2023.
save_calib_msg	bool. Persist the calibration metadata as an MSGpack file in addition to the manifest.
msg_path	string. Explicit MSGpack output path. When omitted, the path is auto-generated by dataset name and sample count.

Returns

string. Path to the generated text file that lists the preprocessed NumPy samples.

Note

Calibration samples are written as float32 NumPy arrays derived from the `pre_ftn` output. When `max_size` is positive and the dataset is larger, a random subset is used. COCO annotations override this sampling to balance categories.

Definition at line 40 of file [make_calib_data.py](#).

15.1.2.6 make_calib()

```
str qbcompiler.calibration.make_calib_data.make_calib (
    Union[str, dict] args_pre,
    str data_dir,
    str save_dir,
    str save_name = None,
    str anno_json = None,
    str file_format = "%012d.jpg",
    int max_size = -1,
    bool remove_npy = False,
    int seed = 2023,
    bool save_calib_msg = False,
    str msg_path = None)
```

Create calibration NumPy files and a manifest using a YAML or dict preprocessing configuration.

Parameters

args_pre	string or dict. Path to a YAML file or an in-memory dictionary describing the preprocessing pipeline. See the preproc_builder class for all supported operators and parameters.
data_dir	string. Directory of calibration images.
save_dir	string. Directory where the generated NumPy files and manifest text file are stored.
save_name	string. Optional basename for outputs. Defaults to the basename of <code>data_dir</code> .
anno_json	string. Optional COCO-format annotation file used to sample images with class balance.
file_format	string. Filename template built from <code>image_idx</code> (defaults to <code>"%012d.jpg"</code>).
max_size	int. Maximum number of calibration samples to generate; -1 keeps all available items.
remove_npy	bool. Remove existing NumPy outputs before running preprocessing.
seed	int. Random seed used for sampling. Defaults to 2023.
save_calib_msg	bool. Save calibration metadata as MSGpack alongside NumPy and text outputs.
msg_path	string. Explicit MSGpack output path. Auto-generated from dataset name and sample count when omitted.

Returns

string. Path to the manifest text file that lists the generated NumPy samples.

Note

The preprocessing pipeline is constructed through [preproc_builder](#) and executed via `make_calib_man`. Use the YAML schema to combine primitives such as `GetImage`, `Resize`, `Normalize`, `Pad`, `CenterCrop`, and `SetOrder` in a Pre-Order list.

Definition at line 139 of file [make_calib_data.py](#).

15.1.2.7 make_calib_llm()

```
List[str] qbcompiler.calibration.make_calib_data.make_calib_llm (
    Union[str, dict] args_pre,
    str dataset_name,
    Union[List[str], str] subset_list,
    str text_field,
    str split,
    str save_dir,
    int max_calib = 512,
    Optional[str] save_prefix = None)
```

Generate calibration embeddings for LLM models using Hugging Face datasets.

Args: args_pre: YAML path or preprocessing configuration dictionary. Must set Datatype to "LLM". dataset_name: Name of the Hugging Face dataset to load. subset_list: List of dataset subsets (e.g. language splits) to iterate. split: HF dataset split to use. text_field: Key containing text in dataset entries. save_dir: Root directory where calibration samples are stored. max_calib: Maximum number of calibration samples per subset. save_prefix: Override prefix used in output directory naming. If None, derives from tokenizer directory name when available.

Returns: List of paths to generated calibration txt files, one per subset.

Definition at line 200 of file [make_calib_data.py](#).

15.1.2.8 preprocess_fnc()

```
qbcompiler.calibration.preprocess.preprocess_fnc (
    fnc,
    Datatype = "Image")
```

Definition at line 28 of file [preprocess.py](#).

15.1.3 Variable Documentation

15.1.3.1 DTYPE_GETDATA_MAP

```
dict qbcompiler.calibration.preprocess.DTYPE_GETDATA_MAP = {"Image": "GetImage", "Npy": "GetNpy",
"LLM": "GetText"}
```

Definition at line 24 of file [preprocess.py](#).

15.2 Compilation Configuration

Compilation configuration definitions used by Mobilint compiler for compilation.

Classes

- class [qbcompiler.configs.compile_config.CompileConfig](#)

Functions

- [qbcompiler.configs.compile_config.get_compile_config](#) (calib_data_path=None, use_random_calib=False, save_path="/tmp.mxq", save_msgpack_name=None, model_nickname="temporary", model_path=None, inference_scheme="single", save_sample=False, cpu_offload=False, optimize_option=1, sample_dtype="float", buffer_mode=1, input_shape_dict=dict(), Optional[[ResourceManagementConfig](#)] resource_management_config=None, Optional[[QuantizationConfig](#)] quantization_config=None, Optional[[LlmConfig](#)] llm_config=None, Optional[[AdvancedQuantizationConfig](#)] advanced_quantization_config=None, Optional[[InputProcessConfig](#)] input_process_config=None, version=__version__, **kwargs)

Create Compilation Config with partial parameters merged with defaults.

15.2.1 Detailed Description

Compilation configuration definitions used by Mobilint compiler for compilation.

15.2.2 Function Documentation

15.2.2.1 get_compile_config()

```
qbcompiler.configs.compile_config.get_compile_config (
    calib_data_path = None,
    use_random_calib = False,
    save_path = "/tmp.mxq",
    save_msgpack_name = None,
    model_nickname = "temporary",
    model_path = None,
    inference_scheme = "single",
    save_sample = False,
    cpu_offload = False,
    optimize_option = 1,
    sample_dtype = "float",
    buffer_mode = 1,
    input_shape_dict = dict(),
    Optional[ResourceManagementConfig] resource_management_config = None,
    Optional[QuantizationConfig] quantization_config = None,
    Optional[LlmConfig] llm_config = None,
    Optional[AdvancedQuantizationConfig] advanced_quantization_config = None,
    Optional[InputProcessConfig] input_process_config = None,
    version = __version__,
    ** kwargs)
```

Create Compilation Config with partial parameters merged with defaults.

Definition at line [351](#) of file [compile_config.py](#).

15.3 Quantization Configuration

Quantization configuration definitions used by Mobilint compiler for model quantization.

Classes

- class `qbcompiler.configs.quantization_config.CalibrationConfig`
Configuration for calibration during quantization.
- class `qbcompiler.configs.quantization_config.BitConfig`
Configuration for bit precision.
- class `qbcompiler.configs.quantization_config.QuantizationConfig`
Unified quantization configuration.

Functions

- `qbcompiler.configs.quantization_config.CalibrationConfig.__init__` (self, int quantization_method, int quantization_output, int quantization_mode, numbers.Number percentile, numbers.Number topk_ratio, int max_each, int max_total, numbers.Number hist_percentile, bool hist_use_gpu, int hist_num_bins, int hist_num_samples, int hist_buffer_size, numbers.Number hist_min_bin_width, numbers.Number hist_search_percentile_min, numbers.Number hist_search_percentile_max, int hist_num_search, int hist_search_type, numbers.Number act_scale_min, numbers.Number weight_scale_min, numbers.Number min_clip_ratio, Dict layer_overrides, bool optional=optional)
Initialize the `CalibrationConfig`.
- `qbcompiler.configs.quantization_config.CalibrationConfig.default_config` (cls)
- `qbcompiler.configs.quantization_config.CalibrationConfig.from_kwargs` (cls, **kwargs)
- `qbcompiler.configs.quantization_config.CalibrationConfig.from_dict` (cls, dict config_dict)
Create `CalibrationConfig` from dictionary (JSON structure)
- `qbcompiler.configs.quantization_config.CalibrationConfig.to_dict` (self)
- `qbcompiler.configs.quantization_config.BitConfig.__init__` (self, int query_act_bits, int key_act_bits, int value_act_bits, int output_act_bits, int ffn_act_bits, int head_act_bits, int query_weight_bits, int key_weight_bits, int value_weight_bits, int output_weight_bits, int ffn_weight_bits, int head_weight_bits, bool mixed_precision_apply, str save_path, str load_path, List[str] activation_16bits, List[str] weight_16bits, bool optional=optional)
Initialize the `BitConfig`.
- `qbcompiler.configs.quantization_config.BitConfig.default_config` (cls)
- `qbcompiler.configs.quantization_config.BitConfig.from_kwargs` (cls, **kwargs)
- `qbcompiler.configs.quantization_config.BitConfig.from_dict` (cls, dict config_dict)
Create `BitConfig` from dictionary (JSON structure)
- `qbcompiler.configs.quantization_config.BitConfig.to_dict` (self)
- `qbcompiler.configs.quantization_config.QuantizationConfig.__init__` (self, bool optional, Optional[CalibrationConfig] calibration=None, Optional[BitConfig] bit=None)
Initialize the `QuantizationConfig`.
- `qbcompiler.configs.quantization_config.QuantizationConfig.default_config` (cls)
- `qbcompiler.configs.quantization_config.QuantizationConfig.from_kwargs` (cls, **kwargs)
- `qbcompiler.configs.quantization_config.QuantizationConfig.from_dict` (cls, dict config_dict)
- `qbcompiler.configs.quantization_config.QuantizationConfig.to_dict` (self)
- `CalibrationConfig qbcompiler.configs.quantization_config.get_calibration_config` (**kwargs)

Create **CalibrationConfig** with partial parameters merged with defaults.

- `BitConfig qbcompiler.configs.quantization_config.get_bit_config (**kwargs)`

Create **BitConfig** with partial parameters merged with defaults.

- `QuantizationConfig qbcompiler.configs.quantization_config.get_quantization_config` (Optional [**CalibrationConfig**] calibration=None, Optional[**BitConfig**] bit=None)

Create **QuantizationConfig** with partial parameters merged with defaults.

Variables

- `bool qbcompiler.configs.quantization_config.CalibrationConfig.optional = False`
- `list qbcompiler.configs.quantization_config.CalibrationConfig.method_list = ["WChALayer", "WChAMulti", "WChALayerZeropoint", "WChAMultiZeropoint"]`
- `list qbcompiler.configs.quantization_config.CalibrationConfig.output_list = ["Layer", "Ch", "Sigmoid"]`
- `list qbcompiler.configs.quantization_config.CalibrationConfig.mode_list = ["max", "maxPercentile", "histogram"]`
- `list qbcompiler.configs.quantization_config.CalibrationConfig.search_type_list = ["percentile", "mse", "kl"]`
- `dict qbcompiler.configs.quantization_config.CalibrationConfig.DEFAULTS`
- `dict qbcompiler.configs.quantization_config.CalibrationConfig.py_to_cpp_name`
- `dict qbcompiler.configs.quantization_config.CalibrationConfig.min_max_check`
- `qbcompiler.configs.quantization_config.CalibrationConfig.quantization_method = quantization_method`
- `qbcompiler.configs.quantization_config.CalibrationConfig.quantization_output = quantization_output`
- `qbcompiler.configs.quantization_config.CalibrationConfig.quantization_mode = quantization_mode`
- `qbcompiler.configs.quantization_config.CalibrationConfig.percentile = percentile`
- `qbcompiler.configs.quantization_config.CalibrationConfig.topk_ratio = topk_ratio`
- `qbcompiler.configs.quantization_config.CalibrationConfig.max_each = max_each`
- `qbcompiler.configs.quantization_config.CalibrationConfig.max_total = max_total`
- `qbcompiler.configs.quantization_config.CalibrationConfig.hist_percentile = hist_percentile`
- `qbcompiler.configs.quantization_config.CalibrationConfig.hist_use_gpu = hist_use_gpu`
- `qbcompiler.configs.quantization_config.CalibrationConfig.hist_num_bins = hist_num_bins`
- `qbcompiler.configs.quantization_config.CalibrationConfig.hist_num_samples = hist_num_samples`
- `qbcompiler.configs.quantization_config.CalibrationConfig.hist_buffer_size = hist_buffer_size`
- `qbcompiler.configs.quantization_config.CalibrationConfig.hist_min_bin_width = hist_min_bin_width`
- `qbcompiler.configs.quantization_config.CalibrationConfig.hist_search_percentile_min = hist_search_percentile_min`
- `qbcompiler.configs.quantization_config.CalibrationConfig.hist_search_percentile_max = hist_search_percentile_max`
- `qbcompiler.configs.quantization_config.CalibrationConfig.hist_num_search = hist_num_search`
- `qbcompiler.configs.quantization_config.CalibrationConfig.hist_search_type = hist_search_type`
- `qbcompiler.configs.quantization_config.CalibrationConfig.act_scale_min = act_scale_min`
- `qbcompiler.configs.quantization_config.CalibrationConfig.weight_scale_min = weight_scale_min`
- `qbcompiler.configs.quantization_config.CalibrationConfig.min_clip_ratio = min_clip_ratio`
- `qbcompiler.configs.quantization_config.CalibrationConfig.layer_overrides = layer_overrides`
- `bool qbcompiler.configs.quantization_config.BitConfig.optional = False`
- `dict qbcompiler.configs.quantization_config.BitConfig.DEFAULTS`

- `dict qbcompiler.configs.quantization_config.BitConfig.py_to_cpp_name`
- `dict qbcompiler.configs.quantization_config.BitConfig.min_max_check`
- `qbcompiler.configs.quantization_config.BitConfig.query_act_bits = query_act_bits`
- `qbcompiler.configs.quantization_config.BitConfig.key_act_bits = key_act_bits`
- `qbcompiler.configs.quantization_config.BitConfig.value_act_bits = value_act_bits`
- `qbcompiler.configs.quantization_config.BitConfig.output_act_bits = output_act_bits`
- `qbcompiler.configs.quantization_config.BitConfig.ffn_act_bits = ffn_act_bits`
- `qbcompiler.configs.quantization_config.BitConfig.head_act_bits = head_act_bits`
- `qbcompiler.configs.quantization_config.BitConfig.query_weight_bits = query_weight_bits`
- `qbcompiler.configs.quantization_config.BitConfig.key_weight_bits = key_weight_bits`
- `qbcompiler.configs.quantization_config.BitConfig.value_weight_bits = value_weight_bits`
- `qbcompiler.configs.quantization_config.BitConfig.output_weight_bits = output_weight_bits`
- `qbcompiler.configs.quantization_config.BitConfig.ffn_weight_bits = ffn_weight_bits`
- `qbcompiler.configs.quantization_config.BitConfig.head_weight_bits = head_weight_bits`
- `qbcompiler.configs.quantization_config.BitConfig.mixed_precision_apply = mixed_precision_apply`
- `qbcompiler.configs.quantization_config.BitConfig.save_path = save_path`
- `qbcompiler.configs.quantization_config.BitConfig.load_path = load_path`
- `qbcompiler.configs.quantization_config.BitConfig.activation_16bits = activation_16bits`
- `qbcompiler.configs.quantization_config.BitConfig.weight_16bits = weight_16bits`
- `bool qbcompiler.configs.quantization_config.QuantizationConfig.optional = False`
- `dict qbcompiler.configs.quantization_config.QuantizationConfig.py_to_cpp_name`
- `dict qbcompiler.configs.quantization_config.QuantizationConfig.min_max_check = {}`
- `qbcompiler.configs.quantization_config.QuantizationConfig.calibration = calibration or CalibrationConfig.default_config()`
- `qbcompiler.configs.quantization_config.QuantizationConfig.bit = bit or BitConfig.default_config()`

15.3.1 Detailed Description

Quantization configuration definitions used by Mobilint compiler for model quantization.

15.3.2 Function Documentation

15.3.2.1 `__init__()` [1/3]

```
qbcompiler.configs.quantization_config.CalibrationConfig.__init__(
    self,
    int quantization_method,
    int quantization_output,
    int quantization_mode,
    numbers.Number percentile,
    numbers.Number topk_ratio,
    int max_each,
    int max_total,
    numbers.Number hist_percentile,
    bool hist_use_gpu,
    int hist_num_bins,
    int hist_num_samples,
    int hist_buffer_size,
```

```

numbers.Number hist_min_bin_width,
numbers.Number hist_search_percentile_min,
numbers.Number hist_search_percentile_max,
int hist_num_search,
int hist_search_type,
numbers.Number act_scale_min,
numbers.Number weight_scale_min,
numbers.Number min_clip_ratio,
Dict layer_overrides,
bool optional = optional)

```

Initialize the [CalibrationConfig](#).

Parameters

quantization_method	int. Calibration method index: 0: WChALayer - Weight per-channel, Activation per-layer. 1: WChAMulti - Weight per-channel, Activation multi-layer. 2: WChALayerZeropoint - Weight per-channel, Activation per-layer with zeropoint. 3: WChAMultiZeropoint - Weight per-channel, Activation multi-layer with zeropoint.
quantization_output	int. Output quantization type index: 0: Layer - Per-layer quantization. 1: Ch - Per-channel quantization. 2: Sigmoid - Sigmoid-based quantization.
quantization_mode	int. Quantization mode index: 0: max. 1: maxPercentile. 2: histogram.
percentile	float. Percentile value for maxPercentile mode.
topk_ratio	float. Top-k ratio used in maxPercentile mode.
max_each	int. Maximum number of samples processed per iteration (maxPercentile).
max_total	int. Total maximum number of samples (maxPercentile).
hist_percentile	float. Percentile for histogram mode (when searchType is percentile).
hist_use_gpu	bool. Whether to use GPU for histogram computation.
hist_num_bins	int. Number of histogram bins.
hist_num_samples	int. Number of representative samples for metric computation.
hist_buffer_size	int. Buffer size for batch histogram computation.
hist_min_bin_width	float. Minimum bin width for histogram.
hist_search_percentile_min	float. Minimum search percentile for MSE/KL search.
hist_search_percentile_max	float. Maximum search percentile for MSE/KL search.
hist_num_search	int. Number of search iterations for MSE/KL search.
hist_search_type	int. Search type: 0=percentile, 1=mse, 2=kl.
act_scale_min	float. Minimum allowed activation scale (lower bound clamp).
weight_scale_min	float. Minimum allowed weight scale (lower bound clamp).
min_clip_ratio	float. Minimum clip ratio constraint applied during calibration.

Parameters

layer_overrides	Dict. Layer-specific override settings.
optional	bool. Indicates whether this config is optional.

Definition at line 106 of file [quantization_config.py](#).

15.3.2.2 default_config() [1/3]

```
qbcompiler.configs.quantization_config.CalibrationConfig.default_config (
    cls)
```

Definition at line 198 of file [quantization_config.py](#).

15.3.2.3 from_kwargs() [1/3]

```
qbcompiler.configs.quantization_config.CalibrationConfig.from_kwargs (
    cls,
    ** kwargs)
```

Definition at line 202 of file [quantization_config.py](#).

15.3.2.4 from_dict() [1/3]

```
qbcompiler.configs.quantization_config.CalibrationConfig.from_dict (
    cls,
    dict config_dict)
```

Create [CalibrationConfig](#) from dictionary (JSON structure)

Definition at line 218 of file [quantization_config.py](#).

15.3.2.5 to_dict() [1/3]

```
qbcompiler.configs.quantization_config.CalibrationConfig.to_dict (
    self)
```

Definition at line 283 of file [quantization_config.py](#).

15.3.2.6 __init__() [2/3]

```
qbcompiler.configs.quantization_config.BitConfig.__init__ (
    self,
    int query_act_bits,
    int key_act_bits,
    int value_act_bits,
    int output_act_bits,
    int ffn_act_bits,
    int head_act_bits,
    int query_weight_bits,
    int key_weight_bits,
    int value_weight_bits,
    int output_weight_bits,
    int ffn_weight_bits,
    int head_weight_bits,
    bool mixed_precision_apply,
    str save_path,
    str load_path,
    List[str] activation_16bits,
    List[str] weight_16bits,
    bool optional = optional)
```

Initialize the [BitConfig](#).

Parameters

query_act_bits	int. Query activation bit-width.
key_act_bits	int. Key activation bit-width.
value_act_bits	int. Value activation bit-width.
output_act_bits	int. Output activation bit-width.
ffn_act_bits	int. FFN activation bit-width.
head_act_bits	int. Head activation bit-width.
query_weight_bits	int. Query weight bit-width.
key_weight_bits	int. Key weight bit-width.
value_weight_bits	int. Value weight bit-width.
output_weight_bits	int. Output weight bit-width.
ffn_weight_bits	int. FFN weight bit-width.
head_weight_bits	int. Head weight bit-width.
mixed_precision_apply	bool. If true, apply mixed-precision according to the specified bit-widths.
save_path	str. (optional) Path to save the bit allocation. If empty, the allocation is not saved.
load_path	str. (optional) Path to load the bit allocation. If empty, allocation is not loaded.
activation_16bits	List[str]. (optional) Layer names to force 16-bit activations.
weight_16bits	List[str]. (optional) Layer names to force 16-bit weights.

Definition at line [378](#) of file [quantization_config.py](#).

15.3.2.7 default_config() [2/3]

```
qbcompiler.configs.quantization_config.BitConfig.default_config (
    cls)
```

Definition at line 441 of file [quantization_config.py](#).

15.3.2.8 from_kwargs() [2/3]

```
qbcompiler.configs.quantization_config.BitConfig.from_kwargs (
    cls,
    ** kwargs)
```

Definition at line 445 of file [quantization_config.py](#).

15.3.2.9 from_dict() [2/3]

```
qbcompiler.configs.quantization_config.BitConfig.from_dict (
    cls,
    dict config_dict)
```

Create [BitConfig](#) from dictionary (JSON structure)

Definition at line 461 of file [quantization_config.py](#).

15.3.2.10 to_dict() [2/3]

```
qbcompiler.configs.quantization_config.BitConfig.to_dict (
    self)
```

Definition at line 525 of file [quantization_config.py](#).

15.3.2.11 __init__() [3/3]

```
qbcompiler.configs.quantization_config.QuantizationConfig.__init__ (
    self,
    bool optional,
    Optional[CalibrationConfig] calibration = None,
    Optional[BitConfig] bit = None)
```

Initialize the [QuantizationConfig](#).

Parameters

calibration	CalibrationConfig . Calibration configuration.
bit	BitConfig . Bit precision configuration.

Definition at line 582 of file [quantization_config.py](#).

15.3.2.12 default_config() [3/3]

```
qbcompiler.configs.quantization_config.QuantizationConfig.default_config (  
    cls)
```

Definition at line 598 of file [quantization_config.py](#).

15.3.2.13 from_kwargs() [3/3]

```
qbcompiler.configs.quantization_config.QuantizationConfig.from_kwargs (  
    cls,  
    ** kwargs)
```

Definition at line 606 of file [quantization_config.py](#).

15.3.2.14 from_dict() [3/3]

```
qbcompiler.configs.quantization_config.QuantizationConfig.from_dict (  
    cls,  
    dict config_dict)
```

Definition at line 614 of file [quantization_config.py](#).

15.3.2.15 to_dict() [3/3]

```
qbcompiler.configs.quantization_config.QuantizationConfig.to_dict (  
    self)
```

Definition at line 624 of file [quantization_config.py](#).

15.3.2.16 get_calibration_config()

```
CalibrationConfig qbcompiler.configs.quantization_config.get_calibration_config (  
    ** kwargs)
```

Create [CalibrationConfig](#) with partial parameters merged with defaults.

Definition at line 633 of file [quantization_config.py](#).

15.3.2.17 get_bit_config()

```
BitConfig qbcompiler.configs.quantization_config.get_bit_config (  
    ** kwargs)
```

Create [BitConfig](#) with partial parameters merged with defaults.

Definition at line 642 of file [quantization_config.py](#).

15.3.2.18 get_quantization_config()

```
QuantizationConfig qbcompiler.configs.quantization_config.get_quantization_config (
    Optional[CalibrationConfig] calibration = None,
    Optional[BitConfig] bit = None)
```

Create [QuantizationConfig](#) with partial parameters merged with defaults.

Definition at line [651](#) of file [quantization_config.py](#).

15.3.3 Variable Documentation

15.3.3.1 optional [1/3]

```
bool qbcompiler.configs.quantization_config.CalibrationConfig.optional = False [static]
```

Definition at line [22](#) of file [quantization_config.py](#).

15.3.3.2 method_list

```
qbcompiler.configs.quantization_config.CalibrationConfig.method_list = ["WChALayer", "WChAMulti",
"WChALayerZeropoint", "WChAMultiZeropoint"] [static]
```

Definition at line [23](#) of file [quantization_config.py](#).

15.3.3.3 output_list

```
qbcompiler.configs.quantization_config.CalibrationConfig.output_list = ["Layer", "Ch", "Sigmoid"]
[static]
```

Definition at line [24](#) of file [quantization_config.py](#).

15.3.3.4 mode_list

```
qbcompiler.configs.quantization_config.CalibrationConfig.mode_list = ["max", "maxPercentile", "his-
togram"] [static]
```

Definition at line [25](#) of file [quantization_config.py](#).

15.3.3.5 search_type_list

```
list qbcompiler.configs.quantization_config.CalibrationConfig.search_type_list = ["percentile",
"mse", "kl"] [static]
```

Definition at line [26](#) of file [quantization_config.py](#).

15.3.3.6 DEFAULTS [1/2]

qbcompiler.configs.quantization_config.CalibrationConfig.DEFAULTS [static]

Initial value:

```
= {
    "quantization_method": 1,
    "quantization_output": 0,
    "quantization_mode": 1, # maxPercentile is now index 1
    # maxPercentile config
    "percentile": 0.9999,
    "topk_ratio": 0.01,
    "max_each": 128,
    "max_total": 65536,
    # histogram config
    "hist_percentile": 0.9999,
    "hist_use_gpu": True,
    "hist_num_bins": 2048,
    "hist_num_samples": 1024,
    "hist_buffer_size": 1024,
    "hist_min_bin_width": 0.0001,
    "hist_search_percentile_min": 0.9999,
    "hist_search_percentile_max": 0.999999,
    "hist_num_search": 1024,
    "hist_search_type": 0, # 0=percentile, 1=mse, 2=kl
    # common
    "act_scale_min": 0.0005,
    "weight_scale_min": 1e-6,
    "min_clip_ratio": -1,
    "layer_overrides": {"actScaleMin": {"0.0005": []}},
}
```

Definition at line 28 of file [quantization_config.py](#).

15.3.3.7 py_to_cpp_name [1/3]

dict qbcompiler.configs.quantization_config.CalibrationConfig.py_to_cpp_name [static]

Initial value:

```
= {
    "quantization_method": "method",
    "quantization_output": "output",
    "quantization_mode": "mode",
    # maxPercentile
    "percentile": "percentile",
    "topk_ratio": "topKRatio",
    "max_each": "maxEach",
    "max_total": "maxTotal",
    # histogram
    "hist_percentile": "percentile",
    "hist_use_gpu": "useGPU",
    "hist_num_bins": "numBins",
    "hist_num_samples": "numSamples",
    "hist_buffer_size": "bufferSize",
    "hist_min_bin_width": "minBinWidth",
    "hist_search_percentile_min": "searchPercentileMin",
}
```

```

    "hist_search_percentile_max": "searchPercentileMax",
    "hist_num_search": "numSearch",
    "hist_search_type": "searchType",
    # common
    "act_scale_min": "actScaleMin",
    "weight_scale_min": "weightScaleMin",
    "min_clip_ratio": "minclipRatio",
}

```

Definition at line 55 of file [quantization_config.py](#).

15.3.3.8 min_max_check [1/3]

dict qbcompiler.configs.quantization_config.CalibrationConfig.min_max_check [static]

Initial value:

```

= {
    "quantization_method": [0, 3],
    "quantization_output": [0, 2],
    "quantization_mode": [0, 2], # 0=max, 1=maxPercentile, 2=histogram
    # maxPercentile
    "percentile": [0, 1],
    "topk_ratio": [0, 1],
    "max_each": [0, None],
    "max_total": [0, None],
    # histogram
    "hist_percentile": [0, 1],
    "hist_num_bins": [1, None],
    "hist_num_samples": [1, None],
    "hist_buffer_size": [1, None],
    "hist_min_bin_width": [0, None],
    "hist_search_percentile_min": [0, 1],
    "hist_search_percentile_max": [0, 1],
    "hist_num_search": [1, None],
    "hist_search_type": [0, 2], # 0=percentile, 1=mse, 2=kl
    # common
    "act_scale_min": [0, 1],
    "weight_scale_min": [0, 1],
    "min_clip_ratio": [-1, 1],
}

```

Definition at line 81 of file [quantization_config.py](#).

15.3.3.9 quantization_method

qbcompiler.configs.quantization_config.CalibrationConfig.quantization_method = quantization_method

Definition at line 171 of file [quantization_config.py](#).

15.3.3.10 quantization_output

qbcompiler.configs.quantization_config.CalibrationConfig.quantization_output = quantization_output

Definition at line 172 of file [quantization_config.py](#).

15.3.3.11 quantization_mode

```
qbcompiler.configs.quantization_config.CalibrationConfig.quantization_mode = quantization_mode
```

Definition at line 173 of file [quantization_config.py](#).

15.3.3.12 percentile

```
qbcompiler.configs.quantization_config.CalibrationConfig.percentile = percentile
```

Definition at line 175 of file [quantization_config.py](#).

15.3.3.13 topk_ratio

```
qbcompiler.configs.quantization_config.CalibrationConfig.topk_ratio = topk_ratio
```

Definition at line 176 of file [quantization_config.py](#).

15.3.3.14 max_each

```
qbcompiler.configs.quantization_config.CalibrationConfig.max_each = max_each
```

Definition at line 177 of file [quantization_config.py](#).

15.3.3.15 max_total

```
qbcompiler.configs.quantization_config.CalibrationConfig.max_total = max_total
```

Definition at line 178 of file [quantization_config.py](#).

15.3.3.16 hist_percentile

```
qbcompiler.configs.quantization_config.CalibrationConfig.hist_percentile = hist_percentile
```

Definition at line 180 of file [quantization_config.py](#).

15.3.3.17 hist_use_gpu

```
qbcompiler.configs.quantization_config.CalibrationConfig.hist_use_gpu = hist_use_gpu
```

Definition at line 181 of file [quantization_config.py](#).

15.3.3.18 hist_num_bins

```
qbcompiler.configs.quantization_config.CalibrationConfig.hist_num_bins = hist_num_bins
```

Definition at line 182 of file [quantization_config.py](#).

15.3.3.19 hist_num_samples

```
qbcompiler.configs.quantization_config.CalibrationConfig.hist_num_samples = hist_num_samples
```

Definition at line 183 of file [quantization_config.py](#).

15.3.3.20 hist_buffer_size

```
qbcompiler.configs.quantization_config.CalibrationConfig.hist_buffer_size = hist_buffer_size
```

Definition at line 184 of file [quantization_config.py](#).

15.3.3.21 hist_min_bin_width

```
qbcompiler.configs.quantization_config.CalibrationConfig.hist_min_bin_width = hist_min_bin_width
```

Definition at line 185 of file [quantization_config.py](#).

15.3.3.22 hist_search_percentile_min

```
qbcompiler.configs.quantization_config.CalibrationConfig.hist_search_percentile_min = hist_search_↵  
percentile_min
```

Definition at line 186 of file [quantization_config.py](#).

15.3.3.23 hist_search_percentile_max

```
qbcompiler.configs.quantization_config.CalibrationConfig.hist_search_percentile_max = hist_search_↵  
percentile_max
```

Definition at line 187 of file [quantization_config.py](#).

15.3.3.24 hist_num_search

```
qbcompiler.configs.quantization_config.CalibrationConfig.hist_num_search = hist_num_search
```

Definition at line 188 of file [quantization_config.py](#).

15.3.3.25 hist_search_type

```
qbcompiler.configs.quantization_config.CalibrationConfig.hist_search_type = hist_search_type
```

Definition at line 189 of file [quantization_config.py](#).

15.3.3.26 act_scale_min

```
qbcompiler.configs.quantization_config.CalibrationConfig.act_scale_min = act_scale_min
```

Definition at line 191 of file [quantization_config.py](#).

15.3.3.27 weight_scale_min

```
qbcompiler.configs.quantization_config.CalibrationConfig.weight_scale_min = weight_scale_min
```

Definition at line 192 of file [quantization_config.py](#).

15.3.3.28 min_clip_ratio

```
qbcompiler.configs.quantization_config.CalibrationConfig.min_clip_ratio = min_clip_ratio
```

Definition at line 193 of file [quantization_config.py](#).

15.3.3.29 layer_overrides

```
qbcompiler.configs.quantization_config.CalibrationConfig.layer_overrides = layer_overrides
```

Definition at line 194 of file [quantization_config.py](#).

15.3.3.30 optional [2/3]

```
bool qbcompiler.configs.quantization_config.BitConfig.optional = False [static]
```

Definition at line 322 of file [quantization_config.py](#).

15.3.3.31 DEFAULTS [2/2]

qbcompiler.configs.quantization_config.BitConfig.DEFAULTS [static]

Initial value:

```
= {
    "query_act_bits": 8,
    "key_act_bits": 8,
    "value_act_bits": 8,
    "output_act_bits": 16,
    "ffn_act_bits": 16,
    "head_act_bits": 8,
    "query_weight_bits": 8,
    "key_weight_bits": 8,
    "value_weight_bits": 8,
    "output_weight_bits": 8,
    "ffn_weight_bits": 8,
    "head_weight_bits": 8,
    "mixed_precision_apply": False,
    "save_path": "",
    "load_path": "",
    "activation_16bits": [],
    "weight_16bits": [],
}
```

Definition at line 324 of file [quantization_config.py](#).**15.3.3.32 py_to_cpp_name [2/3]**

dict qbcompiler.configs.quantization_config.BitConfig.py_to_cpp_name [static]

Initial value:

```
= {
    "query_act_bits": "queryActBits",
    "key_act_bits": "keyActBits",
    "value_act_bits": "valueActBits",
    "output_act_bits": "outputActBits",
    "ffn_act_bits": "ffnActBits",
    "head_act_bits": "headActBits",
    "query_weight_bits": "queryWeightBits",
    "key_weight_bits": "keyWeightBits",
    "value_weight_bits": "valueWeightBits",
    "output_weight_bits": "outputWeightBits",
    "ffn_weight_bits": "ffnWeightBits",
    "head_weight_bits": "headWeightBits",
    "mixed_precision_apply": "mixedPrecisionApply",
    "save_path": "savePath",
    "load_path": "loadPath",
}
```

Definition at line 344 of file [quantization_config.py](#).

15.3.3.33 min_max_check [2/3]

`dict qbcompiler.configs.quantization_config.BitConfig.min_max_check [static]`

Initial value:

```
= {
    "query_act_bits": [1, None],
    "key_act_bits": [1, None],
    "value_act_bits": [1, None],
    "output_act_bits": [1, None],
    "ffn_act_bits": [1, None],
    "head_act_bits": [1, None],
    "query_weight_bits": [1, None],
    "key_weight_bits": [1, None],
    "value_weight_bits": [1, None],
    "output_weight_bits": [1, None],
    "ffn_weight_bits": [1, None],
    "head_weight_bits": [1, None],
}
```

Definition at line [363](#) of file [quantization_config.py](#).

15.3.3.34 query_act_bits

`qbcompiler.configs.quantization_config.BitConfig.query_act_bits = query_act_bits`

Definition at line [421](#) of file [quantization_config.py](#).

15.3.3.35 key_act_bits

`qbcompiler.configs.quantization_config.BitConfig.key_act_bits = key_act_bits`

Definition at line [422](#) of file [quantization_config.py](#).

15.3.3.36 value_act_bits

`qbcompiler.configs.quantization_config.BitConfig.value_act_bits = value_act_bits`

Definition at line [423](#) of file [quantization_config.py](#).

15.3.3.37 output_act_bits

`qbcompiler.configs.quantization_config.BitConfig.output_act_bits = output_act_bits`

Definition at line [424](#) of file [quantization_config.py](#).

15.3.3.38 ffn_act_bits

```
qbcompiler.configs.quantization_config.BitConfig.ffn_act_bits = ffn_act_bits
```

Definition at line [425](#) of file [quantization_config.py](#).

15.3.3.39 head_act_bits

```
qbcompiler.configs.quantization_config.BitConfig.head_act_bits = head_act_bits
```

Definition at line [426](#) of file [quantization_config.py](#).

15.3.3.40 query_weight_bits

```
qbcompiler.configs.quantization_config.BitConfig.query_weight_bits = query_weight_bits
```

Definition at line [427](#) of file [quantization_config.py](#).

15.3.3.41 key_weight_bits

```
qbcompiler.configs.quantization_config.BitConfig.key_weight_bits = key_weight_bits
```

Definition at line [428](#) of file [quantization_config.py](#).

15.3.3.42 value_weight_bits

```
qbcompiler.configs.quantization_config.BitConfig.value_weight_bits = value_weight_bits
```

Definition at line [429](#) of file [quantization_config.py](#).

15.3.3.43 output_weight_bits

```
qbcompiler.configs.quantization_config.BitConfig.output_weight_bits = output_weight_bits
```

Definition at line [430](#) of file [quantization_config.py](#).

15.3.3.44 ffn_weight_bits

```
qbcompiler.configs.quantization_config.BitConfig.ffn_weight_bits = ffn_weight_bits
```

Definition at line [431](#) of file [quantization_config.py](#).

15.3.3.45 head_weight_bits

```
qbcompiler.configs.quantization_config.BitConfig.head_weight_bits = head_weight_bits
```

Definition at line [432](#) of file [quantization_config.py](#).

15.3.3.46 mixed_precision_apply

```
qbcompiler.configs.quantization_config.BitConfig.mixed_precision_apply = mixed_precision_apply
```

Definition at line [433](#) of file [quantization_config.py](#).

15.3.3.47 save_path

```
qbcompiler.configs.quantization_config.BitConfig.save_path = save_path
```

Definition at line [434](#) of file [quantization_config.py](#).

15.3.3.48 load_path

```
qbcompiler.configs.quantization_config.BitConfig.load_path = load_path
```

Definition at line [435](#) of file [quantization_config.py](#).

15.3.3.49 activation_16bits

```
qbcompiler.configs.quantization_config.BitConfig.activation_16bits = activation_16bits
```

Definition at line [436](#) of file [quantization_config.py](#).

15.3.3.50 weight_16bits

```
qbcompiler.configs.quantization_config.BitConfig.weight_16bits = weight_16bits
```

Definition at line [437](#) of file [quantization_config.py](#).

15.3.3.51 optional [3/3]

```
bool qbcompiler.configs.quantization_config.QuantizationConfig.optional = False [static]
```

Definition at line [573](#) of file [quantization_config.py](#).

15.3.3.52 py_to_cpp_name [3/3]

dict qbcompiler.configs.quantization_config.QuantizationConfig.py_to_cpp_name [static]

Initial value:

```
= {
    "calibration": "calibration",
    "bit": "bit",
}
```

Definition at line 574 of file [quantization_config.py](#).

15.3.3.53 min_max_check [3/3]

dict qbcompiler.configs.quantization_config.QuantizationConfig.min_max_check = {} [static]

Definition at line 579 of file [quantization_config.py](#).

15.3.3.54 calibration

qbcompiler.configs.quantization_config.QuantizationConfig.calibration = calibration or Calibration↔
Config.default_config()

Definition at line 594 of file [quantization_config.py](#).

15.3.3.55 bit

qbcompiler.configs.quantization_config.QuantizationConfig.bit = bit or BitConfig.default_config()

Definition at line 595 of file [quantization_config.py](#).

15.4 Resource Management Configuration

Resource Management configuration definitions used by Mobilint compiler for model quantization.

Classes

- class [qbcompiler.configs.resource_management_config.ResourceManagementConfig](#)
Configuration for resource management during model compilation.

Functions

- `qbcompiler.configs.resource_management_config.ResourceManagementConfig.__init__` (self, str weight_dtype, bool use_gpu_only_for_calibration, int weight_memory_method, bool optional=optional)
Initialize the **ResourceManagementConfig**.
- `qbcompiler.configs.resource_management_config.ResourceManagementConfig.default_config` (cls)
- `qbcompiler.configs.resource_management_config.ResourceManagementConfig.from_kwargs` (cls, **kwargs)
- `qbcompiler.configs.resource_management_config.ResourceManagementConfig.from_dict` (cls, dict config_dict)
Create **ResourceManagementConfig** from dictionary (JSON structure)
- `qbcompiler.configs.resource_management_config.ResourceManagementConfig.to_dict` (self)
- `ResourceManagementConfig qbcompiler.configs.resource_management_config.get_resource_management_config` (**kwargs)
Create **ResourceManagementConfig** with partial parameters merged with defaults.

Variables

- bool `qbcompiler.configs.resource_management_config.ResourceManagementConfig.optional` = False
- dict `qbcompiler.configs.resource_management_config.ResourceManagementConfig.DEFAULTS`
- dict `qbcompiler.configs.resource_management_config.ResourceManagementConfig.py_to_cpp_name`
- dict `qbcompiler.configs.resource_management_config.ResourceManagementConfig.min_max_check`
- list `qbcompiler.configs.resource_management_config.ResourceManagementConfig.weight_memory_methods`
- `qbcompiler.configs.resource_management_config.ResourceManagementConfig.weight_dtype` = weight_dtype
- `qbcompiler.configs.resource_management_config.ResourceManagementConfig.use_gpu_only_for_calibration` = use_gpu_only_for_calibration
- `qbcompiler.configs.resource_management_config.ResourceManagementConfig.weight_memory_method` = weight_memory_method

15.4.1 Detailed Description

Resource Management configuration definitions used by Mobilint compiler for model quantization.

15.4.2 Function Documentation

15.4.2.1 __init__()

```
qbcompiler.configs.resource_management_config.ResourceManagementConfig.__init__ (
    self,
    str weight_dtype,
    bool use_gpu_only_for_calibration,
    int weight_memory_method,
    bool optional = optional)
```

Initialize the **ResourceManagementConfig**.

Parameters

weight_dtype	str. Weight data type for calibration (e.g., "float32", "float16").
use_gpu_only_for_calibration	bool. If True, use GPU only during the calibration phase.
weight_memory_method	int. Weight memory management method index. 0: DeleteFloat - Delete float weights after quantization. 1: SaveFloat - Save float weights to disk. 2: MoveFloat - Move float weights to CPU. 3: KeepFloat - Keep float weights in memory. 4: KeepAll - Keep all weights in memory.
optional	bool. Indicates whether this config is optional. Use the default value defined as a class variable.

Definition at line [41](#) of file [resource_management_config.py](#).

15.4.2.2 default_config()

```
qbcompiler.configs.resource_management_config.ResourceManagementConfig.default_config (
    cls)
```

Definition at line [67](#) of file [resource_management_config.py](#).

15.4.2.3 from_kwargs()

```
qbcompiler.configs.resource_management_config.ResourceManagementConfig.from_kwargs (
    cls,
    ** kwargs)
```

Definition at line [71](#) of file [resource_management_config.py](#).

15.4.2.4 from_dict()

```
qbcompiler.configs.resource_management_config.ResourceManagementConfig.from_dict (
    cls,
    dict config_dict)
```

Create [ResourceManagementConfig](#) from dictionary (JSON structure)

Definition at line [87](#) of file [resource_management_config.py](#).

15.4.2.5 to_dict()

```
qbcompiler.configs.resource_management_config.ResourceManagementConfig.to_dict (
    self)
```

Definition at line [105](#) of file [resource_management_config.py](#).

15.4.2.6 get_resource_management_config()

```
ResourceManagementConfig qbcompiler.configs.resource_management_config.get_resource_management_config
(
    ** kwargs)
```

Create [ResourceManagementConfig](#) with partial parameters merged with defaults.

Definition at line 118 of file [resource_management_config.py](#).

15.4.3 Variable Documentation

15.4.3.1 optional

```
bool qbcompiler.configs.resource_management_config.ResourceManagementConfig.optional = False [static]
```

Definition at line 16 of file [resource_management_config.py](#).

15.4.3.2 DEFAULTS

```
qbcompiler.configs.resource_management_config.ResourceManagementConfig.DEFAULTS [static]
```

Initial value:

```
= {
    "weight_dtype": "float32",
    "use_gpu_only_for_calibration": True,
    "weight_memory_method": 0,
}
```

Definition at line 17 of file [resource_management_config.py](#).

15.4.3.3 py_to_cpp_name

```
dict qbcompiler.configs.resource_management_config.ResourceManagementConfig.py_to_cpp_name [static]
```

Initial value:

```
= {
    "weight_dtype": "weightDtype",
    "use_gpu_only_for_calibration": "useGPUOnlyForCalibration",
    "weight_memory_method": "weightMemoryMethod",
}
```

Definition at line 23 of file [resource_management_config.py](#).

15.4.3.4 min_max_check

`dict qbcompiler.configs.resource_management_config.ResourceManagementConfig.min_max_check [static]`

Initial value:

```
= {
    "weight_memory_method": [0, 4],
}
```

Definition at line 29 of file [resource_management_config.py](#).

15.4.3.5 weight_memory_methods

`qbcompiler.configs.resource_management_config.ResourceManagementConfig.weight_memory_methods [static]`

Initial value:

```
= [
    "DeleteFloat",
    "SaveFloat",
    "MoveFloat",
    "KeepFloat",
    "KeepAll",
]
```

Definition at line 33 of file [resource_management_config.py](#).

15.4.3.6 weight_dtype

`qbcompiler.configs.resource_management_config.ResourceManagementConfig.weight_dtype = weight_dtype`

Definition at line 61 of file [resource_management_config.py](#).

15.4.3.7 use_gpu_only_for_calibration

`qbcompiler.configs.resource_management_config.ResourceManagementConfig.use_gpu_only_for_calibration = use_gpu_only_for_calibration`

Definition at line 62 of file [resource_management_config.py](#).

15.4.3.8 weight_memory_method

`qbcompiler.configs.resource_management_config.ResourceManagementConfig.weight_memory_method = weight↵_memory_method`

Definition at line 63 of file [resource_management_config.py](#).

15.5 LLM Configuration

LLM configuration definitions used by Mobilint compiler for LLM model quantization.

Classes

- class `qbcompiler.configs.llm_config.LlmConfig`
Configuration for Large Language Model (LLM) compilation.

Functions

- `qbcompiler.configs.llm_config.LlmConfig.__init__` (self, bool llm_config_apply, int max_data_length, int max_sequence_length, int max_cache_length, int max_core_data_length, int random_seq_length, bool use_full_seq_length, bool use_global_core, int batch_size, List[int] npu_core_ids, bool dynamic_rope, bool optional=optional)
Initialize the `LlmConfig`:
- `qbcompiler.configs.llm_config.LlmConfig.default_config` (cls)
- `qbcompiler.configs.llm_config.LlmConfig.from_kwargs` (cls, **kwargs)
- `qbcompiler.configs.llm_config.LlmConfig.from_dict` (cls, dict config_dict)
Create `LlmConfig` from dictionary (JSON structure)
- `qbcompiler.configs.llm_config.LlmConfig.to_dict` (self)
- `LlmConfig qbcompiler.configs.llm_config.get_llm_config` (**kwargs)
Create `LlmConfig` with partial parameters merged with defaults.

Variables

- bool `qbcompiler.configs.llm_config.LlmConfig.optional` = True
- dict `qbcompiler.configs.llm_config.LlmConfig.DEFAULTS`
- dict `qbcompiler.configs.llm_config.LlmConfig.py_to_cpp_name`
- dict `qbcompiler.configs.llm_config.LlmConfig.min_max_check`
- `qbcompiler.configs.llm_config.LlmConfig.llm_config_apply` = llm_config_apply
- `qbcompiler.configs.llm_config.LlmConfig.max_data_length` = max_data_length
- `qbcompiler.configs.llm_config.LlmConfig.max_sequence_length` = max_sequence_length
- `qbcompiler.configs.llm_config.LlmConfig.max_cache_length` = max_cache_length
- `qbcompiler.configs.llm_config.LlmConfig.max_core_data_length` = max_core_data_length
- `qbcompiler.configs.llm_config.LlmConfig.random_seq_length` = random_seq_length
- `qbcompiler.configs.llm_config.LlmConfig.use_full_seq_length` = use_full_seq_length
- `qbcompiler.configs.llm_config.LlmConfig.use_global_core` = use_global_core
- `qbcompiler.configs.llm_config.LlmConfig.batch_size` = batch_size
- `qbcompiler.configs.llm_config.LlmConfig.npu_core_ids` = npu_core_ids
- `qbcompiler.configs.llm_config.LlmConfig.dynamic_rope` = dynamic_rope

15.5.1 Detailed Description

LLM configuration definitions used by Mobilint compiler for LLM model quantization.

15.5.2 Function Documentation

15.5.2.1 `__init__()`

```
qbcompiler.configs.llm_config.LlmConfig.__init__ (
    self,
    bool llm_config_apply,
    int max_data_length,
    int max_sequence_length,
    int max_cache_length,
    int max_core_data_length,
    int random_seq_length,
    bool use_full_seq_length,
    bool use_global_core,
    int batch_size,
    List[int] npu_core_ids,
    bool dynamic_rope,
    bool optional = optional)
```

Initialize the [LlmConfig](#):

Parameters

llm_config_apply	bool. If True, apply LLM-specific configurations.
max_data_length	int. Maximum data length.
max_sequence_length	int. Maximum sequence length.
max_cache_length	int. Maximum cache length.
max_core_data_length	int. Maximum core data length.
random_seq_length	int. Random sequence length used for calibration.
use_full_seq_length	bool. If True, use the full sequence length for calibration.
use_global_core	bool. If True, use a global core.
batch_size	int. Batch size.
npu_core_ids	List[int]. List of NPU core IDs.
dynamic_rope	bool. If True, enable dynamic RoPE.
optional	bool. Indicates whether this config is optional. Use the default value defined as a class variable.

Definition at line [54](#) of file [llm_config.py](#).

15.5.2.2 `default_config()`

```
qbcompiler.configs.llm_config.LlmConfig.default_config (
    cls)
```

Definition at line [99](#) of file [llm_config.py](#).

15.5.2.3 from_kwargs()

```
qbcompiler.configs.llm_config.LlmConfig.from_kwargs (
    cls,
    ** kwargs)
```

Definition at line 103 of file [llm_config.py](#).

15.5.2.4 from_dict()

```
qbcompiler.configs.llm_config.LlmConfig.from_dict (
    cls,
    dict config_dict)
```

Create [LlmConfig](#) from dictionary (JSON structure)

Definition at line 119 of file [llm_config.py](#).

15.5.2.5 to_dict()

```
qbcompiler.configs.llm_config.LlmConfig.to_dict (
    self)
```

Definition at line 158 of file [llm_config.py](#).

15.5.2.6 get_llm_config()

```
LlmConfig qbcompiler.configs.llm_config.get_llm_config (
    ** kwargs)
```

Create [LlmConfig](#) with partial parameters merged with defaults.

Definition at line 186 of file [llm_config.py](#).

15.5.3 Variable Documentation

15.5.3.1 optional

```
bool qbcompiler.configs.llm_config.LlmConfig.optional = True [static]
```

Definition at line 17 of file [llm_config.py](#).

15.5.3.2 DEFAULTS

qbcompiler.configs.llm_config.LlmConfig.DEFAULTS [static]

Initial value:

```
= {
    "llm_config_apply": False,
    "max_data_length": 4096,
    "max_sequence_length": 4096,
    "max_cache_length": 4096,
    "max_core_data_length": 128,
    "random_seq_length": 80,
    "use_full_seq_length": False,
    "use_global_core": False,
    "batch_size": 1,
    "npu_core_ids": [0],
    "dynamic_rope": False,
}
```

Definition at line 18 of file [llm_config.py](#).

15.5.3.3 py_to_cpp_name

dict qbcompiler.configs.llm_config.LlmConfig.py_to_cpp_name [static]

Initial value:

```
= {
    "llm_config_apply": "apply",
    "max_data_length": "maxDataLength",
    "max_sequence_length": "maxSequenceLength",
    "max_cache_length": "maxCacheLength",
    "max_core_data_length": "maxCoreDataLength",
    "random_seq_length": "randomSeqLength",
    "use_full_seq_length": "useFullSeqLength",
    "use_global_core": "useGlobalCore",
    "batch_size": "batchSize",
    "dynamic_rope": "dynamicRope",
}
```

Definition at line 32 of file [llm_config.py](#).

15.5.3.4 min_max_check

dict qbcompiler.configs.llm_config.LlmConfig.min_max_check [static]

Initial value:

```
= {
    "max_data_length": [0, None],
    "max_sequence_length": [0, None],
    "max_cache_length": [0, None],
    "max_core_data_length": [0, None],
    "random_seq_length": [0, None],
    "batch_size": [0, None],
}
```

Definition at line 45 of file [llm_config.py](#).

15.5.3.5 llm_config_apply

```
qbcompiler.configs.llm_config.LlmConfig.llm_config_apply = llm_config_apply
```

Definition at line [85](#) of file [llm_config.py](#).

15.5.3.6 max_data_length

```
qbcompiler.configs.llm_config.LlmConfig.max_data_length = max_data_length
```

Definition at line [86](#) of file [llm_config.py](#).

15.5.3.7 max_sequence_length

```
qbcompiler.configs.llm_config.LlmConfig.max_sequence_length = max_sequence_length
```

Definition at line [87](#) of file [llm_config.py](#).

15.5.3.8 max_cache_length

```
qbcompiler.configs.llm_config.LlmConfig.max_cache_length = max_cache_length
```

Definition at line [88](#) of file [llm_config.py](#).

15.5.3.9 max_core_data_length

```
qbcompiler.configs.llm_config.LlmConfig.max_core_data_length = max_core_data_length
```

Definition at line [89](#) of file [llm_config.py](#).

15.5.3.10 random_seq_length

```
qbcompiler.configs.llm_config.LlmConfig.random_seq_length = random_seq_length
```

Definition at line [90](#) of file [llm_config.py](#).

15.5.3.11 use_full_seq_length

```
qbcompiler.configs.llm_config.LlmConfig.use_full_seq_length = use_full_seq_length
```

Definition at line [91](#) of file [llm_config.py](#).

15.5.3.12 use_global_core

```
qbcompiler.configs.llm_config.LlmConfig.use_global_core = use_global_core
```

Definition at line 92 of file [llm_config.py](#).

15.5.3.13 batch_size

```
qbcompiler.configs.llm_config.LlmConfig.batch_size = batch_size
```

Definition at line 93 of file [llm_config.py](#).

15.5.3.14 npu_core_ids

```
qbcompiler.configs.llm_config.LlmConfig.npu_core_ids = npu_core_ids
```

Definition at line 94 of file [llm_config.py](#).

15.5.3.15 dynamic_rope

```
qbcompiler.configs.llm_config.LlmConfig.dynamic_rope = dynamic_rope
```

Definition at line 95 of file [llm_config.py](#).

15.6 AdvancedQuantization Configuration

AdvancedQuantization configuration definitions used by Mobilint compiler for model quantization.

Classes

- class [qbcompiler.configs.advanced_quantization_config.AdvancedQuantizationConfig](#)
Configuration for advanced quantization options used during quantization.

Functions

- [qbcompiler.configs.advanced_quantization_config.AdvancedQuantizationConfig.__init__](#) (self, Optional[[OptqConfig](#)] optq=None, Optional[[ModConfig](#)] mod=None, Optional[[EquivalentTransformationConfig](#)] equivalent_transformation=None, Optional[[SearchWeightScaleConfig](#)] search_weight_scale=None, bool optional=optional)
- [qbcompiler.configs.advanced_quantization_config.AdvancedQuantizationConfig.default_config](#) (cls)
- [qbcompiler.configs.advanced_quantization_config.AdvancedQuantizationConfig.from_dict](#) (cls, dict config_dict)
Create [AdvancedQuantizationConfig](#) from dictionary (JSON structure)
- [qbcompiler.configs.advanced_quantization_config.AdvancedQuantizationConfig.to_dict](#) (self)
- [AdvancedQuantizationConfig qbcompiler.configs.advanced_quantization_config.get_advanced_quantization_config](#) (Optional[[OptqConfig](#)] optq=None, Optional[[ModConfig](#)] mod=None, Optional[[EquivalentTransformationConfig](#)] equivalent_transformation=None, Optional[[SearchWeightScaleConfig](#)] search_weight_scale=None)
Create [AdvancedQuantizationConfig](#) with [EquivalentTransformationConfig](#) and [SearchWeightScaleConfig](#).

Variables

- `bool qbcompiler.configs.advanced_quantization_config.AdvancedQuantizationConfig.optional = False`
- `qbcompiler.configs.advanced_quantization_config.AdvancedQuantizationConfig.optq = optq or OptqConfig.default_config()`
- `qbcompiler.configs.advanced_quantization_config.AdvancedQuantizationConfig.mod = mod or ModConfig.default_config()`
- `tuple qbcompiler.configs.advanced_quantization_config.AdvancedQuantizationConfig.equivalent_transformation`
- `tuple qbcompiler.configs.advanced_quantization_config.AdvancedQuantizationConfig.search_weight_scale`

15.6.1 Detailed Description

AdvancedQuantization configuration definitions used by Mobilint compiler for model quantization.

15.6.2 Function Documentation

15.6.2.1 `__init__()`

```
qbcompiler.configs.advanced_quantization_config.AdvancedQuantizationConfig.__init__ (
    self,
    Optional[OptqConfig] optq = None,
    Optional[ModConfig] mod = None,
    Optional[EquivalentTransformationConfig] equivalent_transformation = None,
    Optional[SearchWeightScaleConfig] search_weight_scale = None,
    bool optional = optional)
```

Parameters

optq	OptqConfig . OPTQ configuration.
mod	ModConfig . MOD configuration.
equivalent_transformation	EquivalentTransformationConfig . Equivalent transformation configuration.
search_weight_scale	SearchWeightScaleConfig . Weight scale search configuration.
optional	bool. Indicates whether this config is optional. Use the default value defined as a class variable.

Definition at line [27](#) of file [advanced_quantization_config.py](#).

15.6.2.2 `default_config()`

```
qbcompiler.configs.advanced_quantization_config.AdvancedQuantizationConfig.default_config (
    cls)
```

Definition at line [53](#) of file [advanced_quantization_config.py](#).

15.6.2.3 from_dict()

```
qbcompiler.configs.advanced_quantization_config.AdvancedQuantizationConfig.from_dict (
    cls,
    dict config_dict)
```

Create [AdvancedQuantizationConfig](#) from dictionary (JSON structure)

Definition at line 63 of file [advanced_quantization_config.py](#).

15.6.2.4 to_dict()

```
qbcompiler.configs.advanced_quantization_config.AdvancedQuantizationConfig.to_dict (
    self)
```

Definition at line 77 of file [advanced_quantization_config.py](#).

15.6.2.5 get_advanced_quantization_config()

```
AdvancedQuantizationConfig qbcompiler.configs.advanced_quantization_config.get_advanced_quantization↔
_config (
    Optional[OptqConfig] optq = None,
    Optional[ModConfig] mod = None,
    Optional[EquivalentTransformationConfig] equivalent_transformation = None,
    Optional[SearchWeightScaleConfig] search_weight_scale = None)
```

Create [AdvancedQuantizationConfig](#) with [EquivalentTransformationConfig](#) and [SearchWeightScaleConfig](#).

Definition at line 88 of file [advanced_quantization_config.py](#).

15.6.3 Variable Documentation

15.6.3.1 optional

```
bool qbcompiler.configs.advanced_quantization_config.AdvancedQuantizationConfig.optional = False
[static]
```

Definition at line 24 of file [advanced_quantization_config.py](#).

15.6.3.2 optq

```
qbcompiler.configs.advanced_quantization_config.AdvancedQuantizationConfig.optq = optq or Optq↔
Config.default_config()
```

Definition at line 43 of file [advanced_quantization_config.py](#).

15.6.3.3 mod

```
qbcompiler.configs.advanced_quantization_config.AdvancedQuantizationConfig.mod = mod or ModConfig.default_config()
```

Definition at line 44 of file [advanced_quantization_config.py](#).

15.6.3.4 equivalent_transformation

```
tuple qbcompiler.configs.advanced_quantization_config.AdvancedQuantizationConfig.equivalent_transformation
```

Initial value:

```
= (
    equivalent_transformation or
    EquivalentTransformationConfig.default_config()
)
```

Definition at line 45 of file [advanced_quantization_config.py](#).

15.6.3.5 search_weight_scale

```
tuple qbcompiler.configs.advanced_quantization_config.AdvancedQuantizationConfig.search_weight_scale
```

Initial value:

```
= (
    search_weight_scale or SearchWeightScaleConfig.default_config()
)
```

Definition at line 48 of file [advanced_quantization_config.py](#).

15.7 EquivalentTransformation Configuration

EquivalentTransformation configuration definitions used by Mobilint compiler for model quantization.

Classes

- class [qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig](#)
Configuration for equivalent transformation techniques.

Functions

- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.__init__`
(self, int seed, bool apply_hadamard_rotation_matrix, bool norm_conv_apply, bool norm_conv_learn, float norm_conv_smoothing_factor, float norm_conv_min_gamma, float norm_conv_max_gamma, bool qk_apply, float qk_smoothing_factor, float qk_min_gamma, float qk_max_gamma, bool ud_apply, bool ud_learn, float ud_smoothing_factor, float ud_min_gamma, float ud_max_gamma, bool vo_apply, float vo_smoothing_factor, float vo_min_gamma, float vo_max_gamma, bool ff_multi_lut_apply, List[float] ff_multi_lut_breakpoints, bool spin_r1_apply, str spin_r1_matrix_path, bool head_out_ch_rotation_apply, str head_out_ch_rotation_matrix_path, bool spin_r2_apply, bool spin_r2_learn, str spin_r2_matrix_path, bool qk_rotation_apply, str qk_rotation_matrix_path, bool flatten_quant_apply, bool flatten_quant_learn, float flatten_quant_apply_threshold, float flatten_quant_max_overhead, bool optimize_ffn_apply, int optimize_ffn_ch_per_ffn, bool optional=optional)
Initialize the `EquivalentTransformationConfig`.
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.check_valid`
(self, except_list=["optional"])
Override to skip `py_to_cpp_name` check (uses nested structure in `to_dict`)
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.default_config`
(cls)
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.from_dict`
(cls, dict config_dict)
Create `EquivalentTransformationConfig` from dictionary (JSON structure)
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.to_dict`
(self)
- `EquivalentTransformationConfig qbcompiler.configs.equivalent_transformation_config.get_equivalent_transformation_config(**kwargs)`
Create `EquivalentTransformationConfig` with partial parameters merged with defaults.

Variables

- `bool qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.optional`
= False
- `dict qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.py_to_cpp_name`
= {}
- `dict qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.DEFAULTS`
- `dict qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.min_max_check`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.seed` = seed
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.apply_hadamard_rotation_matrix`
= apply_hadamard_rotation_matrix
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.norm_conv_apply`
= norm_conv_apply
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.norm_conv_learn`
= norm_conv_learn
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.norm_conv_smoothing_factor`
= norm_conv_smoothing_factor
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.norm_conv_min_gamma`
= norm_conv_min_gamma
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.norm_conv_max_gamma`
= norm_conv_max_gamma
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.qk_apply` = qk_apply

- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.qk_smoothing_factor`
= `qk_smoothing_factor`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.qk_min_gamma`
= `qk_min_gamma`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.qk_max_gamma`
= `qk_max_gamma`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.ud_apply` =
`ud_apply`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.ud_learn` =
`ud_learn`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.ud_smoothing_factor`
= `ud_smoothing_factor`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.ud_min_gamma`
= `ud_min_gamma`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.ud_max_gamma`
= `ud_max_gamma`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.vo_apply` =
`vo_apply`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.vo_smoothing_factor`
= `vo_smoothing_factor`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.vo_min_gamma`
= `vo_min_gamma`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.vo_max_gamma`
= `vo_max_gamma`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.ff_multi_lut_apply`
= `ff_multi_lut_apply`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.ff_multi_lut_breakpoints`
= `ff_multi_lut_breakpoints`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.spin_r1_apply`
= `spin_r1_apply`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.spin_r1_matrix_path`
= `spin_r1_matrix_path`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.head_out_ch_rotation_apply`
= `head_out_ch_rotation_apply`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.head_out_ch_rotation_matrix_path`
= `head_out_ch_rotation_matrix_path`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.spin_r2_apply`
= `spin_r2_apply`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.spin_r2_learn`
= `spin_r2_learn`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.spin_r2_matrix_path`
= `spin_r2_matrix_path`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.qk_rotation_apply`
= `qk_rotation_apply`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.qk_rotation_matrix_path`
= `qk_rotation_matrix_path`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.flatten_quant_apply`
= `flatten_quant_apply`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.flatten_quant_learn`
= `flatten_quant_learn`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.flatten_quant_apply_threshold`
= `flatten_quant_apply_threshold`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.flatten_quant_max_overhead`
= `flatten_quant_max_overhead`

- `flatten_quant_max_overhead`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.optimize_ffn_apply`
= `optimize_ffn_apply`
- `qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.optimize_ffn_ch_per_ffn`
= `optimize_ffn_ch_per_ffn`

15.7.1 Detailed Description

EquivalentTransformation configuration definitions used by Mobilint compiler for model quantization.

15.7.2 Function Documentation

15.7.2.1 `__init__()`

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.__init__ (
    self,
    int seed,
    bool apply_hadamard_rotation_matrix,
    bool norm_conv_apply,
    bool norm_conv_learn,
    float norm_conv_smoothing_factor,
    float norm_conv_min_gamma,
    float norm_conv_max_gamma,
    bool qk_apply,
    float qk_smoothing_factor,
    float qk_min_gamma,
    float qk_max_gamma,
    bool ud_apply,
    bool ud_learn,
    float ud_smoothing_factor,
    float ud_min_gamma,
    float ud_max_gamma,
    bool vo_apply,
    float vo_smoothing_factor,
    float vo_min_gamma,
    float vo_max_gamma,
    bool ff_multi_lut_apply,
    List[float] ff_multi_lut_breakpoints,
    bool spin_r1_apply,
    str spin_r1_matrix_path,
    bool head_out_ch_rotation_apply,
    str head_out_ch_rotation_matrix_path,
    bool spin_r2_apply,
    bool spin_r2_learn,
    str spin_r2_matrix_path,
    bool qk_rotation_apply,
    str qk_rotation_matrix_path,
    bool flatten_quant_apply,
    bool flatten_quant_learn,
    float flatten_quant_apply_threshold,
    float flatten_quant_max_overhead,
    bool optimize_ffn_apply,
```

```
int optimize_ffn_ch_per_ffn,
bool optional = optional)
```

Initialize the [EquivalentTransformationConfig](#).

Parameters

seed	int. Random seed used for equivalent transformation operations. Default is 0.
apply_hadamard_rotation_matrix	bool. If True, apply the Hadamard rotation matrix where applicable. Default is True.

15.7.3 NormConv

Parameters

norm_conv_apply	bool. If True, enable NormConv transformation. Default is False.
norm_conv_learn	bool. If True, learn NormConv parameters during calibration/optimization. Default is False.
norm_conv_smoothing_factor	float. Smoothing factor used in NormConv. Default is 0.5.
norm_conv_min_gamma	float. Minimum gamma value (lower bound clamp) for NormConv. Default is 0.0001.
norm_conv_max_gamma	float. Maximum gamma value (upper bound clamp) for NormConv. Default is 10000.0.

15.7.4 QK

Parameters

qk_apply	bool. If True, enable QK transformation. Default is False.
qk_smoothing_factor	float. Smoothing factor used in QK. Default is 0.5.
qk_min_gamma	float. Minimum gamma value (lower bound clamp) for QK. Default is 0.0001.
qk_max_gamma	float. Maximum gamma value (upper bound clamp) for QK. Default is 10000.0.

15.7.5 UD

Parameters

ud_apply	bool. If True, enable UD transformation. Default is False.
ud_learn	bool. If True, learn UD parameters during calibration/optimization. Default is False.
ud_smoothing_factor	float. Smoothing factor used in UD. Default is 0.5.
ud_min_gamma	float. Minimum gamma value (lower bound clamp) for UD. Default is 0.0001.

ud_max_gamma	float. Maximum gamma value (upper bound clamp) for UD. Default is 10000.0.
---------------------	--

15.7.6 VO

Parameters

vo_apply	bool. If True, enable VO transformation. Default is False.
vo_smoothing_factor	float. Smoothing factor used in VO. Default is 0.5.
vo_min_gamma	float. Minimum gamma value (lower bound clamp) for VO. Default is 0.0001.
vo_max_gamma	float. Maximum gamma value (upper bound clamp) for VO. Default is 10000.0.

15.7.7 FeedForwardMultiLUT

Parameters

ff_multi_lut_apply	bool. If True, enable FeedForward Multi-LUT. Default is False.
ff_multi_lut_breakpoints	List[float]. Breakpoints used for FeedForward Multi-LUT piecewise regions. Default is [-8.0, -4.0, 0].

15.7.8 SpinR1

Parameters

spin_r1_apply	bool. If True, enable SpinR1 rotation. Default is False.
spin_r1_matrix_path	str. Path to the SpinR1 rotation matrix file. Default is "".

15.7.9 HeadOutChRotation

Parameters

head_out_ch_rotation_apply	bool. If True, enable HeadOutChRotation. Default is False.
head_out_ch_rotation_matrix_path	str. Path to the HeadOutChRotation matrix file. Default is "".

15.7.10 SpinR2

Parameters

spin_r2_apply	bool. If True, enable SpinR2 rotation. Default is False.
spin_r2_learn	bool. If True, learn the SpinR2 rotation matrix/parameters during calibration/optimization. Default is False.
spin_r2_matrix_path	str. Path to the SpinR2 rotation matrix file. Default is "".

15.7.11 QKRotation

Parameters

qk_rotation_apply	bool. If True, enable QKRotation. Default is False.
qk_rotation_matrix_path	str. Path to the QKRotation matrix file. Default is "".

15.7.12 FlattenQuant

Parameters

flatten_quant_apply	bool. If True, enable FlattenQuant. Default is False.
flatten_quant_learn	bool. If True, learn FlattenQuant parameters during calibration/optimization. Default is False.
flatten_quant_apply_threshold	float. Threshold for applying FlattenQuant. Default is 0.33.
flatten_quant_max_overhead	float. Maximum allowed overhead when applying FlattenQuant. Default is 0.05.

15.7.13 OptimizeFFN

Parameters

optimize_ffn_apply	bool. If True, enable OptimizeFFN. Default is False.
optimize_ffn_ch_per_ffn	int. Number of channels per FFN used when OptimizeFFN is enabled. Default is -1.
optional	bool. Indicates whether this config is optional. Use the default value defined as a class variable.

Definition at line 96 of file [equivalent_transformation_config.py](#).

15.7.13.1 check_valid()

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.check_valid (
    self,
    except_list = ["optional"])
```

Override to skip py_to_cpp_name check (uses nested structure in to_dict)

Definition at line 277 of file [equivalent_transformation_config.py](#).

15.7.13.2 default_config()

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.default_config (
    cls)
```

Definition at line 283 of file [equivalent_transformation_config.py](#).

15.7.13.3 from_dict()

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.from_dict (
    cls,
    dict config_dict)
```

Create [EquivalentTransformationConfig](#) from dictionary (JSON structure)

Definition at line 287 of file [equivalent_transformation_config.py](#).

15.7.13.4 to_dict()

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.to_dict (
    self)
```

Definition at line 418 of file [equivalent_transformation_config.py](#).

15.7.13.5 get_equivalent_transformation_config()

```
EquivalentTransformationConfig qbcompiler.configs.equivalent_transformation_config.get_equivalent_↵
transformation_config (
    ** kwargs)
```

Create [EquivalentTransformationConfig](#) with partial parameters merged with defaults.

Definition at line 482 of file [equivalent_transformation_config.py](#).

15.7.14 Variable Documentation

15.7.14.1 optional

```
bool qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.optional =
False [static]
```

Definition at line 22 of file [equivalent_transformation_config.py](#).

15.7.14.2 py_to_cpp_name

```
dict qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.py_to_cpp_↵
name = {} [static]
```

Definition at line 23 of file [equivalent_transformation_config.py](#).

15.7.14.3 DEFAULTS

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.DEFAULTS [static]
```

Definition at line 27 of file [equivalent_transformation_config.py](#).

15.7.14.4 min_max_check

`dict qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.min_max_check`
`[static]`

Initial value:

```
= {
    "norm_conv_smoothing_factor": [0.0, 1.0],
    "norm_conv_min_gamma": [0.0, None],
    "norm_conv_max_gamma": [0.0, None],
    "qk_smoothing_factor": [0.0, 1.0],
    "qk_min_gamma": [0.0, None],
    "qk_max_gamma": [0.0, None],
    "ud_smoothing_factor": [0.0, 1.0],
    "ud_min_gamma": [0.0, None],
    "ud_max_gamma": [0.0, None],
    "vo_smoothing_factor": [0.0, 1.0],
    "vo_min_gamma": [0.0, None],
    "vo_max_gamma": [0.0, None],
    "flatten_quant_apply_threshold": [0.0, 1.0],
    "flatten_quant_max_overhead": [0.0, None],
}
```

Definition at line 79 of file [equivalent_transformation_config.py](#).

15.7.14.5 seed

`qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.seed = seed`

Definition at line 215 of file [equivalent_transformation_config.py](#).

15.7.14.6 apply_hadamard_rotation_matrix

`qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.apply_hadamard_↵`
`rotation_matrix = apply_hadamard_rotation_matrix`

Definition at line 216 of file [equivalent_transformation_config.py](#).

15.7.14.7 norm_conv_apply

`qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.norm_conv_apply =`
`norm_conv_apply`

Definition at line 219 of file [equivalent_transformation_config.py](#).

15.7.14.8 norm_conv_learn

`qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.norm_conv_learn =`
`norm_conv_learn`

Definition at line 220 of file [equivalent_transformation_config.py](#).

15.7.14.9 norm_conv_smoothing_factor

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.norm_conv_↵  
smoothing_factor = norm_conv_smoothing_factor
```

Definition at line 221 of file [equivalent_transformation_config.py](#).

15.7.14.10 norm_conv_min_gamma

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.norm_conv_min_↵  
gamma = norm_conv_min_gamma
```

Definition at line 222 of file [equivalent_transformation_config.py](#).

15.7.14.11 norm_conv_max_gamma

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.norm_conv_max_↵  
gamma = norm_conv_max_gamma
```

Definition at line 223 of file [equivalent_transformation_config.py](#).

15.7.14.12 qk_apply

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.qk_apply = qk_↵  
apply
```

Definition at line 226 of file [equivalent_transformation_config.py](#).

15.7.14.13 qk_smoothing_factor

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.qk_smoothing_↵  
factor = qk_smoothing_factor
```

Definition at line 227 of file [equivalent_transformation_config.py](#).

15.7.14.14 qk_min_gamma

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.qk_min_gamma =  
qk_min_gamma
```

Definition at line 228 of file [equivalent_transformation_config.py](#).

15.7.14.15 qk_max_gamma

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.qk_max_gamma =  
qk_max_gamma
```

Definition at line 229 of file [equivalent_transformation_config.py](#).

15.7.14.16 ud_apply

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.ud_apply = ud_↔  
apply
```

Definition at line 232 of file [equivalent_transformation_config.py](#).

15.7.14.17 ud_learn

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.ud_learn = ud_↔  
learn
```

Definition at line 233 of file [equivalent_transformation_config.py](#).

15.7.14.18 ud_smoothing_factor

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.ud_smoothing_↔  
factor = ud_smoothing_factor
```

Definition at line 234 of file [equivalent_transformation_config.py](#).

15.7.14.19 ud_min_gamma

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.ud_min_gamma =  
ud_min_gamma
```

Definition at line 235 of file [equivalent_transformation_config.py](#).

15.7.14.20 ud_max_gamma

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.ud_max_gamma =  
ud_max_gamma
```

Definition at line 236 of file [equivalent_transformation_config.py](#).

15.7.14.21 vo_apply

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.vo_apply = vo_↔  
apply
```

Definition at line 239 of file [equivalent_transformation_config.py](#).

15.7.14.22 vo_smoothing_factor

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.vo_smoothing_↔  
factor = vo_smoothing_factor
```

Definition at line 240 of file [equivalent_transformation_config.py](#).

15.7.14.23 vo_min_gamma

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.vo_min_gamma =  
vo_min_gamma
```

Definition at line 241 of file [equivalent_transformation_config.py](#).

15.7.14.24 vo_max_gamma

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.vo_max_gamma =  
vo_max_gamma
```

Definition at line 242 of file [equivalent_transformation_config.py](#).

15.7.14.25 ff_multi_lut_apply

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.ff_multi_lut_apply  
= ff_multi_lut_apply
```

Definition at line 245 of file [equivalent_transformation_config.py](#).

15.7.14.26 ff_multi_lut_breakpoints

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.ff_multi_lut_↔  
breakpoints = ff_multi_lut_breakpoints
```

Definition at line 246 of file [equivalent_transformation_config.py](#).

15.7.14.27 spin_r1_apply

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.spin_r1_apply =  
spin_r1_apply
```

Definition at line 249 of file [equivalent_transformation_config.py](#).

15.7.14.28 spin_r1_matrix_path

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.spin_r1_matrix_↵  
path = spin_r1_matrix_path
```

Definition at line 250 of file [equivalent_transformation_config.py](#).

15.7.14.29 head_out_ch_rotation_apply

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.head_out_ch_↵  
rotation_apply = head_out_ch_rotation_apply
```

Definition at line 253 of file [equivalent_transformation_config.py](#).

15.7.14.30 head_out_ch_rotation_matrix_path

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.head_out_ch_↵  
rotation_matrix_path = head_out_ch_rotation_matrix_path
```

Definition at line 254 of file [equivalent_transformation_config.py](#).

15.7.14.31 spin_r2_apply

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.spin_r2_apply =  
spin_r2_apply
```

Definition at line 257 of file [equivalent_transformation_config.py](#).

15.7.14.32 spin_r2_learn

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.spin_r2_learn =  
spin_r2_learn
```

Definition at line 258 of file [equivalent_transformation_config.py](#).

15.7.14.33 spin_r2_matrix_path

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.spin_r2_matrix_↵  
path = spin_r2_matrix_path
```

Definition at line 259 of file [equivalent_transformation_config.py](#).

15.7.14.34 qk_rotation_apply

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.qk_rotation_apply  
= qk_rotation_apply
```

Definition at line 262 of file [equivalent_transformation_config.py](#).

15.7.14.35 qk_rotation_matrix_path

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.qk_rotation_↵  
matrix_path = qk_rotation_matrix_path
```

Definition at line 263 of file [equivalent_transformation_config.py](#).

15.7.14.36 flatten_quant_apply

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.flatten_quant_↵  
apply = flatten_quant_apply
```

Definition at line 266 of file [equivalent_transformation_config.py](#).

15.7.14.37 flatten_quant_learn

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.flatten_quant_↵  
learn = flatten_quant_learn
```

Definition at line 267 of file [equivalent_transformation_config.py](#).

15.7.14.38 flatten_quant_apply_threshold

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.flatten_quant_↵  
apply_threshold = flatten_quant_apply_threshold
```

Definition at line 268 of file [equivalent_transformation_config.py](#).

15.7.14.39 flatten_quant_max_overhead

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.flatten_quant_↵
max_overhead = flatten_quant_max_overhead
```

Definition at line 269 of file [equivalent_transformation_config.py](#).

15.7.14.40 optimize_ffn_apply

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.optimize_ffn_apply
= optimize_ffn_apply
```

Definition at line 272 of file [equivalent_transformation_config.py](#).

15.7.14.41 optimize_ffn_ch_per_ffn

```
qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig.optimize_ffn_ch_↵
per_ffn = optimize_ffn_ch_per_ffn
```

Definition at line 273 of file [equivalent_transformation_config.py](#).

15.8 SearchWeightScale Configuration

SearchWeightScale configuration definitions used by Mobilint compiler for model quantization.

Classes

- class [qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig](#)
Configuration for searching weight scale.

Functions

- [qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.__init__](#) (self, bool apply, bool query, bool key, bool value, bool out, bool ffn, bool optional=optional)
Initialize the [SearchWeightScaleConfig](#).
- [qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.default_config](#) (cls)
- [qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.from_dict](#) (cls, dict config_dict)
Create [SearchWeightScaleConfig](#) from dictionary (JSON structure)
- [qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.to_dict](#) (self)
- [SearchWeightScaleConfig](#) [qbcompiler.configs.search_weight_scale_config.get_search_weight_scale_config](#) (**kwargs)
Create [SearchWeightScaleConfig](#) with partial parameters merged with defaults.

Variables

- `bool qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.optional = False`
- `dict qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.DEFAULTS`
- `qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.apply = apply`
- `qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.query = query`
- `qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.key = key`
- `qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.value = value`
- `qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.out = out`
- `qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.ffn = ffn`

15.8.1 Detailed Description

SearchWeightScale configuration definitions used by Mobilint compiler for model quantization.

15.8.2 Function Documentation

15.8.2.1 `__init__()`

```
qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.__init__ (
    self,
    bool apply,
    bool query,
    bool key,
    bool value,
    bool out,
    bool ffn,
    bool optional = optional)
```

Initialize the [SearchWeightScaleConfig](#).

Parameters

apply	bool. Whether to apply weight scale search.
query	bool. Apply to query layer.
key	bool. Apply to key layer.
value	bool. Apply to value layer.
out	bool. Apply to output layer.
ffn	bool. Apply to FFN layer.
optional	bool. Indicates whether this config is optional. Use the default value defined as a class variable.

Definition at line 26 of file [search_weight_scale_config.py](#).

15.8.2.2 default_config()

```
qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.default_config (  
    cls)
```

Definition at line 55 of file [search_weight_scale_config.py](#).

15.8.2.3 from_dict()

```
qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.from_dict (  
    cls,  
    dict config_dict)
```

Create [SearchWeightScaleConfig](#) from dictionary (JSON structure)

Definition at line 59 of file [search_weight_scale_config.py](#).

15.8.2.4 to_dict()

```
qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.to_dict (  
    self)
```

Definition at line 81 of file [search_weight_scale_config.py](#).

15.8.2.5 get_search_weight_scale_config()

```
SearchWeightScaleConfig qbcompiler.configs.search_weight_scale_config.get_search_weight_scale_config  
(  
    ** kwargs)
```

Create [SearchWeightScaleConfig](#) with partial parameters merged with defaults.

Definition at line 94 of file [search_weight_scale_config.py](#).

15.8.3 Variable Documentation

15.8.3.1 optional

```
bool qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.optional = False [static]
```

Definition at line 16 of file [search_weight_scale_config.py](#).

15.8.3.2 DEFAULTS

`qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.DEFAULTS` [static]

Initial value:

```
= {  
    "apply": False,  
    "query": False,  
    "key": False,  
    "value": False,  
    "out": False,  
    "ffn": False,  
}
```

Definition at line 17 of file [search_weight_scale_config.py](#).

15.8.3.3 apply

`qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.apply = apply`

Definition at line 47 of file [search_weight_scale_config.py](#).

15.8.3.4 query

`qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.query = query`

Definition at line 48 of file [search_weight_scale_config.py](#).

15.8.3.5 key

`qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.key = key`

Definition at line 49 of file [search_weight_scale_config.py](#).

15.8.3.6 value

`qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.value = value`

Definition at line 50 of file [search_weight_scale_config.py](#).

15.8.3.7 out

`qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.out = out`

Definition at line 51 of file [search_weight_scale_config.py](#).

15.8.3.8 ffn

`qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig.ffn = ffn`

Definition at line 52 of file [search_weight_scale_config.py](#).

15.9 Mod Configuration

Minimum Output Difference configuration definitions used by Mobilint compiler for model quantization.

Classes

- class [qbcompiler.configs.mod_config.ModConfig](#)
Configuration for Minimum Output Difference algorithm.

Functions

- [qbcompiler.configs.mod_config.ModConfig.__init__](#) (self, bool apply, int epochs, int warmup_epochs, numbers.Number lr_act_scale, numbers.Number lr_zeropoint, numbers.Number lr_weight_scale, numbers.Number lr_weight, numbers.Number lr_bias, numbers.Number lr_min_ratio, int batch_size, numbers.Number q_drop, str post_processing, numbers.Number box_conf_thres, numbers.Number box_iou_thres, str loss_type, bool use_outputs, numbers.Number kl_temperature, numbers.Number recon_prob, numbers.Number recon_coeff, numbers.Number lambda_0, numbers.Number lambda_1, numbers.Number lambda_2, List[str] anchors, bool use_xyxy, str save_dir, int seed, List[str] apply_layers, List[str] exclude_layers, str custom_loss_jit_path, str mod_after_layer_name, bool optional=optional)
Initialize the **ModConfig**.
- [qbcompiler.configs.mod_config.ModConfig.default_config](#) (cls)
- [qbcompiler.configs.mod_config.ModConfig.from_dict](#) (cls, dict config_dict)
Create **ModConfig** from dictionary (JSON structure)
- [qbcompiler.configs.mod_config.ModConfig.to_dict](#) (self)
- [ModConfig qbcompiler.configs.mod_config.get_mod_config](#) (**kwargs)
Create **ModConfig** with partial parameters merged with defaults.

Variables

- bool [qbcompiler.configs.mod_config.ModConfig.optional](#) = True
- dict [qbcompiler.configs.mod_config.ModConfig.DEFAULTS](#)
- dict [qbcompiler.configs.mod_config.ModConfig.py_to_cpp_name](#)
- dict [qbcompiler.configs.mod_config.ModConfig.min_max_check](#)
- [qbcompiler.configs.mod_config.ModConfig.apply](#) = apply
- [qbcompiler.configs.mod_config.ModConfig.epochs](#) = epochs
- [qbcompiler.configs.mod_config.ModConfig.warmup_epochs](#) = warmup_epochs
- [qbcompiler.configs.mod_config.ModConfig.lr_act_scale](#) = lr_act_scale
- [qbcompiler.configs.mod_config.ModConfig.lr_zeropoint](#) = lr_zeropoint
- [qbcompiler.configs.mod_config.ModConfig.lr_weight_scale](#) = lr_weight_scale

- `qbcompiler.configs.mod_config.ModConfig.lr_weight = lr_weight`
- `qbcompiler.configs.mod_config.ModConfig.lr_bias = lr_bias`
- `qbcompiler.configs.mod_config.ModConfig.lr_min_ratio = lr_min_ratio`
- `qbcompiler.configs.mod_config.ModConfig.batch_size = batch_size`
- `qbcompiler.configs.mod_config.ModConfig.q_drop = q_drop`
- `qbcompiler.configs.mod_config.ModConfig.post_processing = post_processing`
- `qbcompiler.configs.mod_config.ModConfig.box_conf_thres = box_conf_thres`
- `qbcompiler.configs.mod_config.ModConfig.box_iou_thres = box_iou_thres`
- `qbcompiler.configs.mod_config.ModConfig.loss_type = loss_type`
- `qbcompiler.configs.mod_config.ModConfig.use_outputs = use_outputs`
- `qbcompiler.configs.mod_config.ModConfig.kl_temperature = kl_temperature`
- `qbcompiler.configs.mod_config.ModConfig.recon_prob = recon_prob`
- `qbcompiler.configs.mod_config.ModConfig.recon_coeff = recon_coeff`
- `qbcompiler.configs.mod_config.ModConfig.lambda_0 = lambda_0`
- `qbcompiler.configs.mod_config.ModConfig.lambda_1 = lambda_1`
- `qbcompiler.configs.mod_config.ModConfig.lambda_2 = lambda_2`
- `qbcompiler.configs.mod_config.ModConfig.anchors = anchors`
- `qbcompiler.configs.mod_config.ModConfig.use_xyxy = use_xyxy`
- `qbcompiler.configs.mod_config.ModConfig.save_dir = save_dir`
- `qbcompiler.configs.mod_config.ModConfig.seed = seed`
- `qbcompiler.configs.mod_config.ModConfig.apply_layers = apply_layers`
- `qbcompiler.configs.mod_config.ModConfig.exclude_layers = exclude_layers`
- `qbcompiler.configs.mod_config.ModConfig.custom_loss_jit_path = custom_loss_jit_path`
- `qbcompiler.configs.mod_config.ModConfig.mod_after_layer_name = mod_after_layer_name`

15.9.1 Detailed Description

Minimum Output Difference configuration definitions used by Mobilint compiler for model quantization.

15.9.2 Function Documentation

15.9.2.1 `__init__()`

```
qbcompiler.configs.mod_config.ModConfig.__init__(
    self,
    bool apply,
    int epochs,
    int warmup_epochs,
    numbers.Number lr_act_scale,
    numbers.Number lr_zeropoint,
    numbers.Number lr_weight_scale,
    numbers.Number lr_weight,
    numbers.Number lr_bias,
    numbers.Number lr_min_ratio,
    int batch_size,
    numbers.Number q_drop,
    str post_processing,
    numbers.Number box_conf_thres,
```

```

numbers.Number box_iou_thres,
str loss_type,
bool use_outputs,
numbers.Number kl_temperature,
numbers.Number recon_prob,
numbers.Number recon_coeff,
numbers.Number lambda_0,
numbers.Number lambda_1,
numbers.Number lambda_2,
List anchors,
bool use_xyxy,
str save_dir,
int seed,
List[str] apply_layers,
List[str] exclude_layers,
str custom_loss_jit_path,
str mod_after_layer_name,
bool optional = optional)

```

Initialize the [ModConfig](#).

Parameters

apply	bool. Whether to apply MOD.
epochs	int. Number of training epochs.
warmup_epochs	int. Number of warmup epochs.
lr_act_scale	float. Learning rate for activation scale.
lr_zeropoint	float. Learning rate for zeropoint.
lr_weight_scale	float. Learning rate for weight scale.
lr_weight	float. Learning rate for weight.
lr_bias	float. Learning rate for bias.
lr_min_ratio	float. Minimum learning rate ratio for scheduler.
batch_size	int. Batch size for training.
q_drop	float. QDrop ratio.
post_processing	str. Post-processing function.
box_conf_thres	float. Box confidence threshold for NMS.
box_iou_thres	float. Box IoU threshold for NMS.
loss_type	str. Loss function type.
use_outputs	bool. Compare only intermediate layers.
kl_temperature	float. Temperature for KL divergence loss.
recon_prob	float. Sampling probability for intermediate layers.
recon_coeff	float. Coefficient for intermediate layer loss.
lambda_0	float. First coefficient for AnchorlessDetection loss.
lambda_1	float. Second coefficient for AnchorlessDetection loss.
lambda_2	float. Third coefficient for AnchorlessDetection loss.
anchors	List. Anchor boxes configuration.
use_xyxy	bool. Use XYXY format.
save_dir	str. Directory to save results.
seed	int. Random seed.
apply_layers	List[str]. Layers to apply MOD.

Parameters

exclude_layers	List[str]. Layers to exclude from MOD.
custom_loss_jit_path	str. Path to custom loss JIT file.
mod_after_layer_name	str. Layer name to apply MOD after.
optional	bool. Indicates whether this config is optional. Use the default value defined as a class variable.

Definition at line 112 of file [mod_config.py](#).

15.9.2.2 default_config()

```
qbcompiler.configs.mod_config.ModConfig.default_config (
    cls)
```

Definition at line 216 of file [mod_config.py](#).

15.9.2.3 from_dict()

```
qbcompiler.configs.mod_config.ModConfig.from_dict (
    cls,
    dict config_dict)
```

Create [ModConfig](#) from dictionary (JSON structure)

Definition at line 220 of file [mod_config.py](#).

15.9.2.4 to_dict()

```
qbcompiler.configs.mod_config.ModConfig.to_dict (
    self)
```

Definition at line 304 of file [mod_config.py](#).

15.9.2.5 get_mod_config()

```
ModConfig qbcompiler.configs.mod_config.get_mod_config (
    ** kwargs)
```

Create [ModConfig](#) with partial parameters merged with defaults.

Definition at line 349 of file [mod_config.py](#).

15.9.3 Variable Documentation

15.9.3.1 optional

`bool qbcompiler.configs.mod_config.ModConfig.optional = True [static]`

Definition at line 21 of file [mod_config.py](#).

15.9.3.2 DEFAULTS

`qbcompiler.configs.mod_config.ModConfig.DEFAULTS [static]`

Definition at line 22 of file [mod_config.py](#).

15.9.3.3 py_to_cpp_name

`dict qbcompiler.configs.mod_config.ModConfig.py_to_cpp_name [static]`

Initial value:

```
= {
    "apply": "apply",
    "epochs": "epochs",
    "warmup_epochs": "warmupEpochs",
    "lr_min_ratio": "lrMinRatio",
    "lr_act_scale": "actScaleLR",
    "lr_zeropoint": "zeropointLR",
    "lr_weight_scale": "weightScaleLR",
    "lr_weight": "weightLR",
    "lr_bias": "biasLR",
    "batch_size": "batchSize",
    "q_drop": "qDrop",
    "post_processing": "post",
    "box_conf_thres": "boxConfThres",
    "box_iou_thres": "boxIoUThres",
    "loss_type": "lossType",
    "use_outputs": "useOutputs",
    "kl_temperature": "KLTemperature",
    "recon_prob": "reconProb",
    "recon_coeff": "reconCoeff",
    "lambda_0": "lambda_0",
    "lambda_1": "lambda_1",
    "lambda_2": "lambda_2",
    "use_xyxy": "useXYXY",
    "save_dir": "saveDir",
    "seed": "seed",
    "custom_loss_jit_path": "customLossJITPath",
    "mod_after_layer_name": "modAfterLayerName",
}
```

Definition at line 59 of file [mod_config.py](#).

15.9.3.4 min_max_check

dict qbcompiler.configs.mod_config.ModConfig.min_max_check [static]

Initial value:

```
= {
    "epochs": [0, None],
    "warmup_epochs": [0, None],
    "lr_min_ratio": [0, None],
    "lr_act_scale": [0, None],
    "lr_zeropoint": [0, None],
    "lr_weight_scale": [0, None],
    "lr_weight": [0, None],
    "lr_bias": [0, None],
    "batch_size": [0, None],
    "q_drop": [0, 1],
    "box_conf_thres": [0, None],
    "box_iou_thres": [0, None],
    "kl_temperature": [0, None],
    "recon_prob": [0, 1],
    "recon_coeff": [0, None],
    "lambda_0": [0, None],
    "lambda_1": [0, None],
    "lambda_2": [0, None],
    "seed": [0, None],
}
```

Definition at line 89 of file [mod_config.py](#).

15.9.3.5 apply

qbcompiler.configs.mod_config.ModConfig.apply = apply

Definition at line 181 of file [mod_config.py](#).

15.9.3.6 epochs

qbcompiler.configs.mod_config.ModConfig.epochs = epochs

Definition at line 182 of file [mod_config.py](#).

15.9.3.7 warmup_epochs

qbcompiler.configs.mod_config.ModConfig.warmup_epochs = warmup_epochs

Definition at line 183 of file [mod_config.py](#).

15.9.3.8 lr_act_scale

qbcompiler.configs.mod_config.ModConfig.lr_act_scale = lr_act_scale

Definition at line 184 of file [mod_config.py](#).

15.9.3.9 lr_zeropoint

```
qbcompiler.configs.mod_config.ModConfig.lr_zeropoint = lr_zeropoint
```

Definition at line 185 of file [mod_config.py](#).

15.9.3.10 lr_weight_scale

```
qbcompiler.configs.mod_config.ModConfig.lr_weight_scale = lr_weight_scale
```

Definition at line 186 of file [mod_config.py](#).

15.9.3.11 lr_weight

```
qbcompiler.configs.mod_config.ModConfig.lr_weight = lr_weight
```

Definition at line 187 of file [mod_config.py](#).

15.9.3.12 lr_bias

```
qbcompiler.configs.mod_config.ModConfig.lr_bias = lr_bias
```

Definition at line 188 of file [mod_config.py](#).

15.9.3.13 lr_min_ratio

```
qbcompiler.configs.mod_config.ModConfig.lr_min_ratio = lr_min_ratio
```

Definition at line 189 of file [mod_config.py](#).

15.9.3.14 batch_size

```
qbcompiler.configs.mod_config.ModConfig.batch_size = batch_size
```

Definition at line 190 of file [mod_config.py](#).

15.9.3.15 q_drop

```
qbcompiler.configs.mod_config.ModConfig.q_drop = q_drop
```

Definition at line 191 of file [mod_config.py](#).

15.9.3.16 post_processing

```
qbcompiler.configs.mod_config.ModConfig.post_processing = post_processing
```

Definition at line 192 of file [mod_config.py](#).

15.9.3.17 box_conf_thres

```
qbcompiler.configs.mod_config.ModConfig.box_conf_thres = box_conf_thres
```

Definition at line 193 of file [mod_config.py](#).

15.9.3.18 box_iou_thres

```
qbcompiler.configs.mod_config.ModConfig.box_iou_thres = box_iou_thres
```

Definition at line 194 of file [mod_config.py](#).

15.9.3.19 loss_type

```
qbcompiler.configs.mod_config.ModConfig.loss_type = loss_type
```

Definition at line 195 of file [mod_config.py](#).

15.9.3.20 use_outputs

```
qbcompiler.configs.mod_config.ModConfig.use_outputs = use_outputs
```

Definition at line 196 of file [mod_config.py](#).

15.9.3.21 kl_temperature

```
qbcompiler.configs.mod_config.ModConfig.kl_temperature = kl_temperature
```

Definition at line 197 of file [mod_config.py](#).

15.9.3.22 recon_prob

```
qbcompiler.configs.mod_config.ModConfig.recon_prob = recon_prob
```

Definition at line 198 of file [mod_config.py](#).

15.9.3.23 recon_coeff

```
qbcompiler.configs.mod_config.ModConfig.recon_coeff = recon_coeff
```

Definition at line 199 of file [mod_config.py](#).

15.9.3.24 lambda_0

```
qbcompiler.configs.mod_config.ModConfig.lambda_0 = lambda_0
```

Definition at line 200 of file [mod_config.py](#).

15.9.3.25 lambda_1

```
qbcompiler.configs.mod_config.ModConfig.lambda_1 = lambda_1
```

Definition at line 201 of file [mod_config.py](#).

15.9.3.26 lambda_2

```
qbcompiler.configs.mod_config.ModConfig.lambda_2 = lambda_2
```

Definition at line 202 of file [mod_config.py](#).

15.9.3.27 anchors

```
qbcompiler.configs.mod_config.ModConfig.anchors = anchors
```

Definition at line 203 of file [mod_config.py](#).

15.9.3.28 use_xyxy

```
qbcompiler.configs.mod_config.ModConfig.use_xyxy = use_xyxy
```

Definition at line 204 of file [mod_config.py](#).

15.9.3.29 save_dir

```
qbcompiler.configs.mod_config.ModConfig.save_dir = save_dir
```

Definition at line 205 of file [mod_config.py](#).

15.9.3.30 seed

```
qbcompiler.configs.mod_config.ModConfig.seed = seed
```

Definition at line 206 of file [mod_config.py](#).

15.9.3.31 apply_layers

```
qbcompiler.configs.mod_config.ModConfig.apply_layers = apply_layers
```

Definition at line 207 of file [mod_config.py](#).

15.9.3.32 exclude_layers

```
qbcompiler.configs.mod_config.ModConfig.exclude_layers = exclude_layers
```

Definition at line 208 of file [mod_config.py](#).

15.9.3.33 custom_loss_jit_path

```
qbcompiler.configs.mod_config.ModConfig.custom_loss_jit_path = custom_loss_jit_path
```

Definition at line 209 of file [mod_config.py](#).

15.9.3.34 mod_after_layer_name

```
qbcompiler.configs.mod_config.ModConfig.mod_after_layer_name = mod_after_layer_name
```

Definition at line 210 of file [mod_config.py](#).

15.10 Optq Configuration

OPTQ configuration definitions used by Mobilint compiler for model quantization.

Classes

- class [qbcompiler.configs.optq_config.OptqConfig](#)
Configuration for OPTQ algorithm.

Functions

- `qbcompiler.configs.optq_config.OptqConfig.__init__` (self, bool apply, bool act_order, int block_size, numbers.Number perc_damp, List[str] apply_layers, List[str] exclude_layers, bool optional=optional)
Initialize the **OptqConfig**.
- `qbcompiler.configs.optq_config.OptqConfig.default_config` (cls)
- `qbcompiler.configs.optq_config.OptqConfig.from_dict` (cls, dict config_dict)
Create **OptqConfig** from dictionary (JSON structure)
- `qbcompiler.configs.optq_config.OptqConfig.to_dict` (self)
- `OptqConfig qbcompiler.configs.optq_config.get_optq_config` (**kwargs)
Create **OptqConfig** with partial parameters merged with defaults.

Variables

- bool `qbcompiler.configs.optq_config.OptqConfig.optional` = True
- dict `qbcompiler.configs.optq_config.OptqConfig.DEFAULTS`
- dict `qbcompiler.configs.optq_config.OptqConfig.py_to_cpp_name`
- dict `qbcompiler.configs.optq_config.OptqConfig.min_max_check`
- `qbcompiler.configs.optq_config.OptqConfig.apply` = apply
- `qbcompiler.configs.optq_config.OptqConfig.act_order` = act_order
- `qbcompiler.configs.optq_config.OptqConfig.block_size` = block_size
- `qbcompiler.configs.optq_config.OptqConfig.perc_damp` = perc_damp
- `qbcompiler.configs.optq_config.OptqConfig.apply_layers` = apply_layers
- `qbcompiler.configs.optq_config.OptqConfig.exclude_layers` = exclude_layers

15.10.1 Detailed Description

OPTQ configuration definitions used by Mobilint compiler for model quantization.

15.10.2 Function Documentation

15.10.2.1 __init__()

```
qbcompiler.configs.optq_config.OptqConfig.__init__ (
    self,
    bool apply,
    bool act_order,
    int block_size,
    numbers.Number perc_damp,
    List[str] apply_layers,
    List[str] exclude_layers,
    bool optional = optional)
```

Initialize the **OptqConfig**.

Parameters

apply	bool. If true, apply OPTQ.
act_order	bool. If true, use activation order.
block_size	int. Block size used for OPTQ.
perc_damp	float. Percentage dampening factor.
apply_layers	List[str]. (optional) Layer names to apply OPTQ. If empty, applies to all eligible layers.
exclude_layers	List[str]. (optional) Layer names to exclude from OPTQ. If empty, no layers are excluded.
optional	bool. Indicates whether this config is optional. Use the default value defined as a class variable.

Definition at line [43](#) of file [optq_config.py](#).

15.10.2.2 default_config()

```
qbcompiler.configs.optq_config.OptqConfig.default_config (
    cls)
```

Definition at line [73](#) of file [optq_config.py](#).

15.10.2.3 from_dict()

```
qbcompiler.configs.optq_config.OptqConfig.from_dict (
    cls,
    dict config_dict)
```

Create [OptqConfig](#) from dictionary (JSON structure)

Definition at line [77](#) of file [optq_config.py](#).

15.10.2.4 to_dict()

```
qbcompiler.configs.optq_config.OptqConfig.to_dict (
    self)
```

Definition at line [99](#) of file [optq_config.py](#).

15.10.2.5 get_optq_config()

```
OptqConfig qbcompiler.configs.optq_config.get_optq_config (
    ** kwargs)
```

Create [OptqConfig](#) with partial parameters merged with defaults.

Definition at line [112](#) of file [optq_config.py](#).

15.10.3 Variable Documentation

15.10.3.1 optional

`bool qbcompiler.configs.optq_config.OptqConfig.optional = True [static]`

Definition at line 21 of file `optq_config.py`.

15.10.3.2 DEFAULTS

`qbcompiler.configs.optq_config.OptqConfig.DEFAULTS [static]`

Initial value:

```
= {  
    "apply": False,  
    "act_order": True,  
    "block_size": 128,  
    "perc_damp": 0.01,  
    "apply_layers": [],  
    "exclude_layers": [],  
}
```

Definition at line 22 of file `optq_config.py`.

15.10.3.3 py_to_cpp_name

`dict qbcompiler.configs.optq_config.OptqConfig.py_to_cpp_name [static]`

Initial value:

```
= {  
    "apply": "apply",  
    "act_order": "actOrder",  
    "block_size": "blockSize",  
    "perc_damp": "percDamp",  
}
```

Definition at line 31 of file `optq_config.py`.

15.10.3.4 min_max_check

`dict qbcompiler.configs.optq_config.OptqConfig.min_max_check [static]`

Initial value:

```
= {  
    "block_size": [0, None],  
    "perc_damp": [0.0, 1.0],  
}
```

Definition at line 38 of file `optq_config.py`.

15.10.3.5 apply

```
qbcompiler.configs.optq_config.OptqConfig.apply = apply
```

Definition at line 64 of file [optq_config.py](#).

15.10.3.6 act_order

```
qbcompiler.configs.optq_config.OptqConfig.act_order = act_order
```

Definition at line 65 of file [optq_config.py](#).

15.10.3.7 block_size

```
qbcompiler.configs.optq_config.OptqConfig.block_size = block_size
```

Definition at line 66 of file [optq_config.py](#).

15.10.3.8 perc_damp

```
qbcompiler.configs.optq_config.OptqConfig.perc_damp = perc_damp
```

Definition at line 67 of file [optq_config.py](#).

15.10.3.9 apply_layers

```
qbcompiler.configs.optq_config.OptqConfig.apply_layers = apply_layers
```

Definition at line 68 of file [optq_config.py](#).

15.10.3.10 exclude_layers

```
qbcompiler.configs.optq_config.OptqConfig.exclude_layers = exclude_layers
```

Definition at line 69 of file [optq_config.py](#).

15.11 Input Process Configuration

Input process configuration definitions used by Mobilint compiler for input processing.

Classes

- class `qbcompiler.configs.input_process_config.Uint8InputConfig`
Configuration for uint8 input handling.
- class `qbcompiler.configs.input_process_config.PreprocessingConfig`
Configuration for input preprocessing pipeline.
- class `qbcompiler.configs.input_process_config.InputProcessConfig`
Unified configuration for input processing.

Functions

- `qbcompiler.configs.input_process_config.Uint8InputConfig.__init__` (self, bool apply, List[str] inputs, bool optional=optional)
Initialize the `Uint8InputConfig`.
- `qbcompiler.configs.input_process_config.Uint8InputConfig.default_config` (cls)
- `qbcompiler.configs.input_process_config.Uint8InputConfig.from_dict` (cls, dict config_dict)
Create `Uint8InputConfig` from dictionary (JSON structure)
- `qbcompiler.configs.input_process_config.Uint8InputConfig.to_dict` (self)
- `Uint8InputConfig qbcompiler.configs.input_process_config.get_uint8_input_config` (**kwargs)
Create `Uint8InputConfig` with partial parameters merged with defaults.
- `qbcompiler.configs.input_process_config.PreprocessingConfig.__init__` (self, bool apply, bool auto_convert_format, List[Dict[str, Any]] pipeline, Dict[str, Any] input_configs, bool optional=optional)
Initialize the `PreprocessingConfig`.
- `qbcompiler.configs.input_process_config.PreprocessingConfig.default_config` (cls)
- `qbcompiler.configs.input_process_config.PreprocessingConfig.from_dict` (cls, dict config_dict)
Create `PreprocessingConfig` from dictionary (JSON structure)
- `qbcompiler.configs.input_process_config.PreprocessingConfig.to_dict` (self)
- `PreprocessingConfig qbcompiler.configs.input_process_config.get_preprocessing_config` (**kwargs)
Create `PreprocessingConfig` with partial parameters merged with defaults.
- `qbcompiler.configs.input_process_config.InputProcessConfig.__init__` (self, Optional[`Uint8InputConfig`] uint8_input, int image_channels, Optional[`PreprocessingConfig`] preprocessing, bool optional=optional)
Initialize the `InputProcessConfig`.
- `qbcompiler.configs.input_process_config.InputProcessConfig.default_config` (cls)
- `qbcompiler.configs.input_process_config.InputProcessConfig.from_dict` (cls, dict config_dict)
Create `InputProcessConfig` from dictionary (JSON structure)
- `qbcompiler.configs.input_process_config.InputProcessConfig.to_dict` (self)
- `InputProcessConfig qbcompiler.configs.input_process_config.get_input_process_config` (Optional[`Uint8InputConfig`] uint8_input_config=None, int image_channels=0, Optional[`PreprocessingConfig`] preprocessing_config=None)
Create `InputProcessConfig` with partial parameters merged with defaults.

Variables

- bool `qbcompiler.configs.input_process_config.Uint8InputConfig.optional` = True
- dict `qbcompiler.configs.input_process_config.Uint8InputConfig.DEFAULTS`
- dict `qbcompiler.configs.input_process_config.Uint8InputConfig.py_to_cpp_name`

- `dict qbcompiler.configs.input_process_config.Uint8InputConfig.min_max_check = {}`
- `qbcompiler.configs.input_process_config.Uint8InputConfig.apply = apply`
- `qbcompiler.configs.input_process_config.Uint8InputConfig.inputs = inputs`
- `bool qbcompiler.configs.input_process_config.PreprocessingConfig.optional = True`
- `dict qbcompiler.configs.input_process_config.PreprocessingConfig.DEFAULTS`
- `dict qbcompiler.configs.input_process_config.PreprocessingConfig.py_to_cpp_name`
- `dict qbcompiler.configs.input_process_config.PreprocessingConfig.min_max_check = {}`
- `qbcompiler.configs.input_process_config.PreprocessingConfig.apply = apply`
- `qbcompiler.configs.input_process_config.PreprocessingConfig.auto_convert_format = auto_convert_format`
- `qbcompiler.configs.input_process_config.PreprocessingConfig.pipeline = pipeline`
- `qbcompiler.configs.input_process_config.PreprocessingConfig.input_configs = input_configs`
- `bool qbcompiler.configs.input_process_config.InputProcessConfig.optional = True`
- `dict qbcompiler.configs.input_process_config.InputProcessConfig.DEFAULTS`
- `dict qbcompiler.configs.input_process_config.InputProcessConfig.py_to_cpp_name`
- `dict qbcompiler.configs.input_process_config.InputProcessConfig.min_max_check = {}`
- `tuple qbcompiler.configs.input_process_config.InputProcessConfig.uint8_input`
- `qbcompiler.configs.input_process_config.InputProcessConfig.image_channels = image_channels`
- `tuple qbcompiler.configs.input_process_config.InputProcessConfig.preprocessing`

15.11.1 Detailed Description

Input process configuration definitions used by Mobilint compiler for input processing.

15.11.2 Function Documentation

15.11.2.1 `__init__()` [1/3]

```
qbcompiler.configs.input_process_config.Uint8InputConfig.__init__(
    self,
    bool apply,
    List[str] inputs,
    bool optional = optional)
```

Initialize the `Uint8InputConfig`.

Parameters

apply	bool. If true, treat specified inputs as uint8.
inputs	List[str]. List of input names to treat as uint8. If empty and apply is true, applies to all inputs.
optional	bool. Indicates whether this config is optional. Use the default value defined as a class variable.

Definition at line 33 of file `input_process_config.py`.

15.11.2.2 default_config() [1/3]

```
qbcompiler.configs.input_process_config.Uint8InputConfig.default_config (
    cls)
```

Definition at line 51 of file [input_process_config.py](#).

15.11.2.3 from_dict() [1/3]

```
qbcompiler.configs.input_process_config.Uint8InputConfig.from_dict (
    cls,
    dict config_dict)
```

Create [Uint8InputConfig](#) from dictionary (JSON structure)

Definition at line 55 of file [input_process_config.py](#).

15.11.2.4 to_dict() [1/3]

```
qbcompiler.configs.input_process_config.Uint8InputConfig.to_dict (
    self)
```

Definition at line 66 of file [input_process_config.py](#).

15.11.2.5 get_uint8_input_config()

```
Uint8InputConfig qbcompiler.configs.input_process_config.get_uint8_input_config (
    ** kwargs)
```

Create [Uint8InputConfig](#) with partial parameters merged with defaults.

Definition at line 73 of file [input_process_config.py](#).

15.11.2.6 __init__() [2/3]

```
qbcompiler.configs.input_process_config.PreprocessingConfig.__init__ (
    self,
    bool apply,
    bool auto_convert_format,
    List[Dict[str, Any]] pipeline,
    Dict[str, Any] input_configs,
    bool optional = optional)
```

Initialize the [PreprocessingConfig](#).

Parameters

apply	bool. If true, apply preprocessing pipeline.
auto_convert_format	bool. If true, automatically convert input format.
pipeline	List[Dict[str, Any]]. List of preprocessing operations to apply globally. Each operation is a dict with "op" key and operation-specific parameters. Supported operations: colorConvert, resize, centerCrop, pad, letterbox, normalize, clamp, divideByScale, multiplyByScale, reshape, transpose, permute.
input_configs	Dict[str, Any]. Per-input preprocessing configurations. Keys are input names, values are dicts containing input-specific pipelines.
optional	bool. Indicates whether this config is optional. Use the default value defined as a class variable.

Definition at line 106 of file [input_process_config.py](#).

15.11.2.7 default_config() [2/3]

```
qbcompiler.configs.input_process_config.PreprocessingConfig.default_config (
    cls)
```

Definition at line 134 of file [input_process_config.py](#).

15.11.2.8 from_dict() [2/3]

```
qbcompiler.configs.input_process_config.PreprocessingConfig.from_dict (
    cls,
    dict config_dict)
```

Create [PreprocessingConfig](#) from dictionary (JSON structure)

Definition at line 138 of file [input_process_config.py](#).

15.11.2.9 to_dict() [2/3]

```
qbcompiler.configs.input_process_config.PreprocessingConfig.to_dict (
    self)
```

Definition at line 153 of file [input_process_config.py](#).

15.11.2.10 get_preprocessing_config()

```
PreprocessingConfig qbcompiler.configs.input_process_config.get_preprocessing_config (
    ** kwargs)
```

Create [PreprocessingConfig](#) with partial parameters merged with defaults.

Definition at line 162 of file [input_process_config.py](#).

15.11.2.11 __init__() [3/3]

```
qbcompiler.configs.input_process_config.InputProcessConfig.__init__ (
    self,
    Optional[UInt8InputConfig] uint8_input,
    int image_channels,
    Optional[PreprocessingConfig] preprocessing,
    bool optional = optional)
```

Initialize the [InputProcessConfig](#).

Parameters

uint8_input	UInt8InputConfig . Configuration for uint8 input handling. If None, uses default UInt8InputConfig .
image_channels	int. Number of image channels (0=auto, 1=grayscale, 3=RGB, 4=RGBA).
preprocessing	PreprocessingConfig . Configuration for input preprocessing pipeline. If None, uses default PreprocessingConfig .
optional	bool. Indicates whether this config is optional.

Definition at line [193](#) of file [input_process_config.py](#).

15.11.2.12 default_config() [3/3]

```
qbcompiler.configs.input_process_config.InputProcessConfig.default_config (
    cls)
```

Definition at line [224](#) of file [input_process_config.py](#).

15.11.2.13 from_dict() [3/3]

```
qbcompiler.configs.input_process_config.InputProcessConfig.from_dict (
    cls,
    dict config_dict)
```

Create [InputProcessConfig](#) from dictionary (JSON structure)

Definition at line [233](#) of file [input_process_config.py](#).

15.11.2.14 to_dict() [3/3]

```
qbcompiler.configs.input_process_config.InputProcessConfig.to_dict (
    self)
```

Definition at line [247](#) of file [input_process_config.py](#).

15.11.2.15 get_input_process_config()

```
InputProcessConfig qbcompiler.configs.input_process_config.get_input_process_config (
    Optional[UInt8InputConfig] uint8_input_config = None,
    int image_channels = 0,
    Optional[PreprocessingConfig] preprocessing_config = None)
```

Create [InputProcessConfig](#) with partial parameters merged with defaults.

Parameters

uint8_input_config	UInt8InputConfig . Configuration for uint8 input handling.
image_channels	int. Number of image channels (0=auto, 1=grayscale, 3=RGB, 4=RGBA).
preprocessing_config	PreprocessingConfig . Configuration for input preprocessing pipeline.

Returns

[InputProcessConfig](#). The configured input process configuration.

Definition at line 255 of file [input_process_config.py](#).

15.11.3 Variable Documentation

15.11.3.1 optional [1/3]

```
bool qbcompiler.configs.input_process_config.UInt8InputConfig.optional = True [static]
```

Definition at line 20 of file [input_process_config.py](#).

15.11.3.2 DEFAULTS [1/3]

```
qbcompiler.configs.input_process_config.UInt8InputConfig.DEFAULTS [static]
```

Initial value:

```
= {
    "apply": False,
    "inputs": [],
}
```

Definition at line 21 of file [input_process_config.py](#).

15.11.3.3 py_to_cpp_name [1/3]

```
dict qbcompiler.configs.input_process_config.UInt8InputConfig.py_to_cpp_name [static]
```

Initial value:

```
= {
    "apply": "apply",
}
```

Definition at line 26 of file [input_process_config.py](#).

15.11.3.4 min_max_check [1/3]

```
dict qbcompiler.configs.input_process_config.Uint8InputConfig.min_max_check = {} [static]
```

Definition at line 30 of file [input_process_config.py](#).

15.11.3.5 apply [1/2]

```
qbcompiler.configs.input_process_config.Uint8InputConfig.apply = apply
```

Definition at line 46 of file [input_process_config.py](#).

15.11.3.6 inputs

```
qbcompiler.configs.input_process_config.Uint8InputConfig.inputs = inputs
```

Definition at line 47 of file [input_process_config.py](#).

15.11.3.7 optional [2/3]

```
bool qbcompiler.configs.input_process_config.PreprocessingConfig.optional = True [static]
```

Definition at line 90 of file [input_process_config.py](#).

15.11.3.8 DEFAULTS [2/3]

```
qbcompiler.configs.input_process_config.PreprocessingConfig.DEFAULTS [static]
```

Initial value:

```
= {
    "apply": False,
    "auto_convert_format": False,
    "pipeline": [],
    "input_configs": {},
}
```

Definition at line 91 of file [input_process_config.py](#).

15.11.3.9 py_to_cpp_name [2/3]

```
dict qbcompiler.configs.input_process_config.PreprocessingConfig.py_to_cpp_name [static]
```

Initial value:

```
= {
    "apply": "apply",
    "auto_convert_format": "autoConvertFormat",
}
```

Definition at line 98 of file [input_process_config.py](#).

15.11.3.10 min_max_check [2/3]

```
dict qbcompiler.configs.input_process_config.PreprocessingConfig.min_max_check = {} [static]
```

Definition at line 103 of file [input_process_config.py](#).

15.11.3.11 apply [2/2]

```
qbcompiler.configs.input_process_config.PreprocessingConfig.apply = apply
```

Definition at line 127 of file [input_process_config.py](#).

15.11.3.12 auto_convert_format

```
qbcompiler.configs.input_process_config.PreprocessingConfig.auto_convert_format = auto_convert_format
```

Definition at line 128 of file [input_process_config.py](#).

15.11.3.13 pipeline

```
qbcompiler.configs.input_process_config.PreprocessingConfig.pipeline = pipeline
```

Definition at line 129 of file [input_process_config.py](#).

15.11.3.14 input_configs

```
qbcompiler.configs.input_process_config.PreprocessingConfig.input_configs = input_configs
```

Definition at line 130 of file [input_process_config.py](#).

15.11.3.15 optional [3/3]

```
bool qbcompiler.configs.input_process_config.InputProcessConfig.optional = True [static]
```

Definition at line 179 of file [input_process_config.py](#).

15.11.3.16 DEFAULTS [3/3]

```
dict qbcompiler.configs.input_process_config.InputProcessConfig.DEFAULTS [static]
```

Initial value:

```
= {
    "uint8_input": None, # Will use Uint8InputConfig.default_config()
    "image_channels": 0,
    "preprocessing": None, # Will use PreprocessingConfig.default_config()
}
```

Definition at line 180 of file [input_process_config.py](#).

15.11.3.17 py_to_cpp_name [3/3]

dict qbcompiler.configs.input_process_config.InputProcessConfig.py_to_cpp_name [static]

Initial value:

```
= {
    "image_channels": "imageChannels",
}
```

Definition at line 186 of file [input_process_config.py](#).

15.11.3.18 min_max_check [3/3]

dict qbcompiler.configs.input_process_config.InputProcessConfig.min_max_check = {} [static]

Definition at line 190 of file [input_process_config.py](#).

15.11.3.19 uint8_input

tuple qbcompiler.configs.input_process_config.InputProcessConfig.uint8_input

Initial value:

```
= (
    uint8_input if uint8_input is not None else
    UInt8InputConfig.default_config()
)
```

Definition at line 210 of file [input_process_config.py](#).

15.11.3.20 image_channels

qbcompiler.configs.input_process_config.InputProcessConfig.image_channels = image_channels

Definition at line 213 of file [input_process_config.py](#).

15.11.3.21 preprocessing

tuple qbcompiler.configs.input_process_config.InputProcessConfig.preprocessing

Initial value:

```
= (
    preprocessing
    if preprocessing is not None
    else PreprocessingConfig.default_config()
)
```

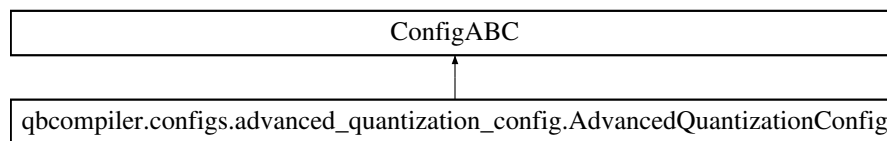
Definition at line 214 of file [input_process_config.py](#).

16 Class Documentation

16.1 qbcompiler.configs.advanced_quantization_config.↵ AdvancedQuantizationConfig Class Reference

Configuration for advanced quantization options used during quantization.

Inheritance diagram for qbcompiler.configs.advanced_quantization_config.AdvancedQuantizationConfig:↵



Public Member Functions

- `__init__` (self, Optional[OptqConfig] optq=None, Optional[ModConfig] mod=None, Optional[EquivalentTransformationConfig] equivalent_transformation=None, Optional[SearchWeightScaleConfig] search_weight_scale=None, bool optional=optional)
- `default_config` (cls)
- `from_dict` (cls, dict config_dict)
Create **AdvancedQuantizationConfig** from dictionary (JSON structure)
- `to_dict` (self)

Public Attributes

- `optq` = optq or OptqConfig.default_config()
- `mod` = mod or ModConfig.default_config()
- tuple `equivalent_transformation`
- tuple `search_weight_scale`

Static Public Attributes

- bool `optional` = False

16.1.1 Detailed Description

Configuration for advanced quantization options used during quantization.

Unified configuration for advanced quantization techniques, including OPTQ, MOD, Equivalent Transformation, and Weight Scale Search.

Definition at line 17 of file [advanced_quantization_config.py](#).

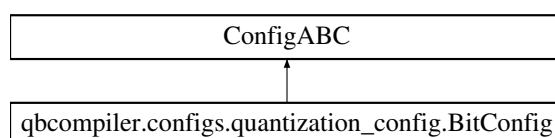
The documentation for this class was generated from the following file:

- `advanced_quantization_config.py`

16.2 qbcompiler.configs.quantization_config.BitConfig Class Reference

Configuration for bit precision.

Inheritance diagram for qbcompiler.configs.quantization_config.BitConfig:



Public Member Functions

- `__init__` (self, int query_act_bits, int key_act_bits, int value_act_bits, int output_act_bits, int ffn_act_bits, int head_act_bits, int query_weight_bits, int key_weight_bits, int value_weight_bits, int output_weight_bits, int ffn_weight_bits, int head_weight_bits, bool mixed_precision_apply, str save_path, str load_path, List[str] activation_16bits, List[str] weight_16bits, bool optional=optional)
Initialize the **BitConfig**.
- `default_config` (cls)
- `from_kwargs` (cls, **kwargs)
- `from_dict` (cls, dict config_dict)
Create **BitConfig** from dictionary (JSON structure)
- `to_dict` (self)

Public Attributes

- `query_act_bits` = query_act_bits
- `key_act_bits` = key_act_bits
- `value_act_bits` = value_act_bits
- `output_act_bits` = output_act_bits
- `ffn_act_bits` = ffn_act_bits
- `head_act_bits` = head_act_bits
- `query_weight_bits` = query_weight_bits
- `key_weight_bits` = key_weight_bits
- `value_weight_bits` = value_weight_bits
- `output_weight_bits` = output_weight_bits
- `ffn_weight_bits` = ffn_weight_bits
- `head_weight_bits` = head_weight_bits
- `mixed_precision_apply` = mixed_precision_apply
- `save_path` = save_path
- `load_path` = load_path
- `activation_16bits` = activation_16bits
- `weight_16bits` = weight_16bits

Static Public Attributes

- bool `optional` = False
- dict `DEFAULTS`
- dict `py_to_cpp_name`
- dict `min_max_check`

16.2.1 Detailed Description

Configuration for bit precision.

Defines bit-width parameterization for activations and weights used in mixed-precision quantization (e.g., attention and FFN components).

Definition at line 316 of file `quantization_config.py`.

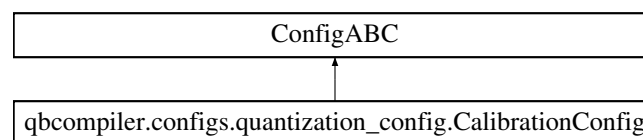
The documentation for this class was generated from the following file:

- `quantization_config.py`

16.3 qbcompiler.configs.quantization_config.CalibrationConfig Class Reference

Configuration for calibration during quantization.

Inheritance diagram for `qbcompiler.configs.quantization_config.CalibrationConfig`:



Public Member Functions

- `__init__` (self, int quantization_method, int quantization_output, int quantization_mode, numbers.Number percentile, numbers.Number topk_ratio, int max_each, int max_total, numbers.Number hist_percentile, bool hist_use_gpu, int hist_num_bins, int hist_num_samples, int hist_buffer_size, numbers.Number hist_min_bin_width, numbers.Number hist_search_percentile_min, numbers.Number hist_search_percentile_max, int hist_num_search, int hist_search_type, numbers.Number act_scale_min, numbers.Number weight_scale_min, numbers.Number min_clip_ratio, Dict layer_overrides, bool optional=optional)
Initialize the **CalibrationConfig**.
- `default_config` (cls)
- `from_kwargs` (cls, **kwargs)
- `from_dict` (cls, dict config_dict)
Create **CalibrationConfig** from dictionary (JSON structure)
- `to_dict` (self)

Public Attributes

- `quantization_method` = `quantization_method`
- `quantization_output` = `quantization_output`
- `quantization_mode` = `quantization_mode`
- `percentile` = `percentile`
- `topk_ratio` = `topk_ratio`
- `max_each` = `max_each`
- `max_total` = `max_total`
- `hist_percentile` = `hist_percentile`
- `hist_use_gpu` = `hist_use_gpu`
- `hist_num_bins` = `hist_num_bins`
- `hist_num_samples` = `hist_num_samples`
- `hist_buffer_size` = `hist_buffer_size`
- `hist_min_bin_width` = `hist_min_bin_width`
- `hist_search_percentile_min` = `hist_search_percentile_min`
- `hist_search_percentile_max` = `hist_search_percentile_max`
- `hist_num_search` = `hist_num_search`
- `hist_search_type` = `hist_search_type`
- `act_scale_min` = `act_scale_min`
- `weight_scale_min` = `weight_scale_min`
- `min_clip_ratio` = `min_clip_ratio`
- `layer_overrides` = `layer_overrides`

Static Public Attributes

- `bool optional` = `False`
- `list method_list` = `["WChALayer", "WChAMulti", "WChALayerZeropoint", "WChAMultiZeropoint"]`
- `list output_list` = `["Layer", "Ch", "Sigmoid"]`
- `list mode_list` = `["max", "maxPercentile", "histogram"]`
- `list search_type_list` = `["percentile", "mse", "kl"]`
- `dict DEFAULTS`
- `dict py_to_cpp_name`
- `dict min_max_check`

16.3.1 Detailed Description

Configuration for calibration during quantization.

Defines calibration and quantization parameterization used to derive activation/weight scales and related statistics during quantized compilation.

Definition at line 14 of file [quantization_config.py](#).

The documentation for this class was generated from the following file:

- `quantization_config.py`

16.4 qbcompiler.calibration.preprocess.CenterCrop Class Reference

Perform a centered crop on the image.

Public Member Functions

- `__init__` (self, List[int] size)
- `__call__` (self, np.ndarray img)

Public Attributes

- int `size` = size * 2

16.4.1 Detailed Description

Perform a centered crop on the image.

Parameters

size	list[int]. Target height and width.
-------------	-------------------------------------

Definition at line 583 of file `preprocess.py`.

16.4.2 Constructor & Destructor Documentation

16.4.2.1 __init__()

```
qbcompiler.calibration.preprocess.CenterCrop.__init__ (
    self,
    List[int] size)
```

Definition at line 590 of file `preprocess.py`.

16.4.3 Member Function Documentation

16.4.3.1 __call__()

```
qbcompiler.calibration.preprocess.CenterCrop.__call__ (
    self,
    np.ndarray img)
```

Definition at line 601 of file `preprocess.py`.

16.4.4 Member Data Documentation

16.4.4.1 size

```
int qbcompiler.calibration.preprocess.CenterCrop.size = size * 2
```

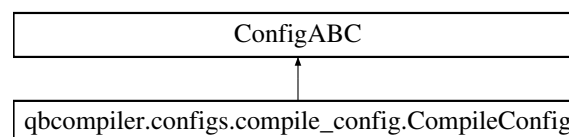
Definition at line 595 of file [preprocess.py](#).

The documentation for this class was generated from the following file:

- [preprocess.py](#)

16.5 qbcompiler.configs.compile_config.CompileConfig Class Reference

Inheritance diagram for qbcompiler.configs.compile_config.CompileConfig:



Public Member Functions

- `__init__` (self, str|None calib_data_path, bool use_random_calib, str save_path, str model_nickname, str|None save_msgpack_name, str|None model_path, str inference_scheme, bool save_sample, bool cpu_offload, int optimize_option, str sample_dtype, int buffer_mode, str version, dict input_shape_dict, [ResourceManagementConfig](#) resource_management, [QuantizationConfig](#) quantization, [AdvancedQuantizationConfig](#) advanced_quantization, [LlmConfig](#) llm_config, [InputProcessConfig](#) input_process, bool optional=optional)
Unified compilation configuration object for Mobilint MXQ compilation.
- `default_config` (cls, **kwargs)
Create default config with optional parameter overrides.
- `from_json` (cls, str json_path, **kwargs)
Load config from JSON file and merge with additional kwargs.
- `check_valid_str_configs` (self)
- `to_dict` (self)

Public Attributes

- `calib_data_path`
- `save_path` = save_path
- `model_nickname` = model_nickname
- `save_msgpack_name` = save_msgpack_name
- `model_path` = model_path

- `inference_scheme` = `inference_scheme`
- `save_sample` = `save_sample`
- `cpu_offload` = `cpu_offload`
- `optimize_option` = `optimize_option`
- `sample_dtype` = `sample_dtype`
- `buffer_mode` = `buffer_mode`
- `version` = `version`
- `input_shape_dict` = `input_shape_dict`
- `resource_management` = `resource_management`
- `quantization` = `quantization`
- `advanced_quantization` = `advanced_quantization`
- `llm_config` = `llm_config`
- `input_process` = `input_process`
- `calib_file` = `tempfile.NamedTemporaryFile(suffix=".txt")`

Static Public Attributes

- `bool optional` = `False`
- `dict py_to_cpp_name`
- `dict min_max_check` = `{"optimize_option": [0, None]}`
- `list inference_scheme_types` = `["single", "multi", "global", "global4", "global8"]`
- `list sample_dtypes` = `["float", "int8"]`

Protected Member Functions

- `_get_calibration_path` (`self`, `str calib_data_path`, `bool use_random_calib`, `dict input_shape_dict=None`)

16.5.1 Detailed Description

Definition at line 65 of file `compile_config.py`.

16.5.2 Constructor & Destructor Documentation

16.5.2.1 `__init__()`

```
qbcompiler.configs.compile_config.CompileConfig.__init__(
    self,
    str | None calib_data_path,
    bool use_random_calib,
    str save_path,
    str model_nickname,
    str | None save_msgpack_name,
    str | None model_path,
    str inference_scheme,
    bool save_sample,
```

```

bool cpu_offload,
int optimize_option,
str sample_dtype,
int buffer_mode,
str version,
dict input_shape_dict,
ResourceManagementConfig resource_management,
QuantizationConfig quantization,
AdvancedQuantizationConfig advanced_quantization,
LlmConfig llm_config,
InputProcessConfig input_process,
bool optional = optional)

```

Unified compilation configuration object for Mobilint MXQ compilation.

Parameters

calib_data_path	str None. Path to calibration dataset. Accepts either: - a directory containing pre-processed NumPy samples, or - a .txt/.json meta file listing NumPy sample file paths. If <code>use_random_calib</code> is True, this may be None. Otherwise, a valid path is expected for quantized compilation workflows.
use_random_calib	bool. When True, uses randomly generated calibration samples (typically for smoke testing and compilability validation). When False, <code>calib_data_path</code> is used as the calibration source.
save_path	str. Output MXQ filename/path to generate.
model_nickname	str. Model nickname used as a fallback identifier when intermediate results need to be saved. It is recommended to set this to the filename component of <code>save_path</code> .
save_msgpack_name	str None. filename/path to save an intermediate serialized representation (msgpack) of the high-level compiled model/graph. Primarily intended for debugging and is now rarely used.
model_path	str None. (deprecated) Outdated argument. Please refer to the model usage in the <code>mxq_compile</code> function.
inference_scheme	str. NPU inference scheme string. Must be one of: "single", "multi", "global", "global4", "global8" ("global" and "global8" behave identically).
save_sample	bool. When True, saves representative input/output tensors for debugging.
cpu_offload	bool. Enables CPU offload for unsupported operator groups/segments during NPU inference. This is a beta feature and is recommended only for special cases.
optimize_option	int. Compiler optimization selector.
sample_dtype	string. Data type for saved sample inference outputs: "float" or "int8". Primarily intended for debugging and is now rarely used.

Parameters

buffer_mode	int. Buffer serialization mode for saving .mblt file. Common conventions: • 0: naïve buffer serialization • 1: mmap-backed buffer serialization
version	str. qbcompiler version.
input_shape_dict	dict. Optional HWC input shape dictionary used for multi-shape compilation (for example {"input0": [[224, 224, 3], [256, 256, 3]]}).
resource_management	ResourceManagementConfig . Resource management configuration class.
quantization	QuantizationConfig . Quantization configuration class (e.g., calibration strategy, bit-widths, per-tensor/per-channel settings, quantization scheme/policies).
advanced_quantization	AdvancedQuantizationConfig . Advanced quantization configuration block for fine-grained or experimental controls beyond the basic quantization settings.
llm_config	LlmConfig . LLM-specific configuration class.
input_process	InputProcessConfig . Unified input processing configuration containing uint8 input handling, image channels, and preprocessing pipeline settings. optional bool. Indicates whether this config is optional. Use the default value defined as a class variable.

Definition at line 90 of file [compile_config.py](#).

16.5.3 Member Function Documentation

16.5.3.1 default_config()

```
qbcompiler.configs.compile_config.CompileConfig.default_config (
    cls,
    ** kwargs)
```

Create default config with optional parameter overrides.

Args: **kwargs: Parameters to override defaults

Definition at line 202 of file [compile_config.py](#).

16.5.3.2 from_json()

```
qbcompiler.configs.compile_config.CompileConfig.from_json (
    cls,
    str json_path,
    ** kwargs)
```

Load config from JSON file and merge with additional kwargs.

Args: json_path: Path to JSON configuration file **kwargs: Additional compile-time parameters (calib_↵_data_path, save_path, etc.)

Definition at line 238 of file [compile_config.py](#).

16.5.3.3 check_valid_str_configs()

```
qbcompiler.configs.compile_config.CompileConfig.check_valid_str_configs (
    self)
```

Definition at line 278 of file [compile_config.py](#).

16.5.3.4 to_dict()

```
qbcompiler.configs.compile_config.CompileConfig.to_dict (
    self)
```

Definition at line 294 of file [compile_config.py](#).

16.5.3.5 _get_calibration_path()

```
qbcompiler.configs.compile_config.CompileConfig._get_calibration_path (
    self,
    str calib_data_path,
    bool use_random_calib,
    dict input_shape_dict = None) [protected]
```

Definition at line 316 of file [compile_config.py](#).

16.5.4 Member Data Documentation

16.5.4.1 optional

```
bool qbcompiler.configs.compile_config.CompileConfig.optional = False [static]
```

Definition at line 66 of file [compile_config.py](#).

16.5.4.2 py_to_cpp_name

```
dict qbcompiler.configs.compile_config.CompileConfig.py_to_cpp_name [static]
```

Initial value:

```
= {
    "calib_data_path": "calibData",
    "save_path": "savePath",
    "model_nickname": "modelNickname",
    "save_msgpack_name": "saveMsgpackName",
    "model_path": "modelPath",
    "inference_scheme": "inferenceScheme",
    "save_sample": "saveSample",
    "cpu_offload": "cpuOffload",
    "optimize_option": "optimizeOption",
    "sample_dtype": "sampleDtype",
    "version": "version",
}
```

Definition at line 67 of file [compile_config.py](#).

16.5.4.3 min_max_check

```
dict qbcompiler.configs.compile_config.CompileConfig.min_max_check = {"optimize_option": [0, None]}  
[static]
```

Definition at line 80 of file [compile_config.py](#).

16.5.4.4 inference_scheme_types

```
list qbcompiler.configs.compile_config.CompileConfig.inference_scheme_types = ["single", "multi",  
"global", "global4", "global8"] [static]
```

Definition at line 81 of file [compile_config.py](#).

16.5.4.5 sample_dtypes

```
list qbcompiler.configs.compile_config.CompileConfig.sample_dtypes = ["float", "int8"] [static]
```

Definition at line 82 of file [compile_config.py](#).

16.5.4.6 calib_data_path

```
qbcompiler.configs.compile_config.CompileConfig.calib_data_path
```

Initial value:

```
= self._get_calibration_path(  
    calib_data_path, use_random_calib, input_shape_dict  
)
```

Definition at line 164 of file [compile_config.py](#).

16.5.4.7 save_path

```
qbcompiler.configs.compile_config.CompileConfig.save_path = save_path
```

Definition at line 167 of file [compile_config.py](#).

16.5.4.8 model_nickname

```
qbcompiler.configs.compile_config.CompileConfig.model_nickname = model_nickname
```

Definition at line 168 of file [compile_config.py](#).

16.5.4.9 save_msgpack_name

```
qbcompiler.configs.compile_config.CompileConfig.save_msgpack_name = save_msgpack_name
```

Definition at line 169 of file [compile_config.py](#).

16.5.4.10 model_path

```
qbcompiler.configs.compile_config.CompileConfig.model_path = model_path
```

Definition at line 170 of file [compile_config.py](#).

16.5.4.11 inference_scheme

```
qbcompiler.configs.compile_config.CompileConfig.inference_scheme = inference_scheme
```

Definition at line 171 of file [compile_config.py](#).

16.5.4.12 save_sample

```
qbcompiler.configs.compile_config.CompileConfig.save_sample = save_sample
```

Definition at line 172 of file [compile_config.py](#).

16.5.4.13 cpu_offload

```
qbcompiler.configs.compile_config.CompileConfig.cpu_offload = cpu_offload
```

Definition at line 173 of file [compile_config.py](#).

16.5.4.14 optimize_option

```
qbcompiler.configs.compile_config.CompileConfig.optimize_option = optimize_option
```

Definition at line 174 of file [compile_config.py](#).

16.5.4.15 sample_dtype

```
qbcompiler.configs.compile_config.CompileConfig.sample_dtype = sample_dtype
```

Definition at line 175 of file [compile_config.py](#).

16.5.4.16 buffer_mode

```
qbcompiler.configs.compile_config.CompileConfig.buffer_mode = buffer_mode
```

Definition at line 176 of file [compile_config.py](#).

16.5.4.17 version

```
qbcompiler.configs.compile_config.CompileConfig.version = version
```

Definition at line 177 of file [compile_config.py](#).

16.5.4.18 input_shape_dict

```
qbcompiler.configs.compile_config.CompileConfig.input_shape_dict = input_shape_dict
```

Definition at line 178 of file [compile_config.py](#).

16.5.4.19 resource_management

```
qbcompiler.configs.compile_config.CompileConfig.resource_management = resource_management
```

Definition at line 181 of file [compile_config.py](#).

16.5.4.20 quantization

```
qbcompiler.configs.compile_config.CompileConfig.quantization = quantization
```

Definition at line 182 of file [compile_config.py](#).

16.5.4.21 advanced_quantization

```
qbcompiler.configs.compile_config.CompileConfig.advanced_quantization = advanced_quantization
```

Definition at line 183 of file [compile_config.py](#).

16.5.4.22 llm_config

```
qbcompiler.configs.compile_config.CompileConfig.llm_config = llm_config
```

Definition at line 184 of file [compile_config.py](#).

16.5.4.23 input_process

qbcompiler.configs.compile_config.CompileConfig.input_process = input_process

Definition at line 185 of file [compile_config.py](#).

16.5.4.24 calib_file

qbcompiler.configs.compile_config.CompileConfig.calib_file = tempfile.NamedTemporaryFile(suffix=".txt")

Definition at line 332 of file [compile_config.py](#).

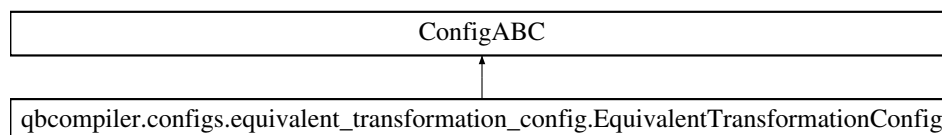
The documentation for this class was generated from the following file:

- [compile_config.py](#)

16.6 qbcompiler.configs.equivalent_transformation_config.↵ EquivalentTransformationConfig Class Reference

Configuration for equivalent transformation techniques.

Inheritance diagram for qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig:↵



Public Member Functions

- [__init__](#) (self, int seed, bool apply_hadamard_rotation_matrix, bool norm_conv_apply, bool norm_conv_learn, float norm_conv_smoothing_factor, float norm_conv_min_gamma, float norm_conv_max_gamma, bool qk_apply, float qk_smoothing_factor, float qk_min_gamma, float qk_max_gamma, bool ud_apply, bool ud_learn, float ud_smoothing_factor, float ud_min_gamma, float ud_max_gamma, bool vo_apply, float vo_smoothing_factor, float vo_min_gamma, float vo_max_gamma, bool ff_multi_lut_apply, List[float] ff_multi_lut_breakpoints, bool spin_r1_apply, str spin_r1_matrix_path, bool head_out_ch_rotation_apply, str head_out_ch_rotation_matrix_path, bool spin_r2_apply, bool spin_r2_learn, str spin_r2_matrix_path, bool qk_rotation_apply, str qk_rotation_matrix_path, bool flatten_quant_apply, bool flatten_quant_learn, float flatten_quant_apply_threshold, float flatten_quant_max_overhead, bool optimize_ffn_apply, int optimize_ffn_ch_per_ffn, bool optional=optional)
Initialize the [EquivalentTransformationConfig](#).
- [check_valid](#) (self, except_list=["optional"])
Override to skip py_to_cpp_name check (uses nested structure in to_dict)
- [default_config](#) (cls)
- [from_dict](#) (cls, dict config_dict)
Create [EquivalentTransformationConfig](#) from dictionary (JSON structure)
- [to_dict](#) (self)

Public Attributes

- `seed` = `seed`
- `apply_hadamard_rotation_matrix` = `apply_hadamard_rotation_matrix`
- `norm_conv_apply` = `norm_conv_apply`
- `norm_conv_learn` = `norm_conv_learn`
- `norm_conv_smoothing_factor` = `norm_conv_smoothing_factor`
- `norm_conv_min_gamma` = `norm_conv_min_gamma`
- `norm_conv_max_gamma` = `norm_conv_max_gamma`
- `qk_apply` = `qk_apply`
- `qk_smoothing_factor` = `qk_smoothing_factor`
- `qk_min_gamma` = `qk_min_gamma`
- `qk_max_gamma` = `qk_max_gamma`
- `ud_apply` = `ud_apply`
- `ud_learn` = `ud_learn`
- `ud_smoothing_factor` = `ud_smoothing_factor`
- `ud_min_gamma` = `ud_min_gamma`
- `ud_max_gamma` = `ud_max_gamma`
- `vo_apply` = `vo_apply`
- `vo_smoothing_factor` = `vo_smoothing_factor`
- `vo_min_gamma` = `vo_min_gamma`
- `vo_max_gamma` = `vo_max_gamma`
- `ff_multi_lut_apply` = `ff_multi_lut_apply`
- `ff_multi_lut_breakpoints` = `ff_multi_lut_breakpoints`
- `spin_r1_apply` = `spin_r1_apply`
- `spin_r1_matrix_path` = `spin_r1_matrix_path`
- `head_out_ch_rotation_apply` = `head_out_ch_rotation_apply`
- `head_out_ch_rotation_matrix_path` = `head_out_ch_rotation_matrix_path`
- `spin_r2_apply` = `spin_r2_apply`
- `spin_r2_learn` = `spin_r2_learn`
- `spin_r2_matrix_path` = `spin_r2_matrix_path`
- `qk_rotation_apply` = `qk_rotation_apply`
- `qk_rotation_matrix_path` = `qk_rotation_matrix_path`
- `flatten_quant_apply` = `flatten_quant_apply`
- `flatten_quant_learn` = `flatten_quant_learn`
- `flatten_quant_apply_threshold` = `flatten_quant_apply_threshold`
- `flatten_quant_max_overhead` = `flatten_quant_max_overhead`
- `optimize_ffn_apply` = `optimize_ffn_apply`
- `optimize_ffn_ch_per_ffn` = `optimize_ffn_ch_per_ffn`

Static Public Attributes

- `bool optional` = `False`
- `dict py_to_cpp_name` = `{}`
- `dict DEFAULTS`
- `dict min_max_check`

16.6.1 Detailed Description

Configuration for equivalent transformation techniques.

This includes various transformation methods like NormConv, QK, UD, VO, FeedForwardMultiLUT, SpinR1, HeadOutChRotation, SpinR2, QKRotation, and FlattenQuant.

Definition at line 14 of file [equivalent_transformation_config.py](#).

The documentation for this class was generated from the following file:

- [equivalent_transformation_config.py](#)

16.7 qbcompiler.calibration.preprocess.GetImage Class Reference

Retrieve an image tensor from a file path or NumPy array using OpenCV.

Public Member Functions

- [__init__](#) (self, bool to_float32=False, str channel_order="bgr")
- [np.ndarray __call__](#) (self, Union[np.ndarray, str] img_input)
Args: **img_input (Union[np.ndarray, str]):** Input image path or numpy tensor.
- [get_image_data](#) (self, str img_input)

Public Attributes

- [to_float32](#) = to_float32
- [str channel_order](#) = channel_order.lower()
- [cvtColor](#) = cvtColor
- [COLOR_BGR2GRAY](#) = COLOR_BGR2GRAY
- [COLOR_BGRA2GRAY](#) = COLOR_BGRA2GRAY
- [COLOR_BGRA2BGR](#) = COLOR_BGRA2BGR
- [COLOR_BGR2RGB](#) = COLOR_BGR2RGB
- [COLOR_BGRA2RGB](#) = COLOR_BGRA2RGB
- [COLOR_BGRA2RGBA](#) = COLOR_BGRA2RGBA
- [COLOR_BGR2BGRA](#) = COLOR_BGR2BGRA
- [COLOR_BGR2RGBA](#) = COLOR_BGR2RGBA
- [int num_channel](#) = 1:

16.7.1 Detailed Description

Retrieve an image tensor from a file path or NumPy array using OpenCV.

Parameters

to_float32	bool. Cast the output image to float32 when True. Defaults to False.
channel_order	string. Channel order to enforce ("bgr", "rgb", "bw", "rgba", or "bgra"). Defaults to "bgr".

Definition at line 146 of file [preprocess.py](#).

16.7.2 Constructor & Destructor Documentation

16.7.2.1 __init__()

```
qbcompiler.calibration.preprocess.GetImage.__init__ (
    self,
    bool to_float32 = False,
    str channel_order = "bgr")
```

Definition at line 154 of file [preprocess.py](#).

16.7.3 Member Function Documentation

16.7.3.1 __call__()

```
np.ndarray qbcompiler.calibration.preprocess.GetImage.__call__ (
    self,
    Union[np.ndarray, str] img_input)
```

Args: img_input (Union[np.ndarray, str]): Input image path or numpy tensor.

Returns: keepdims: Contains image numpy tensor and image meta such as "ori_shape".

Definition at line 175 of file [preprocess.py](#).

16.7.3.2 get_image_data()

```
qbcompiler.calibration.preprocess.GetImage.get_image_data (
    self,
    str img_input)
```

Definition at line 252 of file [preprocess.py](#).

16.7.4 Member Data Documentation

16.7.4.1 to_float32

```
qbcompiler.calibration.preprocess.GetImage.to_float32 = to_float32
```

Definition at line 156 of file [preprocess.py](#).

16.7.4.2 channel_order

```
str qbcompiler.calibration.preprocess.GetImage.channel_order = channel_order.lower()
```

Definition at line 158 of file [preprocess.py](#).

16.7.4.3 cvtColor

```
qbcompiler.calibration.preprocess.GetImage.cvtColor = cvtColor
```

Definition at line 165 of file [preprocess.py](#).

16.7.4.4 COLOR_BGR2GRAY

```
qbcompiler.calibration.preprocess.GetImage.COLOR_BGR2GRAY = COLOR_BGR2GRAY
```

Definition at line 166 of file [preprocess.py](#).

16.7.4.5 COLOR_BGRA2GRAY

```
qbcompiler.calibration.preprocess.GetImage.COLOR_BGRA2GRAY = COLOR_BGRA2GRAY
```

Definition at line 167 of file [preprocess.py](#).

16.7.4.6 COLOR_BGRA2BGR

```
qbcompiler.calibration.preprocess.GetImage.COLOR_BGRA2BGR = COLOR_BGRA2BGR
```

Definition at line 168 of file [preprocess.py](#).

16.7.4.7 COLOR_BGR2RGB

```
qbcompiler.calibration.preprocess.GetImage.COLOR_BGR2RGB = COLOR_BGR2RGB
```

Definition at line 169 of file [preprocess.py](#).

16.7.4.8 COLOR_BGRA2RGB

```
qbcompiler.calibration.preprocess.GetImage.COLOR_BGRA2RGB = COLOR_BGRA2RGB
```

Definition at line 170 of file [preprocess.py](#).

16.7.4.9 COLOR_BGRA2RGBA

`qbcompiler.calibration.preprocess.GetImage.COLOR_BGRA2RGBA = COLOR_BGRA2RGBA`

Definition at line 171 of file [preprocess.py](#).

16.7.4.10 COLOR_BGR2BGRA

`qbcompiler.calibration.preprocess.GetImage.COLOR_BGR2BGRA = COLOR_BGR2BGRA`

Definition at line 172 of file [preprocess.py](#).

16.7.4.11 COLOR_BGR2RGBA

`qbcompiler.calibration.preprocess.GetImage.COLOR_BGR2RGBA = COLOR_BGR2RGBA`

Definition at line 173 of file [preprocess.py](#).

16.7.4.12 num_channel

`int qbcompiler.calibration.preprocess.GetImage.num_channel = 1:`

Definition at line 186 of file [preprocess.py](#).

The documentation for this class was generated from the following file:

- [preprocess.py](#)

16.8 qbcompiler.calibration.preprocess.GetNpy Class Reference

Load NumPy data from an in-memory array or a .npy file.

Public Member Functions

- [__init__](#) (self)
- [np.ndarray __call__](#) (self, Union[[np.ndarray](#), str] npy_input)
Args: `npy_input` (Union[[np.ndarray](#), str]): Input .npy path or numpy data.

16.8.1 Detailed Description

Load NumPy data from an in-memory array or a .npy file.

Definition at line 36 of file [preprocess.py](#).

16.8.2 Constructor & Destructor Documentation

16.8.2.1 `__init__()`

```
qbcompiler.calibration.preprocess.GetNpy.__init__ (
    self)
```

Definition at line 41 of file [preprocess.py](#).

16.8.3 Member Function Documentation

16.8.3.1 `__call__()`

```
np.ndarray qbcompiler.calibration.preprocess.GetNpy.__call__ (
    self,
    Union[np.ndarray, str] npy_input)
```

Args: npy_input (Union[np.ndarray, str]): Input .npy path or numpy data.

Definition at line 44 of file [preprocess.py](#).

The documentation for this class was generated from the following file:

- [preprocess.py](#)

16.9 GetText Class Reference

Extract text strings from raw inputs.

16.9.1 Detailed Description

Extract text strings from raw inputs.

Parameters

text_key	Optional key to pull from dictionary inputs.
strip	Remove leading and trailing whitespace when true.

Definition at line 277 of file [preprocess.py](#).

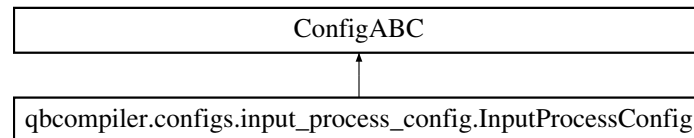
The documentation for this class was generated from the following file:

- [preprocess.py](#)

16.10 qbcompiler.configs.input_process_config.InputProcessConfig Class Reference

Unified configuration for input processing.

Inheritance diagram for qbcompiler.configs.input_process_config.InputProcessConfig:



Public Member Functions

- `__init__` (self, Optional[[UInt8InputConfig](#)] uint8_input, int image_channels, Optional[[PreprocessingConfig](#)] preprocessing, bool optional=optional)
Initialize the [InputProcessConfig](#).
- `default_config` (cls)
- `from_dict` (cls, dict config_dict)
Create [InputProcessConfig](#) from dictionary (JSON structure)
- `to_dict` (self)

Public Attributes

- tuple [uint8_input](#)
- [image_channels](#) = image_channels
- tuple [preprocessing](#)

Static Public Attributes

- bool [optional](#) = True
- dict [DEFAULTS](#)
- dict [py_to_cpp_name](#)
- dict [min_max_check](#) = {}

16.10.1 Detailed Description

Unified configuration for input processing.

Combines uint8 input handling, image channel configuration, and preprocessing pipeline settings into a single configuration object.

Definition at line 171 of file [input_process_config.py](#).

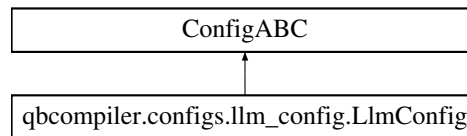
The documentation for this class was generated from the following file:

- [input_process_config.py](#)

16.11 qbcompiler.configs.llm_config.LlmConfig Class Reference

Configuration for Large Language Model (LLM) compilation.

Inheritance diagram for qbcompiler.configs.llm_config.LlmConfig:



Public Member Functions

- `__init__` (self, bool llm_config_apply, int max_data_length, int max_sequence_length, int max_cache_length, int max_core_data_length, int random_seq_length, bool use_full_seq_length, bool use_global_core, int batch_size, List[int] npu_core_ids, bool dynamic_rope, bool optional=optional)
Initialize the **LlmConfig**:
- `default_config` (cls)
- `from_kwargs` (cls, **kwargs)
- `from_dict` (cls, dict config_dict)
Create **LlmConfig** from dictionary (JSON structure)
- `to_dict` (self)

Public Attributes

- `llm_config_apply` = llm_config_apply
- `max_data_length` = max_data_length
- `max_sequence_length` = max_sequence_length
- `max_cache_length` = max_cache_length
- `max_core_data_length` = max_core_data_length
- `random_seq_length` = random_seq_length
- `use_full_seq_length` = use_full_seq_length
- `use_global_core` = use_global_core
- `batch_size` = batch_size
- `npu_core_ids` = npu_core_ids
- `dynamic_rope` = dynamic_rope

Static Public Attributes

- bool `optional` = True
- dict `DEFAULTS`
- dict `py_to_cpp_name`
- dict `min_max_check`

16.11.1 Detailed Description

Configuration for Large Language Model (LLM) compilation.

Definition at line 12 of file `llm_config.py`.

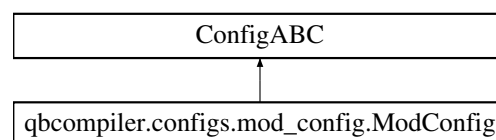
The documentation for this class was generated from the following file:

- `llm_config.py`

16.12 qbcompiler.configs.mod_config.ModConfig Class Reference

Configuration for Minimum Output Difference algorithm.

Inheritance diagram for `qbcompiler.configs.mod_config.ModConfig`:



Public Member Functions

- `__init__` (self, bool apply, int epochs, int warmup_epochs, numbers.Number lr_act_scale, numbers.Number lr_zeropoint, numbers.Number lr_weight_scale, numbers.Number lr_weight, numbers.Number lr_bias, numbers.Number lr_min_ratio, int batch_size, numbers.Number q_drop, str post_processing, numbers.Number box_conf_thres, numbers.Number box_iou_thres, str loss_type, bool use_outputs, numbers.Number kl_temperature, numbers.Number recon_prob, numbers.Number recon_coeff, numbers.Number lambda_0, numbers.Number lambda_1, numbers.Number lambda_2, List anchors, bool use_xyxy, str save_dir, int seed, List[str] apply_layers, List[str] exclude_layers, str custom_loss_jit_path, str mod_after_layer_name, bool optional=optional)
Initialize the **ModConfig**.
- `default_config` (cls)
- `from_dict` (cls, dict config_dict)
Create **ModConfig** from dictionary (JSON structure)
- `to_dict` (self)

Public Attributes

- `apply` = apply
- `epochs` = epochs
- `warmup_epochs` = warmup_epochs
- `lr_act_scale` = lr_act_scale
- `lr_zeropoint` = lr_zeropoint
- `lr_weight_scale` = lr_weight_scale

- `lr_weight` = `lr_weight`
- `lr_bias` = `lr_bias`
- `lr_min_ratio` = `lr_min_ratio`
- `batch_size` = `batch_size`
- `q_drop` = `q_drop`
- `post_processing` = `post_processing`
- `box_conf_thres` = `box_conf_thres`
- `box_iou_thres` = `box_iou_thres`
- `loss_type` = `loss_type`
- `use_outputs` = `use_outputs`
- `kl_temperature` = `kl_temperature`
- `recon_prob` = `recon_prob`
- `recon_coeff` = `recon_coeff`
- `lambda_0` = `lambda_0`
- `lambda_1` = `lambda_1`
- `lambda_2` = `lambda_2`
- `anchors` = `anchors`
- `use_xyxy` = `use_xyxy`
- `save_dir` = `save_dir`
- `seed` = `seed`
- `apply_layers` = `apply_layers`
- `exclude_layers` = `exclude_layers`
- `custom_loss_jit_path` = `custom_loss_jit_path`
- `mod_after_layer_name` = `mod_after_layer_name`

Static Public Attributes

- bool `optional` = `True`
- dict `DEFAULTS`
- dict `py_to_cpp_name`
- dict `min_max_check`

16.12.1 Detailed Description

Configuration for Minimum Output Difference algorithm.

Defines parameters controlling whether and how Mod is applied during quantization, including layer-level inclusion/exclusion lists.

Definition at line 14 of file `mod_config.py`.

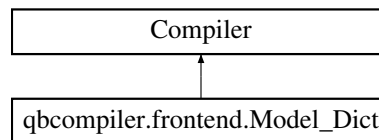
The documentation for this class was generated from the following file:

- `mod_config.py`

16.13 qbcompiler.frontend.Model_Dict Class Reference

Wrapper around the Mobilint compiler to support compilation and inference workflows.

Inheritance diagram for qbcompiler.frontend.Model_Dict:



Public Member Functions

- `__init__` (self, model, backend="onnx", device="cpu", feed_dict=None, dynamic_axes=None, in_dformats=None, yolo_decode_include=False, use_custom_mask=False, exclude_first_subgraph=False, **kwargs)
Initialize the Mobilint compiler wrapper.
- `compile` (self, str calib_data_path=None, str save_path=None, int save_subgraph_type=0, output_subgraph_path="", str model_nickname=None, save_sample=False, use_random_calib=False, inference_scheme="single", cpu_offload=False, optimize_option=1, str sample_dtype="float", buffer_mode=1, compile_config=None, **kwargs)
Compile the wrapped model into an MXQ artifact.

16.13.1 Detailed Description

Wrapper around the Mobilint compiler to support compilation and inference workflows.

Capabilities include:

- Compilation of models into MXQ artifacts runnable on Mobilint NPUs.
- Inference using the full-precision high-level compiled model on CPU or GPU.
- Inference with the quantized model on CPU or GPU.

Definition at line 23 of file [frontend.py](#).

16.13.2 Constructor & Destructor Documentation

16.13.2.1 __init__()

```

qbcompiler.frontend.Model_Dict.__init__ (
    self,
    model,
    backend = "onnx",
    device = "cpu",
    feed_dict = None,

```

```
dynamic_axes = None,  
in_dformats = None,  
yolo_decode_include = False,  
use_custom_mask = False,  
exclude_first_subgraph = False,  
** kwargs)
```

Initialize the Mobilint compiler wrapper.

Parameters

model	string or model instance. Model path or in-memory model to compile.
backend	string. Framework identifier for the model (for example "onnx"). Defaults to "onnx".
device	string. Target device for inference ("cpu" or "gpu"). Defaults to "cpu".
feed_dict	dict. Example inputs for shape resolution.
dynamic_axes	dict. Marks model axes as dynamic.
in_dformats	dict. Describes input data formats.
yolo_decode_include	bool. Runs YOLO decode on NPU when True.
use_custom_mask	bool. For non-LLM models that include a mask, use a custom mask provided as a separate input instead of the causal mask.
exclude_first_subgraph	bool. Applies only when CPU offloading is enabled: exclude the first subgraph from the final graph if it is unsupported.
kwargs	dict. Additional compiler arguments.

Definition at line 33 of file [frontend.py](#).

16.13.3 Member Function Documentation

16.13.3.1 compile()

```
qbcompiler.frontend.Model_Dict.compile (
    self,
    str  calib_data_path = None,
    str  save_path = None,
    int  save_subgraph_type = 0,
    str  output_subgraph_path = "",
    str  model_nickname = None,
    bool save_sample = False,
    bool use_random_calib = False,
    str  inference_scheme = "single",
    bool cpu_offload = False,
    int  optimize_option = 1,
    str  sample_dtype = "float",
    int  buffer_mode = 1,
    dict compile_config = None,
    ** kwargs)
```

Compile the wrapped model into an MXQ artifact.

Parameters

calib_data_path	string. Calibration dataset directory or meta file (.txt/.json) listing NumPy samples.
save_path	string. Output MXQ filename.
save_subgraph_type	int. Controls optional MBLT exports: 0 disables; 1 saves graph structure; 2 saves structure and weights; 3 splits structure into multiple subgraphs; 4 splits structure and weights.

Parameters

output_subgraph_path	string. Destination for exported .mblt when save_subgraph_type is 1-4.
model_nickname	string. Friendly name stored in qbcompiler for faster recompile.
save_sample	bool. Persists inputs/outputs for debugging in ./sampleInOut.
use_random_calib	bool. Uses randomly generated calibration samples when True.
inference_scheme	string. NPU inference configuration ("single", "multi", "global", etc.).
cpu_offload	bool. Enables CPU offload for unsupported groups.
optimize_option	int. Compiler optimization selector.
sample_dtype	string. Data type used for saved sample outputs.
buffer_mode	int. Buffer serialization mode (0 naive, 1 mmap).
compile_config	CompileConfig. Compile configuration object.
kwargs	dict. Additional arguments forwarded to the compiler.

Definition at line 73 of file [frontend.py](#).

The documentation for this class was generated from the following file:

- [frontend.py](#)

16.14 qbcompiler.calibration.preprocess.Normalize Class Reference

Apply mean/std normalization to images.

Public Member Functions

- [__init__](#) (self, Union[List[float], np.ndarray] mean, Union[List[float], np.ndarray] std, bool to_float_[↗](#) div255=False)
- [__call__](#) (self, np.ndarray img)

Public Attributes

- [mean](#) = np.float32(mean.reshape(1, -1))
- [std](#) = np.float32(std.reshape(1, -1))
- [to_float_div255](#) = to_float_div255

16.14.1 Detailed Description

Apply mean/std normalization to images.

Parameters

mean	list[float] or np.ndarray. Normalization mean.
std	list[float] or np.ndarray. Normalization standard deviation.
to_float_div255	bool. Divide by 255 before normalization when True. Defaults to False.

Definition at line [482](#) of file [preprocess.py](#).

16.14.2 Constructor & Destructor Documentation

16.14.2.1 __init__()

```
qbcompiler.calibration.preprocess.Normalize.__init__ (
    self,
    Union[List[float], np.ndarray] mean,
    Union[List[float], np.ndarray] std,
    bool to_float_div255 = False)
```

Definition at line [491](#) of file [preprocess.py](#).

16.14.3 Member Function Documentation

16.14.3.1 __call__()

```
qbcompiler.calibration.preprocess.Normalize.__call__ (
    self,
    np.ndarray img)
```

Definition at line [514](#) of file [preprocess.py](#).

16.14.4 Member Data Documentation

16.14.4.1 mean

```
qbcompiler.calibration.preprocess.Normalize.mean = np.float32(mean.reshape(1, -1))
```

Definition at line [509](#) of file [preprocess.py](#).

16.14.4.2 std

```
qbcompiler.calibration.preprocess.Normalize.std = np.float32(std.reshape(1, -1))
```

Definition at line [511](#) of file [preprocess.py](#).

16.14.4.3 to_float_div255

qbcompiler.calibration.preprocess.Normalize.to_float_div255 = to_float_div255

Definition at line 512 of file [preprocess.py](#).

The documentation for this class was generated from the following file:

- [preprocess.py](#)

16.15 qbcompiler.calibration.preprocess.NpyNormalize Class Reference

Public Member Functions

- [__init__](#) (self, Union[List[float], np.ndarray] mean, Union[List[float], np.ndarray] std, bool to_float_div255=False)
- [__call__](#) (self, np.ndarray img)

Public Attributes

- [mean](#) = np.array(mean) if isinstance(mean, (list, tuple)) else mean
- [std](#) = np.array(std) if isinstance(std, (list, tuple)) else std
- [to_float_div255](#) = to_float_div255

16.15.1 Detailed Description

Definition at line 62 of file [preprocess.py](#).

16.15.2 Constructor & Destructor Documentation

16.15.2.1 __init__()

```
qbcompiler.calibration.preprocess.NpyNormalize.__init__ (
    self,
    Union[List[float], np.ndarray] mean,
    Union[List[float], np.ndarray] std,
    bool to_float_div255 = False)
```

Definition at line 63 of file [preprocess.py](#).

16.15.3 Member Function Documentation

16.15.3.1 `__call__()`

```
qbcompiler.calibration.preprocess.NpyNormalize.__call__ (
    self,
    np.ndarray img)
```

Definition at line 77 of file [preprocess.py](#).

16.15.4 Member Data Documentation

16.15.4.1 `mean`

```
qbcompiler.calibration.preprocess.NpyNormalize.mean = np.array (mean) if isinstance (mean, (list, tuple)) else mean
```

Definition at line 73 of file [preprocess.py](#).

16.15.4.2 `std`

```
qbcompiler.calibration.preprocess.NpyNormalize.std = np.array (std) if isinstance (std, (list, tuple)) else std
```

Definition at line 74 of file [preprocess.py](#).

16.15.4.3 `to_float_div255`

```
qbcompiler.calibration.preprocess.NpyNormalize.to_float_div255 = to_float_div255
```

Definition at line 75 of file [preprocess.py](#).

The documentation for this class was generated from the following file:

- [preprocess.py](#)

16.16 qbcompiler.calibration.preprocess.NpyPad Class Reference

[Pad](#) NumPy arrays to a desired shape.

Public Member Functions

- [__init__](#) (self, List[int] shape, str padding_type, Optional[float] pad_val=0)
- [__call__](#) (self, np.ndarray data)
- [slice_padded_data](#) (self, data)

Public Attributes

- `shape` = `np.array(shape)`
- `padding_type` = `padding_type`
- `pad_val` = `pad_val`

16.16.1 Detailed Description

`Pad` NumPy arrays to a desired shape.

Parameters

shape	list[int]. Expected output shape after padding.
padding_type	string. Padding mode, either "constant" or "reflect".
pad_val	float. Fill value when padding_mode is 'constant'. Defaults to 0.

Definition at line 90 of file `preprocess.py`.

16.16.2 Constructor & Destructor Documentation

16.16.2.1 `__init__()`

```
qbcompiler.calibration.preprocess.NpyPad.__init__ (
    self,
    List[int] shape,
    str padding_type,
    Optional[float] pad_val = 0)
```

Definition at line 99 of file `preprocess.py`.

16.16.3 Member Function Documentation

16.16.3.1 `__call__()`

```
qbcompiler.calibration.preprocess.NpyPad.__call__ (
    self,
    np.ndarray data)
```

Definition at line 110 of file `preprocess.py`.

16.16.3.2 `slice_padded_data()`

```
qbcompiler.calibration.preprocess.NpyPad.slice_padded_data (
    self,
    data)
```

Definition at line 135 of file `preprocess.py`.

16.16.4 Member Data Documentation

16.16.4.1 shape

`qbcompiler.calibration.preprocess.NpyPad.shape = np.array(shape)`

Definition at line 106 of file [preprocess.py](#).

16.16.4.2 padding_type

`str qbcompiler.calibration.preprocess.NpyPad.padding_type = padding_type`

Definition at line 107 of file [preprocess.py](#).

16.16.4.3 pad_val

`qbcompiler.calibration.preprocess.NpyPad.pad_val = pad_val`

Definition at line 108 of file [preprocess.py](#).

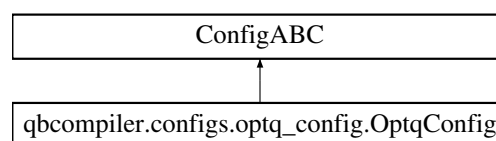
The documentation for this class was generated from the following file:

- [preprocess.py](#)

16.17 qbcompiler.configs.optq_config.OptqConfig Class Reference

Configuration for OPTQ algorithm.

Inheritance diagram for `qbcompiler.configs.optq_config.OptqConfig`:



Public Member Functions

- `__init__` (self, bool apply, bool act_order, int block_size, numbers.Number perc_damp, List[str] apply_layers, List[str] exclude_layers, bool optional=optional)
Initialize the **OptqConfig**.
- `default_config` (cls)
- `from_dict` (cls, dict config_dict)
Create **OptqConfig** from dictionary (JSON structure)
- `to_dict` (self)

Public Attributes

- `apply` = `apply`
- `act_order` = `act_order`
- `block_size` = `block_size`
- `perc_damp` = `perc_damp`
- `apply_layers` = `apply_layers`
- `exclude_layers` = `exclude_layers`

Static Public Attributes

- bool `optional` = `True`
- dict `DEFAULTS`
- dict `py_to_cpp_name`
- dict `min_max_check`

16.17.1 Detailed Description

Configuration for OPTQ algorithm.

Defines parameters controlling whether and how OPTQ is applied during quantization, including layer-level inclusion/exclusion lists.

Definition at line 14 of file `optq_config.py`.

The documentation for this class was generated from the following file:

- `optq_config.py`

16.18 qbcompiler.calibration.preprocess.Pad Class Reference

`Pad` images to a target shape or to meet a size divisor.

Public Member Functions

- `__init__` (self, shape=None, int size_divisor=None, float pad_val=0.0, bool right_bottom=False)
- `__call__` (self, np.ndarray img)

Public Attributes

- `pad_val` = `pad_val`
- `size_divisor` = `size_divisor`
- `shape` = `shape`
- `right_bottom` = `right_bottom`

16.18.1 Detailed Description

`Pad` images to a target shape or to meet a size divisor.

Parameters

shape	tuple[int]. Expected output shape (h, w). Defaults to None.
size_divisor	int. Ensures width and height are divisible by this value. Defaults to None.
pad_val	float. Fill value used during padding. Defaults to 0.0.
right_bottom	bool. When True, pad on the right and bottom only. Defaults to False.

Definition at line [437](#) of file [preprocess.py](#).

16.18.2 Constructor & Destructor Documentation

16.18.2.1 __init__()

```
qbcompiler.calibration.preprocess.Pad.__init__ (
    self,
    shape = None,
    int size_divisor = None,
    float pad_val = 0.0,
    bool right_bottom = False)
```

Definition at line [447](#) of file [preprocess.py](#).

16.18.3 Member Function Documentation

16.18.3.1 __call__()

```
qbcompiler.calibration.preprocess.Pad.__call__ (
    self,
    np.ndarray img)
```

Definition at line [460](#) of file [preprocess.py](#).

16.18.4 Member Data Documentation

16.18.4.1 pad_val

```
qbcompiler.calibration.preprocess.Pad.pad_val = pad_val
```

Definition at line [455](#) of file [preprocess.py](#).

16.18.4.2 size_divisor

```
qbcompiler.calibration.preprocess.Pad.size_divisor = size_divisor
```

Definition at line [456](#) of file [preprocess.py](#).

16.18.4.3 shape

```
qbcompiler.calibration.preprocess.Pad.shape = shape
```

Definition at line 457 of file [preprocess.py](#).

16.18.4.4 right_bottom

```
qbcompiler.calibration.preprocess.Pad.right_bottom = right_bottom
```

Definition at line 458 of file [preprocess.py](#).

The documentation for this class was generated from the following file:

- [preprocess.py](#)

16.19 qbcompiler.calibration.preprocess.preproc_builder Class Reference

Build a calibration preprocessing pipeline from a YAML or dict configuration.

Public Member Functions

- [__init__](#) (self, args, to_tensor=False)
- [__call__](#) (self, img)
- [__repr__](#) (self)
- [__str__](#) (self)
- [get_pre_list](#) (self, args)
- [check_yaml_pre](#) (self, args)

Public Attributes

- list [preprocess](#) = []
- [to_tensor](#) = to_tensor
- [args](#) = args
- [datatype](#) = args["Datatype"]
- [loadtype](#) = DTYPE_GETDATA_MAP[self.datatype]

16.19.1 Detailed Description

Build a calibration preprocessing pipeline from a YAML or dict configuration.

Supported preprocessing types:

- **Datatype**: Selects how input data is loaded. Supported value: Image.
- **GetImage**: Loads an image tensor via OpenCV or PIL. Must appear before other operators.
- **Pre-Order**: Explicit list of preprocessing stages to execute sequentially; entries must match the preprocessing definitions.
- **Pad**: Adds padding to the tensor.
- **Normalize**: Applies mean/std normalization.
- **ResizeTorch**: Resizes images using `torchvision.transforms.functional.resize`.
- **Resize**: Resizes images using `cv2.resize`.
- **CenterCrop**: Performs a center crop.
- **SetOrder**: Reorders axes; must be the final operator.

YAML structure

```
[Pre-processing Type]
  [Parameter]: [Argument]
  ...

# Example
Datatype: Image
GetImage:
  to_float32: false
  channel_order: RGB
Pre-Order: [ResizeTorch, CenterCrop, Normalize, SetOrder]
Pre-processing:
  ResizeTorch:
    size: [256, 256]
    interpolation: bilinear
  CenterCrop:
    size: [224, 224]
  Normalize:
    mean: [0.485, 0.456, 0.406]
    std: [0.229, 0.224, 0.225]
    to_float_div255: true
  SetOrder:
    shape: HWC
```

Operator parameters

- **GetImage**: `to_float32` (bool, default False); `channel_order` (string, default "bgr").
- **Pad**: `shape` (tuple[int]); `size_divisor` (int); `pad_val` (float, default 0); `right_bottom` (bool, default False).
- **Normalize**: `mean` (list/ndarray); `std` (list/ndarray); `to_float_div255` (bool, default False).
- **ResizeTorch**: `size` (list[int]); `interpolation` (nearest|bilinear|bicubic|box|hamming|lanczos).
- **Resize**: `img_scale` (float or tuple[int, int]); `keep_ratio` (bool, default False); `interpolation` (nearest|bilinear|bicubic|area|lanczos).
- **CenterCrop**: `size` (list[int]).
- **SetOrder**: `shape` (HWC|CHW|BHCW|BCHW, default HWC).

Parameters

args	dict. Parsed preprocessing configuration.
to_tensor	bool. When True, converts the resulting NumPy array to a Torch tensor.

Definition at line [719](#) of file [preprocess.py](#).

16.19.2 Constructor & Destructor Documentation

16.19.2.1 __init__()

```
qbcompiler.calibration.preprocess.preproc_builder.__init__ (
    self,
    args,
    to_tensor = False)
```

Definition at line [775](#) of file [preprocess.py](#).

16.19.3 Member Function Documentation

16.19.3.1 __call__()

```
qbcompiler.calibration.preprocess.preproc_builder.__call__ (
    self,
    img)
```

Definition at line [781](#) of file [preprocess.py](#).

16.19.3.2 __repr__()

```
qbcompiler.calibration.preprocess.preproc_builder.__repr__ (
    self)
```

Definition at line [788](#) of file [preprocess.py](#).

16.19.3.3 __str__()

```
qbcompiler.calibration.preprocess.preproc_builder.__str__ (
    self)
```

Definition at line [797](#) of file [preprocess.py](#).

16.19.3.4 get_pre_list()

```
qbcompiler.calibration.preprocess.preproc_builder.get_pre_list (  
    self,  
    args)
```

Definition at line 800 of file [preprocess.py](#).

16.19.3.5 check_yaml_pre()

```
qbcompiler.calibration.preprocess.preproc_builder.check_yaml_pre (  
    self,  
    args)
```

Definition at line 819 of file [preprocess.py](#).

16.19.4 Member Data Documentation

16.19.4.1 preprocess

```
list qbcompiler.calibration.preprocess.preproc_builder.preprocess = [ ]
```

Definition at line 776 of file [preprocess.py](#).

16.19.4.2 to_tensor

```
qbcompiler.calibration.preprocess.preproc_builder.to_tensor = to_tensor
```

Definition at line 777 of file [preprocess.py](#).

16.19.4.3 args

```
qbcompiler.calibration.preprocess.preproc_builder.args = args
```

Definition at line 778 of file [preprocess.py](#).

16.19.4.4 datatype

```
qbcompiler.calibration.preprocess.preproc_builder.datatype = args["Datatype"]
```

Definition at line 837 of file [preprocess.py](#).

16.19.4.5 loadtype

```
qbcompiler.calibration.preprocess.preproc_builder.loadtype = DTYPE_GETDATA_MAP[self.datatype]
```

Definition at line 838 of file [preprocess.py](#).

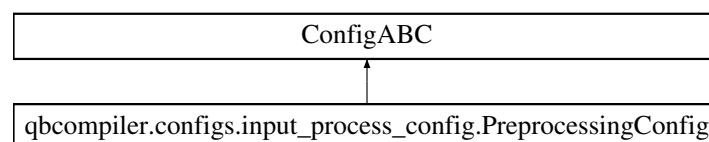
The documentation for this class was generated from the following file:

- [preprocess.py](#)

16.20 qbcompiler.configs.input_process_config.PreprocessingConfig Class Reference

Configuration for input preprocessing pipeline.

Inheritance diagram for qbcompiler.configs.input_process_config.PreprocessingConfig:



Public Member Functions

- `__init__` (self, bool apply, bool auto_convert_format, List[Dict[str, Any]] pipeline, Dict[str, Any] input_configs, bool optional=optional)
Initialize the **PreprocessingConfig**.
- `default_config` (cls)
- `from_dict` (cls, dict config_dict)
Create **PreprocessingConfig** from dictionary (JSON structure)
- `to_dict` (self)

Public Attributes

- `apply` = apply
- `auto_convert_format` = auto_convert_format
- `pipeline` = pipeline
- `input_configs` = input_configs

Static Public Attributes

- bool `optional` = True
- dict `DEFAULTS`
- dict `py_to_cpp_name`
- dict `min_max_check` = {}

16.20.1 Detailed Description

Configuration for input preprocessing pipeline.

Defines preprocessing operations to be applied to model inputs, including operations like resize, normalize, color conversion, etc.

Definition at line 82 of file [input_process_config.py](#).

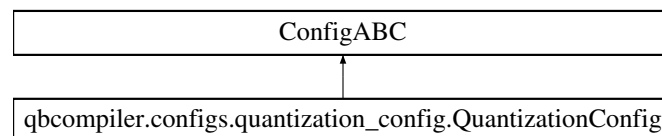
The documentation for this class was generated from the following file:

- [input_process_config.py](#)

16.21 qbcompiler.configs.quantization_config.QuantizationConfig Class Reference

Unified quantization configuration.

Inheritance diagram for qbcompiler.configs.quantization_config.QuantizationConfig:



Public Member Functions

- `__init__` (self, bool optional, Optional[[CalibrationConfig](#)] calibration=None, Optional[[BitConfig](#)] bit=None)
Initialize the [QuantizationConfig](#).
- `default_config` (cls)
- `from_kwargs` (cls, **kwargs)
- `from_dict` (cls, dict config_dict)
- `to_dict` (self)

Public Attributes

- `calibration` = calibration or [CalibrationConfig.default_config\(\)](#)
- `bit` = bit or [BitConfig.default_config\(\)](#)

Static Public Attributes

- bool `optional` = False
- dict `py_to_cpp_name`
- dict `min_max_check` = {}

16.21.1 Detailed Description

Unified quantization configuration.

Groups calibration and bit-precision configurations into a single configuration.

Definition at line 566 of file [quantization_config.py](#).

The documentation for this class was generated from the following file:

- [quantization_config.py](#)

16.22 qbcompiler.calibration.preprocess.Resize Class Reference

[Resize](#) images with optional aspect-ratio preservation.

Public Member Functions

- [__init__](#) (self, Union[float, Tuple[int, int]] img_scale, bool keep_ratio, str interpolation)
- [__call__](#) (self, np.ndarray img)

Public Attributes

- [img_scale](#) = img_scale
- [keep_ratio](#) = keep_ratio
- [interpolation](#) = interpolation

16.22.1 Detailed Description

[Resize](#) images with optional aspect-ratio preservation.

Parameters

img_scale	float or tuple[int, int]. Scaling factor or target size (h, w).
keep_ratio	bool. Preserve aspect ratio when True. Defaults to False.
interpolation	string. Interpolation mode ("nearest", "bilinear", "bicubic", "area", or "lanczos").

Definition at line 542 of file [preprocess.py](#).

16.22.2 Constructor & Destructor Documentation

16.22.2.1 `__init__()`

```
qbcompiler.calibration.preprocess.Resize.__init__ (  
    self,  
    Union[float, Tuple[int, int]] img_scale,  
    bool keep_ratio,  
    str interpolation)
```

Definition at line [551](#) of file [preprocess.py](#).

16.22.3 Member Function Documentation

16.22.3.1 `__call__()`

```
qbcompiler.calibration.preprocess.Resize.__call__ (  
    self,  
    np.ndarray img)
```

Definition at line [562](#) of file [preprocess.py](#).

16.22.4 Member Data Documentation

16.22.4.1 `img_scale`

```
qbcompiler.calibration.preprocess.Resize.img_scale = img_scale
```

Definition at line [558](#) of file [preprocess.py](#).

16.22.4.2 `keep_ratio`

```
qbcompiler.calibration.preprocess.Resize.keep_ratio = keep_ratio
```

Definition at line [559](#) of file [preprocess.py](#).

16.22.4.3 `interpolation`

```
qbcompiler.calibration.preprocess.Resize.interpolation = interpolation
```

Definition at line [560](#) of file [preprocess.py](#).

The documentation for this class was generated from the following file:

- `preprocess.py`

16.23 qbcompiler.calibration.preprocess.ResizeTorch Class Reference

Public Member Functions

- `__init__` (self, List[int] size, str interpolation)
- `__call__` (self, np.ndarray img)

Public Attributes

- `size` = size
- `interpolation` = interpolation

16.23.1 Detailed Description

Definition at line 524 of file [preprocess.py](#).

16.23.2 Constructor & Destructor Documentation

16.23.2.1 __init__()

```
qbcompiler.calibration.preprocess.ResizeTorch.__init__ (  
    self,  
    List[int] size,  
    str interpolation)
```

Definition at line 525 of file [preprocess.py](#).

16.23.3 Member Function Documentation

16.23.3.1 __call__()

```
qbcompiler.calibration.preprocess.ResizeTorch.__call__ (  
    self,  
    np.ndarray img)
```

Definition at line 530 of file [preprocess.py](#).

16.23.4 Member Data Documentation

16.23.4.1 size

```
qbcompiler.calibration.preprocess.ResizeTorch.size = size
```

Definition at line 527 of file [preprocess.py](#).

16.23.4.2 interpolation

`qbcompiler.calibration.preprocess.ResizeTorch.interpolation = interpolation`

Definition at line 528 of file [preprocess.py](#).

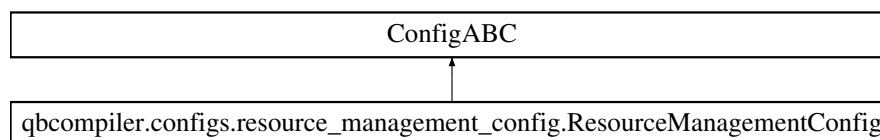
The documentation for this class was generated from the following file:

- [preprocess.py](#)

16.24 qbcompiler.configs.resource_management_config.ResourceManagementConfig Class Reference

Configuration for resource management during model compilation.

Inheritance diagram for `qbcompiler.configs.resource_management_config.ResourceManagementConfig`:



Public Member Functions

- `__init__` (self, str weight_dtype, bool use_gpu_only_for_calibration, int weight_memory_method, bool optional=optional)
Initialize the **ResourceManagementConfig**.
- `default_config` (cls)
- `from_kwargs` (cls, **kwargs)
- `from_dict` (cls, dict config_dict)
Create **ResourceManagementConfig** from dictionary (JSON structure)
- `to_dict` (self)

Public Attributes

- `weight_dtype` = weight_dtype
- `use_gpu_only_for_calibration` = use_gpu_only_for_calibration
- `weight_memory_method` = weight_memory_method

Static Public Attributes

- bool `optional` = False
- dict `DEFAULTS`
- dict `py_to_cpp_name`
- dict `min_max_check`
- list `weight_memory_methods`

16.24.1 Detailed Description

Configuration for resource management during model compilation.

Definition at line 11 of file [resource_management_config.py](#).

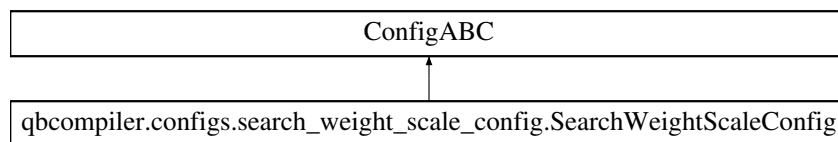
The documentation for this class was generated from the following file:

- [resource_management_config.py](#)

16.25 qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig Class Reference

Configuration for searching weight scale.

Inheritance diagram for qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig:



Public Member Functions

- `__init__` (self, bool apply, bool query, bool key, bool value, bool out, bool ffn, bool optional=optional)
Initialize the [SearchWeightScaleConfig](#).
- `default_config` (cls)
- `from_dict` (cls, dict config_dict)
Create [SearchWeightScaleConfig](#) from dictionary (JSON structure)
- `to_dict` (self)

Public Attributes

- `apply` = apply
- `query` = query
- `key` = key
- `value` = value
- `out` = out
- `ffn` = ffn

Static Public Attributes

- bool `optional` = False
- dict `DEFAULTS`

16.25.1 Detailed Description

Configuration for searching weight scale.

Definition at line 11 of file [search_weight_scale_config.py](#).

The documentation for this class was generated from the following file:

- [search_weight_scale_config.py](#)

16.26 qbcompiler.calibration.preprocess.SetOrder Class Reference

Public Member Functions

- [__init__](#) (self, str shape="HWC")
- [__call__](#) (self, np.ndarray img)

Public Attributes

- [shape](#) = shape

16.26.1 Detailed Description

Definition at line 703 of file [preprocess.py](#).

16.26.2 Constructor & Destructor Documentation

16.26.2.1 __init__()

```
qbcompiler.calibration.preprocess.SetOrder.__init__ (
    self,
    str shape = "HWC")
```

Definition at line 704 of file [preprocess.py](#).

16.26.3 Member Function Documentation

16.26.3.1 __call__()

```
qbcompiler.calibration.preprocess.SetOrder.__call__ (
    self,
    np.ndarray img)
```

Definition at line 709 of file [preprocess.py](#).

16.26.4 Member Data Documentation

16.26.4.1 shape

qbcompiler.calibration.preprocess.SetOrder.shape = shape

Definition at line 707 of file [preprocess.py](#).

The documentation for this class was generated from the following file:

- [preprocess.py](#)

16.27 TokenizeToEmbedding Class Reference

Convert tokenized text into embeddings using a Hugging Face model.

16.27.1 Detailed Description

Convert tokenized text into embeddings using a Hugging Face model.

Parameters

tokenizer_dir	Directory containing tokenizer resources.
model_dir	Optional directory for the model weights; defaults to tokenizer_dir.
device	Device string determining where inference runs.
trust_remote_code	Whether to load custom code from the model repo.
min_seqlen	Minimum token length required for a sample.
max_seqlen	Maximum token length; longer sequences are truncated.
pad_to_max	When true, pad sequences up to max_seqlen with zeros.
dtype	NumPy dtype used for exported embeddings.

Definition at line 321 of file [preprocess.py](#).

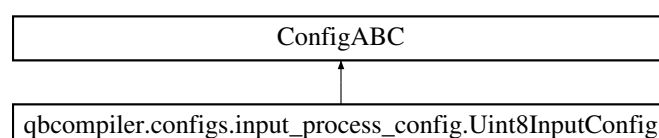
The documentation for this class was generated from the following file:

- [preprocess.py](#)

16.28 qbcompiler.configs.input_process_config.Uint8InputConfig Class Reference

Configuration for uint8 input handling.

Inheritance diagram for qbcompiler.configs.input_process_config.Uint8InputConfig:



Public Member Functions

- `__init__` (self, bool apply, List[str] inputs, bool optional=optional)
Initialize the `UInt8InputConfig`.
- `default_config` (cls)
- `from_dict` (cls, dict config_dict)
Create `UInt8InputConfig` from dictionary (JSON structure)
- `to_dict` (self)

Public Attributes

- `apply` = apply
- `inputs` = inputs

Static Public Attributes

- bool `optional` = True
- dict `DEFAULTS`
- dict `py_to_cpp_name`
- dict `min_max_check` = {}

16.28.1 Detailed Description

Configuration for uint8 input handling.

Defines whether inputs should be treated as uint8 and which specific inputs to apply this to.

Definition at line 13 of file `input_process_config.py`.

The documentation for this class was generated from the following file:

- `input_process_config.py`

16.29 YoloPre Class Reference

Apply YOLO-style resize, padding, and normalization to images.

16.29.1 Detailed Description

Apply YOLO-style resize, padding, and normalization to images.

Parameters

size	Target image size provided as [height, width].
-------------	--

Definition at line 641 of file `preprocess.py`.

The documentation for this class was generated from the following file:

- `preprocess.py`

Index

- `__call__`
 - `qbcompiler.calibration.preprocess.CenterCrop`, [140](#)
 - `qbcompiler.calibration.preprocess.GetImage`, [152](#)
 - `qbcompiler.calibration.preprocess.GetNpy`, [155](#)
 - `qbcompiler.calibration.preprocess.Normalize`, [164](#)
 - `qbcompiler.calibration.preprocess.NpyNormalize`, [166](#)
 - `qbcompiler.calibration.preprocess.NpyPad`, [167](#)
 - `qbcompiler.calibration.preprocess.Pad`, [170](#)
 - `qbcompiler.calibration.preprocess.preproc_builder`, [173](#)
 - `qbcompiler.calibration.preprocess.Resize`, [178](#)
 - `qbcompiler.calibration.preprocess.ResizeTorch`, [179](#)
 - `qbcompiler.calibration.preprocess.SetOrder`, [182](#)
- `__init__`
 - `AdvancedQuantization Configuration`, [93](#)
 - `EquivalentTransformation Configuration`, [98](#)
 - `Input Process Configuration`, [128–130](#)
 - `LLM Configuration`, [88](#)
 - `Mod Configuration`, [114](#)
 - `Optq Configuration`, [123](#)
 - `qbcompiler.calibration.preprocess.CenterCrop`, [140](#)
 - `qbcompiler.calibration.preprocess.GetImage`, [152](#)
 - `qbcompiler.calibration.preprocess.GetNpy`, [155](#)
 - `qbcompiler.calibration.preprocess.Normalize`, [164](#)
 - `qbcompiler.calibration.preprocess.NpyNormalize`, [165](#)
 - `qbcompiler.calibration.preprocess.NpyPad`, [167](#)
 - `qbcompiler.calibration.preprocess.Pad`, [170](#)
 - `qbcompiler.calibration.preprocess.preproc_builder`, [173](#)
 - `qbcompiler.calibration.preprocess.Resize`, [178](#)
 - `qbcompiler.calibration.preprocess.ResizeTorch`, [179](#)
 - `qbcompiler.calibration.preprocess.SetOrder`, [182](#)
 - `qbcompiler.configs.compile_config.CompileConfig`, [142](#)
 - `qbcompiler.frontend.Model_Dict`, [160](#)
 - `Quantization Configuration`, [66](#), [68](#), [70](#)
 - `Resource Management Configuration`, [83](#)
 - `SearchWeightScale Configuration`, [110](#)
- `__repr__`
 - `qbcompiler.calibration.preprocess.preproc_builder`, [173](#)
- `__str__`
 - `qbcompiler.calibration.preprocess.preproc_builder`, [173](#)
- `_get_calibration_path`
 - `qbcompiler.configs.compile_config.CompileConfig`, [145](#)
- `act_order`
 - `Optq Configuration`, [126](#)
- `act_scale_min`
 - `Quantization Configuration`, [77](#)
- `activation_16bits`
 - `Quantization Configuration`, [81](#)
- `advanced_quantization`
 - `qbcompiler.configs.compile_config.CompileConfig`, [148](#)
- `AdvancedQuantization Configuration`, [92](#)
- `__init__`, [93](#)
- `default_config`, [93](#)
- `equivalent_transformation`, [95](#)
- `from_dict`, [93](#)
- `get_advanced_quantization_config`, [94](#)
- `mod`, [94](#)
- `optional`, [94](#)
- `optq`, [94](#)
- `search_weight_scale`, [95](#)
- `to_dict`, [94](#)
- `anchors`
 - `Mod Configuration`, [121](#)
- `API Reference`, [53](#)
- `DTYPE_GETDATA_MAP`, [62](#)
- `get_yaml`, [59](#)
- `make_calib`, [60](#)
- `make_calib_llm`, [61](#)
- `make_calib_man`, [59](#)
- `mblt_compile`, [58](#)
- `mxq_compile`, [54](#)
- `preprocess_fnc`, [62](#)
- `run_mblt_model`, [59](#)
- `apply`
 - `Input Process Configuration`, [133](#), [134](#)
 - `Mod Configuration`, [118](#)
 - `Optq Configuration`, [125](#)
 - `SearchWeightScale Configuration`, [112](#)
- `apply_hadamard_rotation_matrix`
 - `EquivalentTransformation Configuration`, [103](#)
- `apply_layers`
 - `Mod Configuration`, [122](#)
 - `Optq Configuration`, [126](#)
- `args`
 - `qbcompiler.calibration.preprocess.preproc_builder`, [174](#)
- `auto_convert_format`
 - `Input Process Configuration`, [134](#)

- batch_size
 - LLM Configuration, [92](#)
 - Mod Configuration, [119](#)
- bit
 - Quantization Configuration, [82](#)
- block_size
 - Optq Configuration, [126](#)
- box_conf_thres
 - Mod Configuration, [120](#)
- box_iou_thres
 - Mod Configuration, [120](#)
- buffer_mode
 - qbcompiler.configs.compile_config.Compile-Config, [147](#)
- calib_data_path
 - qbcompiler.configs.compile_config.Compile-Config, [146](#)
- calib_file
 - qbcompiler.configs.compile_config.Compile-Config, [149](#)
- calibration
 - Quantization Configuration, [82](#)
- Changelog, [42](#)
- channel_order
 - qbcompiler.calibration.preprocess.GetImage, [152](#)
- check_valid
 - EquivalentTransformation Configuration, [101](#)
- check_valid_str_configs
 - qbcompiler.configs.compile_config.Compile-Config, [144](#)
- check_yaml_pre
 - qbcompiler.calibration.preprocess.pre-proc_builder, [174](#)
- COLOR_BGR2BGRA
 - qbcompiler.calibration.preprocess.GetImage, [154](#)
- COLOR_BGR2GRAY
 - qbcompiler.calibration.preprocess.GetImage, [153](#)
- COLOR_BGR2RGB
 - qbcompiler.calibration.preprocess.GetImage, [153](#)
- COLOR_BGR2RGBA
 - qbcompiler.calibration.preprocess.GetImage, [154](#)
- COLOR_BGRA2BGR
 - qbcompiler.calibration.preprocess.GetImage, [153](#)
- COLOR_BGRA2GRAY
 - qbcompiler.calibration.preprocess.GetImage, [153](#)
- COLOR_BGRA2RGB
 - qbcompiler.calibration.preprocess.GetImage, [153](#)
- COLOR_BGRA2RGBA
 - qbcompiler.calibration.preprocess.GetImage, [153](#)
- qbcompiler.calibration.preprocess.GetImage, [153](#)
- Compilation Configuration, [62](#)
 - get_compile_config, [63](#)
- compile
 - qbcompiler.frontend.Model_Dict, [162](#)
- CPU Offloading (Beta Version), [41](#)
- cpu_offload
 - qbcompiler.configs.compile_config.Compile-Config, [147](#)
- custom_loss_jit_path
 - Mod Configuration, [122](#)
- cvtColor
 - qbcompiler.calibration.preprocess.GetImage, [153](#)
- datatype
 - qbcompiler.calibration.preprocess.pre-proc_builder, [174](#)
- default_config
 - AdvancedQuantization Configuration, [93](#)
 - EquivalentTransformation Configuration, [101](#)
 - Input Process Configuration, [128](#), [130](#), [131](#)
 - LLM Configuration, [88](#)
 - Mod Configuration, [116](#)
 - Optq Configuration, [124](#)
 - qbcompiler.configs.compile_config.Compile-Config, [144](#)
 - Quantization Configuration, [68–70](#)
 - Resource Management Configuration, [84](#)
 - SearchWeightScale Configuration, [110](#)
- DEFAULTS
 - EquivalentTransformation Configuration, [102](#)
 - Input Process Configuration, [132–134](#)
 - LLM Configuration, [89](#)
 - Mod Configuration, [117](#)
 - Optq Configuration, [125](#)
 - Quantization Configuration, [72, 77](#)
 - Resource Management Configuration, [85](#)
 - SearchWeightScale Configuration, [111](#)
- DTYPE_GETDATA_MAP
 - API Reference, [62](#)
- dynamic_rope
 - LLM Configuration, [92](#)
- epochs
 - Mod Configuration, [118](#)
- equivalent_transformation
 - AdvancedQuantization Configuration, [95](#)
- EquivalentTransformation Configuration, [95](#)
 - __init__, [98](#)
 - apply_hadamard_rotation_matrix, [103](#)
 - check_valid, [101](#)
 - default_config, [101](#)
 - DEFAULTS, [102](#)
 - ff_multi_lut_apply, [106](#)
 - ff_multi_lut_breakpoints, [106](#)

- flatten_quant_apply, [108](#)
- flatten_quant_apply_threshold, [108](#)
- flatten_quant_learn, [108](#)
- flatten_quant_max_overhead, [108](#)
- from_dict, [101](#)
- get_equivalent_transformation_config, [102](#)
- head_out_ch_rotation_apply, [107](#)
- head_out_ch_rotation_matrix_path, [107](#)
- min_max_check, [102](#)
- norm_conv_apply, [103](#)
- norm_conv_learn, [103](#)
- norm_conv_max_gamma, [104](#)
- norm_conv_min_gamma, [104](#)
- norm_conv_smoothing_factor, [103](#)
- optimize_ffn_apply, [109](#)
- optimize_ffn_ch_per_ffn, [109](#)
- optional, [102](#)
- py_to_cpp_name, [102](#)
- qk_apply, [104](#)
- qk_max_gamma, [104](#)
- qk_min_gamma, [104](#)
- qk_rotation_apply, [108](#)
- qk_rotation_matrix_path, [108](#)
- qk_smoothing_factor, [104](#)
- seed, [103](#)
- spin_r1_apply, [106](#)
- spin_r1_matrix_path, [107](#)
- spin_r2_apply, [107](#)
- spin_r2_learn, [107](#)
- spin_r2_matrix_path, [107](#)
- to_dict, [102](#)
- ud_apply, [105](#)
- ud_learn, [105](#)
- ud_max_gamma, [105](#)
- ud_min_gamma, [105](#)
- ud_smoothing_factor, [105](#)
- vo_apply, [105](#)
- vo_max_gamma, [106](#)
- vo_min_gamma, [106](#)
- vo_smoothing_factor, [106](#)
- exclude_layers
 - Mod Configuration, [122](#)
 - Optq Configuration, [126](#)
- ff_multi_lut_apply
 - EquivalentTransformation Configuration, [106](#)
- ff_multi_lut_breakpoints
 - EquivalentTransformation Configuration, [106](#)
- ffn
 - SearchWeightScale Configuration, [112](#)
- ffn_act_bits
 - Quantization Configuration, [79](#)
- ffn_weight_bits
 - Quantization Configuration, [80](#)
- flatten_quant_apply
 - EquivalentTransformation Configuration, [108](#)
- flatten_quant_apply_threshold
 - EquivalentTransformation Configuration, [108](#)
- flatten_quant_learn
 - EquivalentTransformation Configuration, [108](#)
- flatten_quant_max_overhead
 - EquivalentTransformation Configuration, [108](#)
- from_dict
 - AdvancedQuantization Configuration, [93](#)
 - EquivalentTransformation Configuration, [101](#)
 - Input Process Configuration, [129–131](#)
 - LLM Configuration, [89](#)
 - Mod Configuration, [116](#)
 - Optq Configuration, [124](#)
 - Quantization Configuration, [68, 70, 71](#)
 - Resource Management Configuration, [84](#)
 - SearchWeightScale Configuration, [111](#)
- from_json
 - qbcompiler.configs.compile_config.Compile-Config, [144](#)
- from_kwargs
 - LLM Configuration, [88](#)
 - Quantization Configuration, [68, 70, 71](#)
 - Resource Management Configuration, [84](#)
- get_advanced_quantization_config
 - AdvancedQuantization Configuration, [94](#)
- get_bit_config
 - Quantization Configuration, [71](#)
- get_calibration_config
 - Quantization Configuration, [71](#)
- get_compile_config
 - Compilation Configuration, [63](#)
- get_equivalent_transformation_config
 - EquivalentTransformation Configuration, [102](#)
- get_image_data
 - qbcompiler.calibration.preprocess.GetImage, [152](#)
- get_input_process_config
 - Input Process Configuration, [131](#)
- get_llm_config
 - LLM Configuration, [89](#)
- get_mod_config
 - Mod Configuration, [116](#)
- get_optq_config
 - Optq Configuration, [124](#)
- get_pre_list
 - qbcompiler.calibration.preprocess.pre-proc_builder, [173](#)
- get_preprocessing_config
 - Input Process Configuration, [130](#)
- get_quantization_config
 - Quantization Configuration, [71](#)
- get_resource_management_config
 - Resource Management Configuration, [84](#)
- get_search_weight_scale_config
 - SearchWeightScale Configuration, [111](#)
- get_uint8_input_config
 - Input Process Configuration, [129](#)

- get_yaml
 - API Reference, 59
- GetText, 155
- head_act_bits
 - Quantization Configuration, 80
- head_out_ch_rotation_apply
 - EquivalentTransformation Configuration, 107
- head_out_ch_rotation_matrix_path
 - EquivalentTransformation Configuration, 107
- head_weight_bits
 - Quantization Configuration, 80
- hist_buffer_size
 - Quantization Configuration, 76
- hist_min_bin_width
 - Quantization Configuration, 76
- hist_num_bins
 - Quantization Configuration, 75
- hist_num_samples
 - Quantization Configuration, 76
- hist_num_search
 - Quantization Configuration, 76
- hist_percentile
 - Quantization Configuration, 75
- hist_search_percentile_max
 - Quantization Configuration, 76
- hist_search_percentile_min
 - Quantization Configuration, 76
- hist_search_type
 - Quantization Configuration, 76
- hist_use_gpu
 - Quantization Configuration, 75
- HuggingFace로 배포된 Llama 모델, 18
- image_channels
 - Input Process Configuration, 135
- img_scale
 - qbcompiler.calibration.preprocess.Resize, 178
- inference_scheme
 - qbcompiler.configs.compile_config.Compile-Config, 147
- inference_scheme_types
 - qbcompiler.configs.compile_config.Compile-Config, 146
- Input Process Configuration, 126
 - __init__, 128–130
 - apply, 133, 134
 - auto_convert_format, 134
 - default_config, 128, 130, 131
 - DEFAULTS, 132–134
 - from_dict, 129–131
 - get_input_process_config, 131
 - get_preprocessing_config, 130
 - get_uint8_input_config, 129
 - image_channels, 135
 - input_configs, 134
 - inputs, 133
 - min_max_check, 132, 133, 135
 - optional, 132–134
 - pipeline, 134
 - preprocessing, 135
 - py_to_cpp_name, 132–134
 - to_dict, 129–131
 - uint8_input, 135
- input_configs
 - Input Process Configuration, 134
- input_process
 - qbcompiler.configs.compile_config.Compile-Config, 148
- input_shape_dict
 - qbcompiler.configs.compile_config.Compile-Config, 148
- inputs
 - Input Process Configuration, 133
- interpolation
 - qbcompiler.calibration.preprocess.Resize, 178
 - qbcompiler.calibration.preprocess.Resize-Torch, 179
- keep_ratio
 - qbcompiler.calibration.preprocess.Resize, 178
- key
 - SearchWeightScale Configuration, 112
- key_act_bits
 - Quantization Configuration, 79
- key_weight_bits
 - Quantization Configuration, 80
- kl_temperature
 - Mod Configuration, 120
- lambda_0
 - Mod Configuration, 121
- lambda_1
 - Mod Configuration, 121
- lambda_2
 - Mod Configuration, 121
- layer_overrides
 - Quantization Configuration, 77
- LLM Configuration, 87
 - __init__, 88
 - batch_size, 92
 - default_config, 88
 - DEFAULTS, 89
 - dynamic_rope, 92
 - from_dict, 89
 - from_kwargs, 88
 - get_llm_config, 89
 - llm_config_apply, 90
 - max_cache_length, 91
 - max_core_data_length, 91
 - max_data_length, 91

- max_sequence_length, 91
- min_max_check, 90
- npu_core_ids, 92
- optional, 89
- py_to_cpp_name, 90
- random_seq_length, 91
- to_dict, 89
- use_full_seq_length, 91
- use_global_core, 91
- llm_config
 - qbcompiler.configs.compile_config.Compile-
Config, 148
- llm_config_apply
 - LLM Configuration, 90
- load_path
 - Quantization Configuration, 81
- loadtype
 - qbcompiler.calibration.preprocess.pre-
proc_builder, 174
- loss_type
 - Mod Configuration, 120
- lr_act_scale
 - Mod Configuration, 118
- lr_bias
 - Mod Configuration, 119
- lr_min_ratio
 - Mod Configuration, 119
- lr_weight
 - Mod Configuration, 119
- lr_weight_scale
 - Mod Configuration, 119
- lr_zeroint
 - Mod Configuration, 118
- make_calib
 - API Reference, 60
- make_calib_llm
 - API Reference, 61
- make_calib_man
 - API Reference, 59
- max_cache_length
 - LLM Configuration, 91
- max_core_data_length
 - LLM Configuration, 91
- max_data_length
 - LLM Configuration, 91
- max_each
 - Quantization Configuration, 75
- max_sequence_length
 - LLM Configuration, 91
- max_total
 - Quantization Configuration, 75
- mblt_compile
 - API Reference, 58
- mean
 - qbcompiler.calibration.preprocess.Normalize,
164
- qbcompiler.calibration.preprocess.NpyNor-
malize, 166
- method_list
 - Quantization Configuration, 72
- min_clip_ratio
 - Quantization Configuration, 77
- min_max_check
 - EquivalentTransformation Configuration, 102
 - Input Process Configuration, 132, 133, 135
 - LLM Configuration, 90
 - Mod Configuration, 117
 - Optq Configuration, 125
 - qbcompiler.configs.compile_config.Compile-
Config, 145
 - Quantization Configuration, 74, 78, 82
 - Resource Management Configuration, 85
- mixed_precision_apply
 - Quantization Configuration, 81
- mod
 - AdvancedQuantization Configuration, 94
- Mod Configuration, 113
 - __init__, 114
 - anchors, 121
 - apply, 118
 - apply_layers, 122
 - batch_size, 119
 - box_conf_thres, 120
 - box_iou_thres, 120
 - custom_loss_jit_path, 122
 - default_config, 116
 - DEFAULTS, 117
 - epochs, 118
 - exclude_layers, 122
 - from_dict, 116
 - get_mod_config, 116
 - kl_temperature, 120
 - lambda_0, 121
 - lambda_1, 121
 - lambda_2, 121
 - loss_type, 120
 - lr_act_scale, 118
 - lr_bias, 119
 - lr_min_ratio, 119
 - lr_weight, 119
 - lr_weight_scale, 119
 - lr_zeroint, 118
 - min_max_check, 117
 - mod_after_layer_name, 122
 - optional, 117
 - post_processing, 119
 - py_to_cpp_name, 117
 - q_drop, 119
 - recon_coeff, 120
 - recon_prob, 120
 - save_dir, 121
 - seed, 121
 - to_dict, 116

- use_outputs, 120
 - use_xyxy, 121
 - warmup_epochs, 118
- mod_after_layer_name
 - Mod Configuration, 122
- mode_list
 - Quantization Configuration, 72
- model_nickname
 - qbcompiler.configs.compile_config.Compile-
Config, 146
- model_path
 - qbcompiler.configs.compile_config.Compile-
Config, 147
- mxq_compile
 - API Reference, 54
- norm_conv_apply
 - EquivalentTransformation Configuration, 103
- norm_conv_learn
 - EquivalentTransformation Configuration, 103
- norm_conv_max_gamma
 - EquivalentTransformation Configuration, 104
- norm_conv_min_gamma
 - EquivalentTransformation Configuration, 104
- norm_conv_smoothing_factor
 - EquivalentTransformation Configuration, 103
- npu_core_ids
 - LLM Configuration, 92
- num_channel
 - qbcompiler.calibration.preprocess.GetImage,
154
- Open Source License Notice, 48
- optimize_ffn_apply
 - EquivalentTransformation Configuration, 109
- optimize_ffn_ch_per_ffn
 - EquivalentTransformation Configuration, 109
- optimize_option
 - qbcompiler.configs.compile_config.Compile-
Config, 147
- optional
 - AdvancedQuantization Configuration, 94
 - EquivalentTransformation Configuration, 102
 - Input Process Configuration, 132–134
 - LLM Configuration, 89
 - Mod Configuration, 117
 - Optq Configuration, 125
 - qbcompiler.configs.compile_config.Compile-
Config, 145
 - Quantization Configuration, 72, 77, 81
 - Resource Management Configuration, 85
 - SearchWeightScale Configuration, 111
- optq
 - AdvancedQuantization Configuration, 94
- Optq Configuration, 122
 - __init__, 123
 - act_order, 126
 - apply, 125
 - apply_layers, 126
 - block_size, 126
 - default_config, 124
 - DEFAULTS, 125
 - exclude_layers, 126
 - from_dict, 124
 - get_optq_config, 124
 - min_max_check, 125
 - optional, 125
 - perc_damp, 126
 - py_to_cpp_name, 125
 - to_dict, 124
- out
 - SearchWeightScale Configuration, 112
- output_act_bits
 - Quantization Configuration, 79
- output_list
 - Quantization Configuration, 72
- output_weight_bits
 - Quantization Configuration, 80
- pad_val
 - qbcompiler.calibration.preprocess.NpyPad,
168
 - qbcompiler.calibration.preprocess.Pad, 170
- padding_type
 - qbcompiler.calibration.preprocess.NpyPad,
168
- perc_damp
 - Optq Configuration, 126
- percentile
 - Quantization Configuration, 75
- pipeline
 - Input Process Configuration, 134
- post_processing
 - Mod Configuration, 119
- preprocess
 - qbcompiler.calibration.preprocess.pre-
proc_builder, 174
- preprocess_fnc
 - API Reference, 62
- preprocessing
 - Input Process Configuration, 135
- py_to_cpp_name
 - EquivalentTransformation Configuration, 102
 - Input Process Configuration, 132–134
 - LLM Configuration, 90
 - Mod Configuration, 117
 - Optq Configuration, 125
 - qbcompiler.configs.compile_config.Compile-
Config, 145
 - Quantization Configuration, 73, 78, 81
 - Resource Management Configuration, 85
- q_drop
 - Mod Configuration, 119

qbcompiler.calibration.preprocess.CenterCrop,
 140
 __call__, 140
 __init__, 140
 size, 141
 qbcompiler.calibration.preprocess.GetImage, 151
 __call__, 152
 __init__, 152
 channel_order, 152
 COLOR_BGR2BGRA, 154
 COLOR_BGR2GRAY, 153
 COLOR_BGR2RGB, 153
 COLOR_BGR2RGBA, 154
 COLOR_BGRA2BGR, 153
 COLOR_BGRA2GRAY, 153
 COLOR_BGRA2RGB, 153
 COLOR_BGRA2RGBA, 153
 cvtColor, 153
 get_image_data, 152
 num_channel, 154
 to_float32, 152
 qbcompiler.calibration.preprocess.GetNpy, 154
 __call__, 155
 __init__, 155
 qbcompiler.calibration.preprocess.Normalize, 163
 __call__, 164
 __init__, 164
 mean, 164
 std, 164
 to_float_div255, 164
 qbcompiler.calibration.preprocess.NpyNormalize,
 165
 __call__, 166
 __init__, 165
 mean, 166
 std, 166
 to_float_div255, 166
 qbcompiler.calibration.preprocess.NpyPad, 166
 __call__, 167
 __init__, 167
 pad_val, 168
 padding_type, 168
 shape, 168
 slice_padded_data, 167
 qbcompiler.calibration.preprocess.Pad, 169
 __call__, 170
 __init__, 170
 pad_val, 170
 right_bottom, 171
 shape, 170
 size_divisor, 170
 qbcompiler.calibration.preprocess.preproc_builder,
 171
 __call__, 173
 __init__, 173
 __repr__, 173
 __str__, 173
 args, 174
 check_yaml_pre, 174
 datatype, 174
 get_pre_list, 173
 loadtype, 174
 preprocess, 174
 to_tensor, 174
 qbcompiler.calibration.preprocess.Resize, 177
 __call__, 178
 __init__, 178
 img_scale, 178
 interpolation, 178
 keep_ratio, 178
 qbcompiler.calibration.preprocess.ResizeTorch,
 179
 __call__, 179
 __init__, 179
 interpolation, 179
 size, 179
 qbcompiler.calibration.preprocess.SetOrder, 182
 __call__, 182
 __init__, 182
 shape, 183
 qbcompiler.configs.advanced_quantization_con-
 fig.AdvancedQuantizationConfig, 136
 qbcompiler.configs.compile_config.CompileCon-
 fig, 141
 __init__, 142
 _get_calibration_path, 145
 advanced_quantization, 148
 buffer_mode, 147
 calib_data_path, 146
 calib_file, 149
 check_valid_str_configs, 144
 cpu_offload, 147
 default_config, 144
 from_json, 144
 inference_scheme, 147
 inference_scheme_types, 146
 input_process, 148
 input_shape_dict, 148
 llm_config, 148
 min_max_check, 145
 model_nickname, 146
 model_path, 147
 optimize_option, 147
 optional, 145
 py_to_cpp_name, 145
 quantization, 148
 resource_management, 148
 sample_dtype, 147
 sample_dtypes, 146
 save_msgpack_name, 146
 save_path, 146
 save_sample, 147
 to_dict, 145
 version, 148

qbcompiler.configs.equivalent_transformation_config.EquivalentTransformationConfig, 149
 qbcompiler.configs.input_process_config.InputProcessConfig, 156
 qbcompiler.configs.input_process_config.PreprocessingConfig, 175
 qbcompiler.configs.input_process_config.Uint8InputConfig, 183
 qbcompiler.configs.llm_config.LlmConfig, 157
 qbcompiler.configs.mod_config.ModConfig, 158
 qbcompiler.configs.optq_config.OptqConfig, 168
 qbcompiler.configs.quantization_config.BitConfig, 137
 qbcompiler.configs.quantization_config.CalibrationConfig, 138
 qbcompiler.configs.quantization_config.QuantizationConfig, 176
 qbcompiler.configs.resource_management_config.ResourceManagementConfig, 180
 qbcompiler.configs.search_weight_scale_config.SearchWeightScaleConfig, 181
 qbcompiler.frontend.Model_Dict, 160
 __init__, 160
 compile, 162
 qk_apply
 EquivalentTransformation Configuration, 104
 qk_max_gamma
 EquivalentTransformation Configuration, 104
 qk_min_gamma
 EquivalentTransformation Configuration, 104
 qk_rotation_apply
 EquivalentTransformation Configuration, 108
 qk_rotation_matrix_path
 EquivalentTransformation Configuration, 108
 qk_smoothing_factor
 EquivalentTransformation Configuration, 104
 quantization
 qbcompiler.configs.compile_config.CompileConfig, 148
 Quantization Configuration, 64
 __init__, 66, 68, 70
 act_scale_min, 77
 activation_16bits, 81
 bit, 82
 calibration, 82
 default_config, 68–70
 DEFAULTS, 72, 77
 ffn_act_bits, 79
 ffn_weight_bits, 80
 from_dict, 68, 70, 71
 from_kwargs, 68, 70, 71
 get_bit_config, 71
 get_calibration_config, 71
 get_quantization_config, 71
 head_act_bits, 80
 head_weight_bits, 80
 hist_buffer_size, 76
 hist_min_bin_width, 76
 hist_num_bins, 75
 hist_num_samples, 76
 hist_num_search, 76
 hist_percentile, 75
 hist_search_percentile_max, 76
 hist_search_percentile_min, 76
 hist_search_type, 76
 hist_use_gpu, 75
 key_act_bits, 79
 key_weight_bits, 80
 layer_overrides, 77
 load_path, 81
 max_each, 75
 max_total, 75
 method_list, 72
 min_clip_ratio, 77
 min_max_check, 74, 78, 82
 mixed_precision_apply, 81
 mode_list, 72
 optional, 72, 77, 81
 output_act_bits, 79
 output_list, 72
 output_weight_bits, 80
 percentile, 75
 py_to_cpp_name, 73, 78, 81
 quantization_method, 74
 quantization_mode, 74
 quantization_output, 74
 query_act_bits, 79
 query_weight_bits, 80
 save_path, 81
 search_type_list, 72
 to_dict, 68, 70, 71
 topk_ratio, 75
 value_act_bits, 79
 value_weight_bits, 80
 weight_16bits, 81
 weight_scale_min, 77
 quantization_method
 Quantization Configuration, 74
 quantization_mode
 Quantization Configuration, 74
 quantization_output
 Quantization Configuration, 74
 query
 SearchWeightScale Configuration, 112
 query_act_bits
 Quantization Configuration, 79
 query_weight_bits
 Quantization Configuration, 80

 random_seq_length
 LLM Configuration, 91
 recon_coeff
 Mod Configuration, 120
 recon_prob

- Mod Configuration, [120](#)
- Resource Management Configuration, [82](#)
 - `__init__`, [83](#)
 - `default_config`, [84](#)
 - `DEFAULTS`, [85](#)
 - `from_dict`, [84](#)
 - `from_kwargs`, [84](#)
 - `get_resource_management_config`, [84](#)
 - `min_max_check`, [85](#)
 - `optional`, [85](#)
 - `py_to_cpp_name`, [85](#)
 - `to_dict`, [84](#)
 - `use_gpu_only_for_calibration`, [86](#)
 - `weight_dtype`, [86](#)
 - `weight_memory_method`, [86](#)
 - `weight_memory_methods`, [86](#)
- `resource_management`
 - `qbc_compiler.configs.compile_config.Compile-Config`, [148](#)
- `right_bottom`
 - `qbc_compiler.calibration.preprocess.Pad`, [171](#)
- `run_mblt_model`
 - API Reference, [59](#)
- `sample_dtype`
 - `qbc_compiler.configs.compile_config.Compile-Config`, [147](#)
- `sample_dtypes`
 - `qbc_compiler.configs.compile_config.Compile-Config`, [146](#)
- `save_dir`
 - Mod Configuration, [121](#)
- `save_msgpack_name`
 - `qbc_compiler.configs.compile_config.Compile-Config`, [146](#)
- `save_path`
 - `qbc_compiler.configs.compile_config.Compile-Config`, [146](#)
 - Quantization Configuration, [81](#)
- `save_sample`
 - `qbc_compiler.configs.compile_config.Compile-Config`, [147](#)
- `search_type_list`
 - Quantization Configuration, [72](#)
- `search_weight_scale`
 - AdvancedQuantization Configuration, [95](#)
- SearchWeightScale Configuration, [109](#)
 - `__init__`, [110](#)
 - `apply`, [112](#)
 - `default_config`, [110](#)
 - `DEFAULTS`, [111](#)
 - `ffn`, [112](#)
 - `from_dict`, [111](#)
 - `get_search_weight_scale_config`, [111](#)
 - `key`, [112](#)
 - `optional`, [111](#)
 - `out`, [112](#)
 - `query`, [112](#)
 - `to_dict`, [111](#)
 - `value`, [112](#)
- `seed`
 - EquivalentTransformation Configuration, [103](#)
 - Mod Configuration, [121](#)
- `shape`
 - `qbc_compiler.calibration.preprocess.NpyPad`, [168](#)
 - `qbc_compiler.calibration.preprocess.Pad`, [170](#)
 - `qbc_compiler.calibration.preprocess.SetOrder`, [183](#)
- `size`
 - `qbc_compiler.calibration.preprocess.Center-Crop`, [141](#)
 - `qbc_compiler.calibration.preprocess.Resize-Torch`, [179](#)
- `size_divisor`
 - `qbc_compiler.calibration.preprocess.Pad`, [170](#)
- `slice_padded_data`
 - `qbc_compiler.calibration.preprocess.NpyPad`, [167](#)
- `spin_r1_apply`
 - EquivalentTransformation Configuration, [106](#)
- `spin_r1_matrix_path`
 - EquivalentTransformation Configuration, [107](#)
- `spin_r2_apply`
 - EquivalentTransformation Configuration, [107](#)
- `spin_r2_learn`
 - EquivalentTransformation Configuration, [107](#)
- `spin_r2_matrix_path`
 - EquivalentTransformation Configuration, [107](#)
- `std`
 - `qbc_compiler.calibration.preprocess.Normalize`, [164](#)
 - `qbc_compiler.calibration.preprocess.NpyNormalize`, [166](#)
- `to_dict`
 - AdvancedQuantization Configuration, [94](#)
 - EquivalentTransformation Configuration, [102](#)
 - Input Process Configuration, [129–131](#)
 - LLM Configuration, [89](#)
 - Mod Configuration, [116](#)
 - Optq Configuration, [124](#)
 - `qbc_compiler.configs.compile_config.Compile-Config`, [145](#)
 - Quantization Configuration, [68, 70, 71](#)
 - Resource Management Configuration, [84](#)
 - SearchWeightScale Configuration, [111](#)
- `to_float32`
 - `qbc_compiler.calibration.preprocess.GetImage`, [152](#)
- `to_float_div255`
 - `qbc_compiler.calibration.preprocess.Normalize`, [164](#)

- qbcompiler.calibration.preprocess.NpyNormalize, [166](#)
- to_tensor
 - qbcompiler.calibration.preprocess.preproc_builder, [174](#)
- TokenizeToEmbedding, [183](#)
- topk_ratio
 - Quantization Configuration, [75](#)
- TorchVision의 이미지 분류 모델, [11](#)
- ud_apply
 - EquivalentTransformation Configuration, [105](#)
- ud_learn
 - EquivalentTransformation Configuration, [105](#)
- ud_max_gamma
 - EquivalentTransformation Configuration, [105](#)
- ud_min_gamma
 - EquivalentTransformation Configuration, [105](#)
- ud_smoothing_factor
 - EquivalentTransformation Configuration, [105](#)
- uint8_input
 - Input Process Configuration, [135](#)
- Ultralytics의 YOLO 모델, [15](#)
- use_full_seq_length
 - LLM Configuration, [91](#)
- use_global_core
 - LLM Configuration, [91](#)
- use_gpu_only_for_calibration
 - Resource Management Configuration, [86](#)
- use_outputs
 - Mod Configuration, [120](#)
- use_xyxy
 - Mod Configuration, [121](#)
- value
 - SearchWeightScale Configuration, [112](#)
- value_act_bits
 - Quantization Configuration, [79](#)
- value_weight_bits
 - Quantization Configuration, [80](#)
- version
 - qbcompiler.configs.compile_config.CompileConfig, [148](#)
- vo_apply
 - EquivalentTransformation Configuration, [105](#)
- vo_max_gamma
 - EquivalentTransformation Configuration, [106](#)
- vo_min_gamma
 - EquivalentTransformation Configuration, [106](#)
- vo_smoothing_factor
 - EquivalentTransformation Configuration, [106](#)
- warmup_epochs
 - Mod Configuration, [118](#)
- weight_16bits
 - Quantization Configuration, [81](#)
- weight_dtype
 - Resource Management Configuration, [86](#)
- weight_memory_method
 - Resource Management Configuration, [86](#)
- weight_memory_methods
 - Resource Management Configuration, [86](#)
- weight_scale_min
 - Quantization Configuration, [77](#)
- YoloPre, [184](#)
- 빠른 시작, [4](#)
- 설치, [2](#)
- 소개, [1](#)
- 지원 프레임워크, [25](#)
- 튜토리얼 개요, [10](#)



Mobilint, Inc.

3F, Narakium Yeoksam B Building,
35 Seolleung-ro 93-gil, Gangnam-
gu, Seoul, Republic of Korea (06151)

Corporate URL

www.mobilint.com

Contact

tech-support@mobilint.com

+82 2 6281 0489